强化学习的数学原理

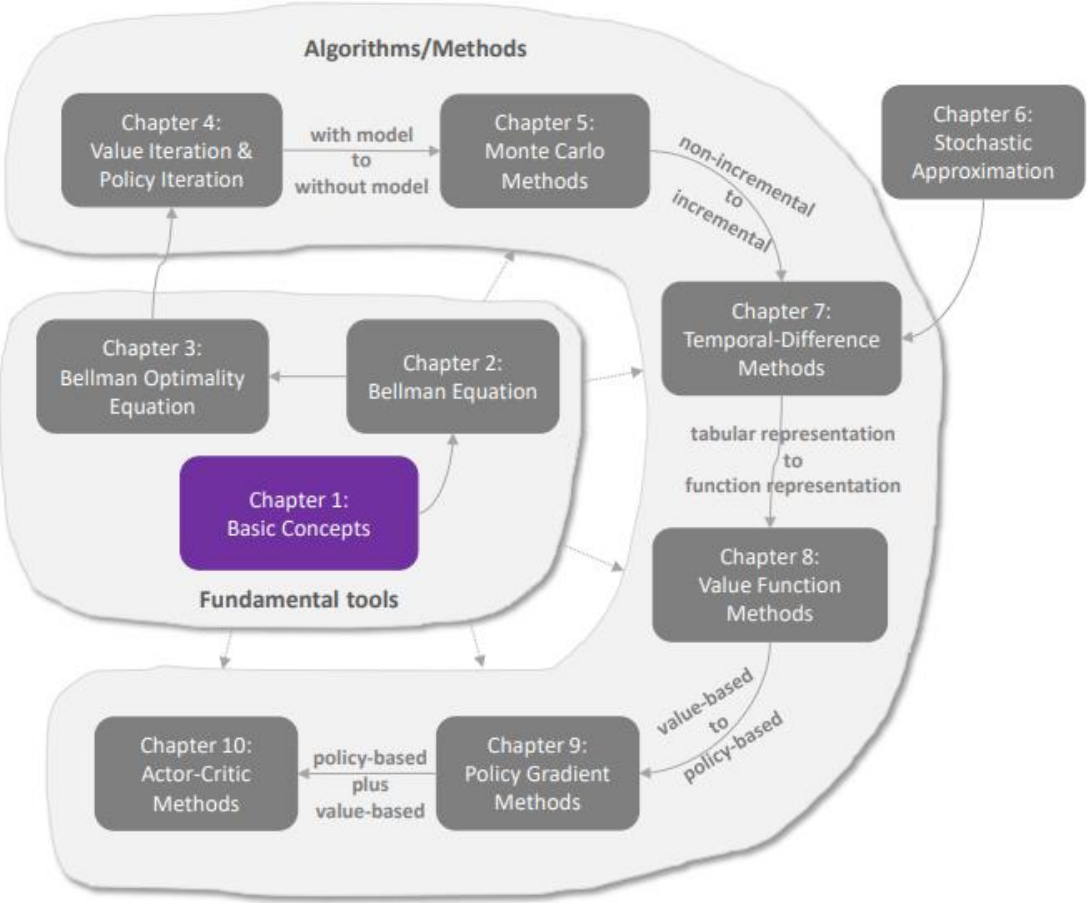
By Li Wei

目录

Chapter0 整体关系图.....4
Chapter1 强化学习的基本概念.....5
1.1 状态、动作、策略.....5
1.2 奖励、回报、MDP.....8
Chapter2 贝尔曼方程.....12
2.1 回报的重要性.....12
2.2 状态价值.....13
2.3 贝尔曼方程推导.....14
2.4 贝尔曼方程求解.....16
2.5 动作价值.....17
Chapter3 贝尔曼最优公式.....18
3.1 策略改进.....18
3.2 最优策略及公式推导.....19
3.3 公式求解及最优性.....21
3.4 最优策略性质.....22
Chapter4 值迭代与策略迭代.....24
4.1 值迭代算法.....24
4.2 策略迭代算法.....26
4.3 截断策略迭代算法.....30
Chapter5 蒙特卡洛方法.....32
5.1 蒙特卡洛引入.....32
5.2 MC-basic 算法.....32
5.3 MC-basic 算法用例.....34
5.4 MC-exploring starts 算法.....36
5.5 MC-epsilon greedy 算法.....37
5.6 MC-epsilon greedy 算法用例.....39
Chapter6 随机近似与随机梯度下降.....41
6.1 均值估计问题回顾.....41
6.2 RM 算法介绍.....42
6.3 RM 算法收敛性及应用.....43
6.4 SGD 算法介绍.....45
6.5 SGD 算法收敛性及应用.....46
6.6 SGD 算法有趣的性质.....47
6.7 SGD 算法对比.....49

Chapter7 时序差分方法.....49
7.1 时序差分引入.....49
7.2 时序差分算法.....50
7.3 时序差分算法收敛性.....52
7.4 Sarsa 算法.....53
7.5 Expected Sarsa 和 n-step Sarsa 算法.....55
7.6 Q-learning 算法.....56
7.7 Q-learning 例子.....58
7.8 TD 算法的统一形式.....59
Chapter8 值函数近似.....60
8.1 曲线拟合.....60
8.2 目标函数.....61
8.3 优化算法.....62
8.4 示例与分析.....64
8.5 Sarsa 和 Q-learning.....67
8.6 DQN-basic.....68
8.7 DQN-Experience.....69
8.8 DQN 代码与例子.....70
Chapter9 策略梯度方法.....72
9.1 基本思路.....72
9.2 价值目标函数.....73
9.3 奖励目标函数.....74
9.4 梯度计算.....75
9.5 梯度上升算法.....77
Chapter10 Actor-Critic 方法.....78
10.1 QAC 算法.....78
10.2 A2C 算法.....79
10.3 重要性采样与 Off-Policy.....82
10.4 DPG 算法.....85

Chapter0 整体关系图



Chapter1：介绍基础工具、概念和算法。

Chapter2：介绍状态价值、贝尔曼方程、策略评估。

Chapter3：介绍一个特殊的贝尔曼方程、最优策略、最优状态价值和贝尔曼最优方程（不动点迭代、基本问题、方程解法）。

Chapter4：介绍一个优化策略的算法、值迭代算法、策略迭代算法、截断策略迭代。策略更新与值更新。

Chapter5：介绍如何无模型进行学习。通过数据采样进行期望估计。第一个无模型的 RL 算法，包括基本 MC、探索 MC 和贪婪 MC。

Chapter6：介绍 RM 算法、随机梯度下降（SGD）、BGD、MBGD。

Chapter7：介绍 RL 算法分类：基于 TD 误差的状态价值学习，基于 TD 误差的动作价值学习（Sarsa），基于 TD 误差的最优动作价值学习（Q-learning）。On-policy 和 Off-plocy。

Chapter8：介绍基于值函数近似的状态价值期望。引入神经网络到 RL 体系。

Chapter9：介绍从基于值到基于策略。定义最优策略和介绍梯度下降算法。

Chapter10：介绍基于值和基于策略的算法：QAC、A2C 和 DPG。

Chapter1 强化学习的基本概念

1.1 状态、动作、策略

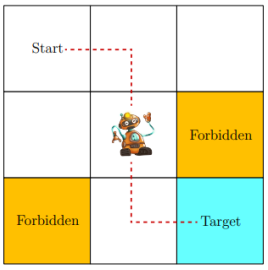


图 1.1.1 网格世界

本文所有的例子均建立在以上的网格世界中，其遵循以下规则：

- 网格单元：包括可行性、不可通行、终点网格和边界。
- 机器人在网格世界中的行为很容易被理解并且在例子中非常形象。

在网格世界中我们的任务是：

- 任意选择一个初始网格单元，找到一个可以到达终点的最优路径。
- 怎么定义“最优”的概念，如何避免进入不可通行和边界？

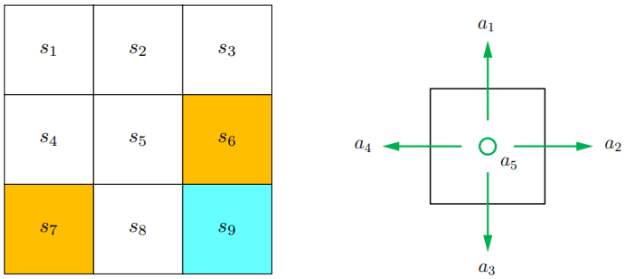


图 1.1.2 状态和动作

状态（State）：智能体与环境之间的关系状态。在以上的网格世界中，机器人当前所在环境中的位置是机器人当前的状态，环境中机器人可以在 9 个不同的位置上，因此机器人具有 9 种不同的状态：S_1,……,S_9。

状态空间（State space）：所有状态的集合为状态空间 $\boldsymbol{S}=\{S_i\}$。

动作（Action）：对于网格中每一个智能体状态，智能体可以执行 5 个不同的动作a_1,….,a_5，分别代表向左、向右、向下、向上和原地不动。

状态的动作空间（Action space of a state）：对于一个状态，它能够执行所有动作的集合 A(S)=$\{a_i\}$。

问题：不同的状态能否有不同的动作集合呢？

答：当然是可以的。状态和动作均由人主观设置，对于边界上的网格，我们可以不定义进入边界的动作，也可以定义若下一步是网格，则选择原地动作，或者选择反弹动作等等。

状态转移（State transition）：当智能体采取了一个动作后，智能体就会由一

个网格进入另一个网格，智能体就会从当前状态进入另一个状态，这个过程叫做状态转移。

- 在状态 S_1 时，如果采取动作a_2，下一个状态会变成什么？

$$s_1 \xrightarrow{a_2} s_2 \tag{1}$$

- 在状态 S_1 时，如果采取动作a_1，下一个状态会变成什么？

$$s_1 \xrightarrow{a_1} s_1 \tag{2}$$

状态转移定义了智能体如何与环境之间交互。

问题：我们能否通过其他方式定义状态转移？

回答：上面的例子是采取一个动作，会转移到确定的状态，当然在某些场景下，机器人有概率落入其他的状态，比如受到其他因素的干扰。

特别关注一下不可通行区域，在状态 S_5 时，如果我们采取动作a_2，下一个状态是什么？

- 情况 1：不可通行区域可以进入，但是它会返回惩罚。
- 情况 2：不可以通行区域不可以进入。

一般情况下，我们考虑第一种情况，因为这种情况更具有普遍性，同时也保留了机器人的探索行为，更符合我们的试错特性。

表 1.1.1 表格表示状态转移

	a_1 (upward)	a_2 (rightward)	a_3 (downward)	a_4 (leftward)	a_5 (still)
s_1	s_1	s_2	s_4	s_1	s_1
s_2	s_2	s_3	s_5	s_1	s_2
s_3	s_3	s_3	s_6	s_2	s_3
s_4	s_1	s_5	s_7	s_4	s_4
s_5	s_2	s_6	s_8	s_4	s_5
s_6	s_3	s_6	s_9	s_5	s_6
s_7	s_4	s_8	s_7	s_7	s_7
s_8	s_5	s_9	s_8	s_7	s_8
s_9	s_6	s_9	s_9	s_8	s_9

状态转移概率（State transition probability）：从一个状态跳转到下一个不唯一的状态，跳转的过程为概率选择，这个概率交状态转移概率。

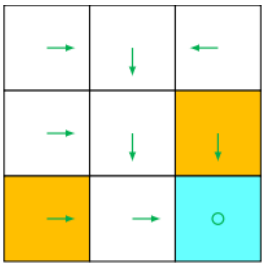
- 例如，在状态 S_1 时，如果我们选择动作a_2，跳转到下一个状态为 S_2。
- 用概率的形式表示这个状态转移过程：

$$\begin{aligned} p(s_2 \mid s_1, a_2) &= 1 \\ p(s_i \mid s_1, a_2) &= 0 \end{aligned} \tag{3}$$

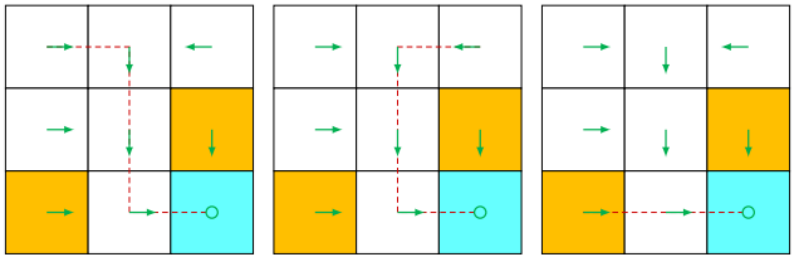
这里的例子是一个确定情况，一个状态只能转换到确定的下一个状态。实际上的状态转移是依概率随机的，机器人可以有各自一般的概率选择左右两个动作，从而跳转到两个不同的状态，也可以更多。

策略（Policy）：策略代表智能体在当前状态采取什么动作。

在下面的例子中，箭头方向代表机器人处于当前状态选择的动作。我们通过从不同的起点出发，通过不同的轨迹到达终点。



（a）一个确定的策略



（b）在上面策略中机器人的多个轨迹

图 1.1.3 策略与轨迹

我们可以用数学公式描述我们所定义的这个策略。首先，定义策略符号为 $\pi(a|s)$，它代表在状态 S 采取动作 a 的条件概率分布，在上面的例子中，对于状态 S_1，策略的数学公式是：

$$\begin{aligned}\pi(a_2 \mid s_1) &= 1 \\ \pi(a_i \mid s_1) &= 0\end{aligned} \tag{4}$$

$\pi(a|s)$是一个确定的策略。下面是一个随机策略的例子。

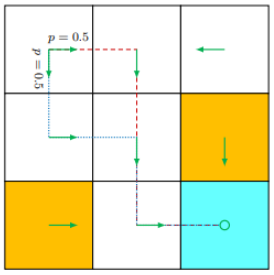


图 1.1.4 随机策略

在这个环境中，对于状态 S_1 的策略是：

$$\begin{aligned}&\pi(a_2 \mid s_1) = 0.5,\ \pi(a_3 \mid s_1) = 0.5 \\ &\pi(a_i \mid s_1) = 0\end{aligned} \tag{5}$$

我们用一个表格表示上面的随机策略：

表 1.1.2 表格表示策略

	a_1 (upward)	a_2 (rightward)	a_3 (downward)	a_4 (leftward)	a_5 (still)
s_1	0	0.5	0.5	0	0
s_2	0	0	1	0	0
s_3	0	0	0	1	0
s_4	0	1	0	0	0
s_5	0	0	1	0	0
s_6	0	0	1	0	0
s_7	0	1	0	0	0
s_8	0	1	0	0	0
s_9	0	0	0	0	1

1.2 奖励、回报、MDP

奖励（Reward）：当我们采取一个动作后，环境会反馈给智能体一个数值。这个值一般情况下代表我们对这个动作的好坏程度的主观评价。

- 一个正的奖励值代表我们鼓励采取此类动作。
- 一个负的奖励值代表我们惩罚采取此类动作。

问题：奖励值为 0 代表什么？

回答：代表此类行为不惩罚和不鼓励。

问题：一个正的奖励值可以代表惩罚吗？

回答：当然可以。奖励与惩罚值是人主观定义的。

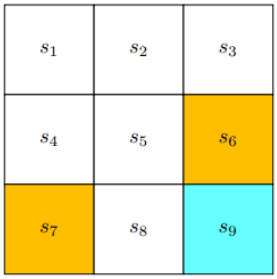


图 1.2.1 网格世界

在网格世界的例子中，我们可以设计如下的奖励规则：

- 如果智能体尝试跳出边界的动作，给予奖励值为-1。
- 如果智能体尝试进入障碍物网格，给予奖励值为-1。
- 如果智能体到达终点，给予奖励值为+1。
- 其他情况，给予奖励值为 0。

奖励本质上可以理解为人与机器的交互指南，它代表着人类的评价去指导着智能体做一件人类期望它做的事。例如，在上面的规则设定下，智能体在学习中会避免跳出边界或进入障碍物网格。

表 1.2.1 表格表示奖励

	a_1 (upward)	a_2 (rightward)	a_3 (downward)	a_4 (leftward)	a_5 (still)
s_1	r_{boundary}	0	0	r_{boundary}	0
s_2	r_{boundary}	0	0	0	0
s_3	r_{boundary}	r_{boundary}	$r_{\text{forbidden}}$	0	0
s_4	0	0	$r_{\text{forbidden}}$	r_{boundary}	0
s_5	0	$r_{\text{forbidden}}$	0	0	0
s_6	0	r_{boundary}	r_{target}	0	$r_{\text{forbidden}}$
s_7	0	0	r_{boundary}	r_{boundary}	$r_{\text{forbidden}}$
s_8	0	r_{target}	r_{boundary}	$r_{\text{forbidden}}$	0
s_9	$r_{\text{forbidden}}$	r_{boundary}	r_{boundary}	0	r_{target}

可以用条件概率描述奖励值。

- 例如，对于状态 S_1，采取动作a_1，得到奖励值为-1。
- 用概率形式表达为：$p(r=-1|s_1,a_1)=1, p(r\neq -1|s_1,a_1)=0$。

这是一个得到奖励值确定的情况。但实际中，奖励值转移可以是随机的，例如你学习努力，就会得到奖励，但具体数值是多少，这个是不确定的。

轨迹（Trajectory）:一连串的状态-动作-奖励值序列。

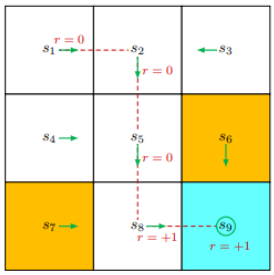


图 1.2.2 策略 1 的轨迹

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} s_8 \xrightarrow{a_2} s_9 \tag{6}$$

回报（Return）：一条轨迹上获得所有奖励值的总和。

$$\text{return} = 0+0+0+1=1 \tag{7}$$

同一个环境中，不同的策略会得到不同的轨迹：

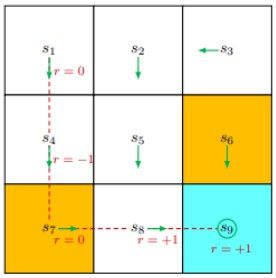


图 1.2.3 策略 2 的轨迹

$$s_1 \xrightarrow{a_3} s_4 \xrightarrow{a_3} s_7 \xrightarrow{a_2} s_8 \xrightarrow{a_2} s_9 \tag{8}$$

$$\text{return} = 0-1+0+1=0 \tag{9}$$

问题：如何判断那个策略更好？

回答：从直观上看，第一个策略更好，因为它没有触碰到障碍物。从数学

上来看，第一个策略更好，因为它有更大的回报。因此，回报能够用来评估一个策略的好坏。

一条轨迹可能是无限的：

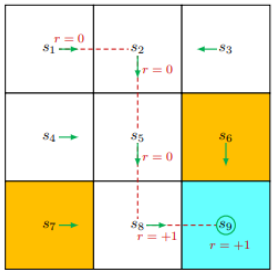


图 1.2.4 无限长的轨迹

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} s_8 \xrightarrow{a_2} s_9 \xrightarrow{a_5} s_9 \xrightarrow{a_5} s_9 \ldots \tag{10}$$

其回报为：

$$\text{return} = 0+0+0+1+1+1+\cdots = \infty \tag{11}$$

可以发现这个回报是无穷发散的，无法进行具体分析，但现实中，这种轨迹在可以存在，因此我们需要进行一些处理来使这种情况普遍化，引入了折扣因子（Discount rate）。

引入折扣因子后的回报（Discount return）：

$$\text{discounted return} = 0+\gamma 0+\gamma^2 0+\gamma^3 1+\gamma^4 1+\gamma^5 1+\ldots \tag{12}$$

我们发现当 $\gamma \in [0,1)$时，discount return 是收敛的，计算出来为：

$$\text{discounted return} = \gamma^3(1+\gamma+\gamma^2+\ldots) = \gamma^3 \frac{1}{1-\gamma} \tag{13}$$

从式 13 可以看出折扣因子引入后的效果：1）回报不在发散，而是收敛。2）通过折扣因子平衡了近期奖励和未来趋势。3）如果 γ 接近 0，说明越看重近期奖励，整个回报受到近期奖励值的支配。4）如果 γ 接近 1，说明越看重未来趋势，整个回报受到未来趋势的支配。

实际上，我们不会使交互无限进行下去，所以在交互中往往会设置一些终点状态，当智能体到达终点后就结束交互，这个过程被称为一个轮次（Episode）。

例如：

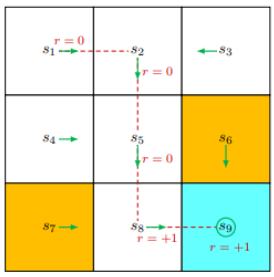


图 1.2.5 一个轮次

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} s_8 \xrightarrow{a_2} s_9 \tag{14}$$

一个轮次通常是有限的，回合中需要完成的任务成为回合任务（Episode tasks）。一些回合任务可能没有终点状态，意味着智能体与环境的交互永远不会停止，这些任务成为连续任务（Continuing tasks）。

问题：在网格世界的例子中，我们是否需要在到达终点后停止交互呢？

回答：事实上，我们可以将事件性任务转换为轮次或连续性任务来统一数学表达。1）将终点状态视作一种特殊的吸收状态，一旦智能体到达吸收状态，它将永远不会离开，但是奖励值一直为 0。2）智能体可以离开终点状态，但是当其再次进入终点状态时仍然获得+1 奖励值。通常情况下，我们会选择第二种情况，这样不需要额外增加新的状态。

马尔科夫决策过程（Markov decision processes，MDP）是描述随机动力系统的一般框架。下面列出了 MDP 的关键成分。

序列（Set）：包括状态空间 $\boldsymbol{S}$、动作空间 $\boldsymbol{A}(s)$由状态 $s \in \boldsymbol{S}$ 支配、奖励 R(s,a)。

概率分布（Probability distribution）：包括状态转移概率（在状态 s 采取动作 a跳转到状态s'的概率为$p(s'|s,a)$，对于任何的(s,a)组都满足$\sum_{s' \in S} p(s'|s,a) = 1$）、奖励转移概率（状态 s 采取动作a 得到奖励 r 的概率为$p(r|s,a)$，对于任何的(s,a)组都满足$\sum_{r \in R(s,a)} P(r|s,a) = 1$）。

策略（Policy）：在状态 s 采取动作a的概率为$\pi(a|s)$，对于任何的(s,a)组都满足$\sum_{a \in A(s)} \pi(s,a) = 1$。

马尔科夫属性（Markov property）：指的是马尔科夫随机过程的无记忆性，下一个状态只有当前状态决定，与历史状态无关。

$$\begin{aligned} p(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) &= p(s_{t+1} \mid s_t, a_t) \\ p(r_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) &= p(r_{t+1} \mid s_t, a_t) \end{aligned} \tag{15}$$

其中，t 表示当前时刻，t+1 表示下一个时间时刻。

于是，网格世界能够用一个更通用的模型来抽象表达——马尔科夫过程。

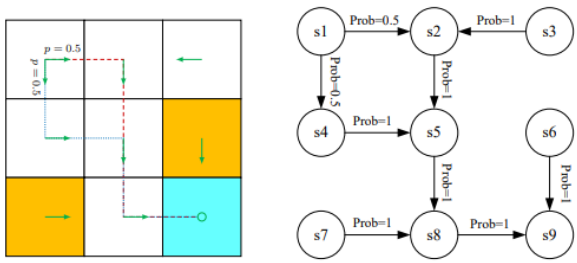


图 1.2.6 MDP 过程

其中的圆圈代表状态，箭头代表状态转移方向。一旦策略给定，马尔科夫决策过程就变为了确定的马尔科夫链。

Chapter2 贝尔曼方程

2.1 回报的重要性

借用动力学的例子来阐述为什么 return 是非常的重要的。

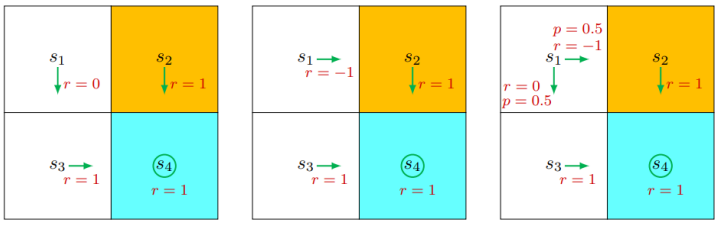


图 2.1.1 三种策略

设定 S_1 是智能体的起点，S_4 是终点，设定路径规划任务为智能体越快到达终点越好，从直观上可以明显的看出：

- 策略 1 是最好的，因为它无论在何处，都可以直接一步达到终点。
- 策略 2 是最差的，因为它进入了障碍物区域，这是任务中不希望看到的。

进而，我们需要用数学关系来表达这种好与坏，return 就是一个非常适合评价策略好坏的指标。

在策略 1 中，智能体从起点 S_1 出发，跳转到 S_3，然后再跳转到终点，按照 discount return 的定义，其值为：

$$\text{return}_1 = 0 + \gamma 1 + \gamma^2 1 + \ldots = \gamma(1 + \gamma + \gamma^2 + \ldots) = \frac{\gamma}{1-\gamma} \tag{16}$$

在策略 2 中，智能体从起点 S_1 出发，跳转到 S_2，然后再跳转到终点，按照 discount return 的定义，其值为：

$$\text{return}_2 = -1 + \gamma 1 + \gamma^2 1 + \ldots = -1 + \gamma(1 + \gamma + \gamma^2 + \ldots) = -1 + \frac{\gamma}{1-\gamma} \tag{17}$$

在策略 3 中，智能体从起点 S_1 出发，有 50%概率到 S_2，50%概率到 S_3，因此轨迹有两条，按照 discount return 的定义，其值为：

$$\text{return}_3 = 0.5\left(-1 + \frac{\gamma}{1-\gamma}\right) + 0.5\left(\frac{\gamma}{1-\gamma}\right) = -0.5 + \frac{\gamma}{1-\gamma} \tag{18}$$

有大小关系：

$$\text{return}_1 > \text{return}_3 > \text{return}_2 \tag{19}$$

上述关于 return 的不等式告诉我们，策略 1 是最好的，策略 3 其次，策略 2 最差。这与我们直观分析是一致的。

既然 return 是如此的重要，那么我们需要有一个更普适的 return 计算的公式，下面给出 2 种计算方法。

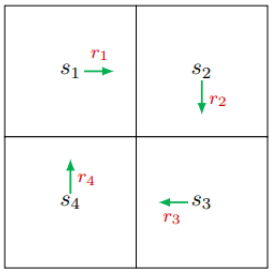


图 2.1.2 策略与奖励值

法 1：定义 v_i 代表从状态 S_i 开始的所有轨迹的 return(i=1,2,3,4)。

$$\begin{aligned} v_1 &= r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots, \\ v_2 &= r_2 + \gamma r_3 + \gamma^2 r_4 + \ldots, \\ v_3 &= r_3 + \gamma r_4 + \gamma^2 r_1 + \ldots, \\ v_4 &= r_4 + \gamma r_1 + \gamma^2 r_2 + \ldots. \end{aligned} \tag{20}$$

法 2：注意到存在嵌套关系，将上式转换为：

$$\begin{aligned} v_1 &= r_1 + \gamma r_2 + \gamma^2 r_3 + \ldots = r_1 + \gamma v_2 \\ v_2 &= r_2 + \gamma r_3 + \gamma^2 r_4 + \ldots = r_2 + \gamma v_3 \\ v_3 &= r_3 + \gamma r_4 + \gamma^2 r_1 + \ldots = r_3 + \gamma v_4 \\ v_4 &= r_4 + \gamma r_1 + \gamma^2 r_2 + \ldots = r_4 + \gamma v_1 \end{aligned} \tag{21}$$

这种方法在强化学习中称为自举（bootstrapping）。现在的问题是如何求解这个方程组，我们将其写成矩阵向量形式：

$$\underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}}_{v} = \begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix} + \begin{bmatrix} \gamma v_2 \\ \gamma v_3 \\ \gamma v_4 \\ \gamma v_1 \end{bmatrix} = \underbrace{\begin{bmatrix} r_1 \\ r_2 \\ r_3 \\ r_4 \end{bmatrix}}_{r} + \gamma \underbrace{\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}}_{p} \underbrace{\begin{bmatrix} v_1 \\ v_2 \\ v_3 \\ v_4 \end{bmatrix}}_{v} \tag{22}$$

值得注意的是，其中的矩阵 p 代表了每个状态之间的转换关系，也称为状态转移矩阵，继续将式 22 写成更简洁的形式：

$$v = r + \gamma P v \tag{23}$$

显然，只有 v 是未知矩阵，将其移到左边，求出逆矩阵即可求解，但在矩阵维度很高时，这个矩阵的逆矩阵很难计算，所以我们一般采用不动点迭代的方法进行迭代近似求解。

同时，这还是一个贝尔曼方程（在一个特殊的确定的例子中）。通过这个例子，我们也发现了一个重要的现象，一个状态的价值是依赖于其他状态的价值的。

2.2 状态价值

思考一个单步的过程：

$$S_t \xrightarrow{A_t} S_{t+1}, R_{t+1} \tag{24}$$

- t,t+1：离散时间点。
- S_t：智能体在时间 t 时的状态。
- A_t：在状态时，智能体采取的动作。
- S_{t+1}:智能体在时间 t 时采取动作 A_t 跳转到的下一状态。

这个单步过程受到以下概率分布的支配：

- $S_t \rightarrow A_t$ 受到$\pi(A_t = a|S_t = s)$的支配。
- S_t,$A_t \rightarrow R_{t+1}$ 受到$p(R_{t+1} = r|S_t = s, A_t = a)$的支配。
- S_t,$A_t \rightarrow S_{t+1}$ 受到$p(S_{t+1} = s'|S_t = s, A_t = a)$的支配。

现在，假设我们已经知道了模型（也就是以上所有概率分布）。考虑一个多步过程的轨迹：

$$S_t \xrightarrow{A_t} S_{t+1}, R_{t+1} \xrightarrow{A_{t+1}} S_{t+2}, R_{t+2} \xrightarrow{A_{t+2}} S_{t+3}, R_{t+3} \ldots \tag{25}$$

计算其 discount return 为：

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots \tag{26}$$

显然 G_t 为随机变量。

我们定义 G_t 的期望叫做状态价值函数或状态价值（State value）：

$$v_\pi(s) \doteq \mathbb{E}[G_t \mid S_t = s] \tag{27}$$

- 状态价值是关于状态 s 的函数，它是一个条件期望值，条件是智能体状态从 s 开始。
- 它依赖于策略 π，不同的策略下每个状态的状态价值不同。
- 它代表一个状态的“值”，如果当前策略下状态价值更高，代表当前策略要比其他策略更好，能够得到的奖励值更高。

问题：return 与状态价值的关系？

回答：状态价值是当前策略下从当前状态出发所有能被观测到的可能轨迹的 return 的期望值，如果模型已知，return 值就是状态价值，因为能够直接计算出（这里的直接计算指的是若策略确定，那么一步就可以计算出，若策略随机，则利用求期望的计算公式可计算出），如果模型未知，那么状态价值就是通过采样样本的期望值来近似总体 return 的期望，这是后面 MC 方法的思想。

2.3 贝尔曼方程推导

既然状态价值是非常的重要，那么如何去进行计算呢？方法就是贝尔曼方程。一句话来说，贝尔曼方程描述了所有状态价值之间的关系。接下来，我们将推导贝尔曼方程。

考虑一个随机的轨迹：

$$S_t \xrightarrow{A_t} S_{t+1}, R_{t+1} \xrightarrow{A_{t+1}} S_{t+2}, R_{t+2} \xrightarrow{A_{t+2}} S_{t+3}, R_{t+3} \ldots \tag{28}$$

其 G_t 计算如下：

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \ldots) = R_{t+1} + \gamma G_{t+1} \tag{29}$$

然后，根据状态价值的定义公式，带入上述式子：

$$\begin{aligned} v_\pi(s) &= \mathbb{E}[G_t \mid S_t = s] \\ &= \mathbb{E}[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \end{aligned} \tag{30}$$

也就是求解在状态为 s 的情况下，获得的立即奖励值和下一个状态折扣回报的期望值。由期望的叠加性可知，随机变量和的期望，等于随机变量期望之和：

$$v_\pi(s) = \mathbb{E}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}[G_{t+1} \mid S_t = s] \tag{31}$$

接下来，需要求解两部分各自的期望值。

（1）在状态为 s 的情况下，奖励值 R 的期望是，每一个动作的奖励值期望值×该动作的执行概率，全部求和。也就是每一个动作采取该动作获得对应奖励值的概率×奖励值×采取该动作的概率，全部求和。

$$\begin{aligned} \mathbb{E}[R_{t+1} \mid S_t = s] &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a] \\ &= \sum_{a \in \mathcal{A}} \pi(a \mid s) \sum_{r \in \mathcal{R}} p(r \mid s, a) r \end{aligned} \tag{32}$$

可以看出，第一部分仅仅计算了当前状态采取一个动作立即获得的奖励值的期望，也称一步奖励值。

（2）在状态为 s 的情况下，下一个状态的 G_t 的期望是，首先需要确定下一个状态是什么，然后求其期望，是每一个可能的下个状态的 G_t 的期望×该状态转移的概率。然后，由于马尔科夫过程的无记忆性，下一个状态的 G_t 的期望和历史状态无关。然后，对应下一个状态的 G_t 的期望，也就是下一个状态的状态价值。然后，从状态 s 到下一个状态 s' 的概率是，在当前状态 s 采取动作 a 跳转到状态 s' 的概率×采取该动作 a 的概率，全部求和。

最后，在状态为 s 的情况下，下一个状态的 G_t 的期望是，在当前状态 s 采取动作 a 的概率×采取该动作 a 跳转到 s' 的概率，全部求和，求得达到状态 s' 的概率，然后×该状态 s' 的状态价值，最后将所有可能的下一个状态都依次计算完即可。

$$\begin{aligned} \mathbb{E}[G_{t+1} \mid S_t = s] &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1} \mid S_t = s, S_{t+1} = s'] p(s' \mid s) \\ &= \sum_{s' \in \mathcal{S}} \mathbb{E}[G_{t+1} \mid S_{t+1} = s'] p(s' \mid s) \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s') p(s' \mid s) \\ &= \sum_{s' \in \mathcal{S}} v_\pi(s') \sum_{a \in \mathcal{A}} p(s' \mid s, a) \pi(a \mid s) \end{aligned} \tag{33}$$

可以看出，第二个部分考虑了跳转到下一个状态未来可能获得所有奖励值的

期望。

因此，可以得到：

$$
\begin{aligned}
v_\pi(s) &= \mathbb{E}[R_{t+1} \mid S_t = s] + \gamma \mathbb{E}[G_{t+1} \mid S_t = s] \\
&= \underbrace{\sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{r\in\mathcal{R}} p(r \mid s,a) r}_{\text{立即奖励期望}} + \gamma \underbrace{\sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{s'\in\mathcal{S}} p(s' \mid s,a) v_\pi(s')}_{\text{未来奖励期望}} \\
&= \sum_{a\in\mathcal{A}} \pi(a \mid s) \left[\sum_{r\in\mathcal{R}} p(r \mid s,a) r + \gamma \sum_{s'\in\mathcal{S}} p(s' \mid s,a) v_\pi(s') \right]
\end{aligned} \tag{34}
$$

上面这个方程叫贝尔曼方程，它描述了状态价值与不同状态之间的关系。它包含两个部分，立即奖励部分和未来奖励部分，所有的状态都具有这样的式子。

2.4 贝尔曼方程求解

从上面的贝尔曼方程可以看出，求解的未知变量依赖于另一个未知变量；上面的方程对于任何一个状态 s 都是有效的，所以可以列出 n 个类似的方程（n 是状态空间的大小）；将所有方程组合在一起，我们就得到一个线性方程组，因此就可以写成矩阵的形式，继而可以求解。

对于式 34，我们将其写成：

$$
v_\pi(s) = r_\pi(s) + \gamma \sum_{s'\in\mathcal{S}} p_\pi(s' \mid s) v_\pi(s') \tag{35}
$$

其中：

$$
\begin{aligned}
r_\pi(s) &\doteq \sum_{a\in\mathcal{A}} \pi(a \mid s) \sum_{r\in\mathcal{R}} p(r \mid s,a) r \\
p_\pi(s' \mid s) &\doteq \sum_{a\in\mathcal{A}} \pi(a \mid s) p(s' \mid s,a)
\end{aligned} \tag{36}
$$

设定 s_i 代表每个状态。对于每个状态 s_i，都有如下贝尔曼方程：

$$
v_\pi(s_i) = r_\pi(s_i) + \gamma \sum_{s_j\in\mathcal{S}} p_\pi(s_j \mid s_i) v_\pi(s_j) \tag{37}
$$

将所有状态 s_i 的贝尔曼方程列在一起，写成矩阵形式，可得：

$$
v_\pi = r_\pi + \gamma P_\pi v_\pi \tag{38}
$$

- $v_\pi = [v_\pi(s_1), \dots, v_\pi(s_n)]^T \in \mathbb{R}^n$。
- $r_\pi = [r_\pi(s_1), \dots, r_\pi(s_n)]^T \in \mathbb{R}^n$。
- $P_\pi \in \mathbb{R}^{n\times n}$（State transition matrix）。

假设目前网格世界有有 4 个状态，按照式 38 列出具体的计算公式：

$$
\underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix}}_{v_\pi}
=
\underbrace{\begin{bmatrix} r_\pi(s_1) \\ r_\pi(s_2) \\ r_\pi(s_3) \\ r_\pi(s_4) \end{bmatrix}}_{r_\pi}
+ \gamma
\underbrace{\begin{bmatrix}
p_\pi(s_1 \mid s_1) & p_\pi(s_2 \mid s_1) & p_\pi(s_3 \mid s_1) & p_\pi(s_4 \mid s_1) \\
p_\pi(s_1 \mid s_2) & p_\pi(s_2 \mid s_2) & p_\pi(s_3 \mid s_2) & p_\pi(s_4 \mid s_2) \\
p_\pi(s_1 \mid s_3) & p_\pi(s_2 \mid s_3) & p_\pi(s_3 \mid s_3) & p_\pi(s_4 \mid s_3) \\
p_\pi(s_1 \mid s_4) & p_\pi(s_2 \mid s_4) & p_\pi(s_3 \mid s_4) & p_\pi(s_4 \mid s_4)
\end{bmatrix}}_{p_\pi}
\underbrace{\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix}}_{v_\pi} \tag{39}
$$

给定模型，带入数值并计算：

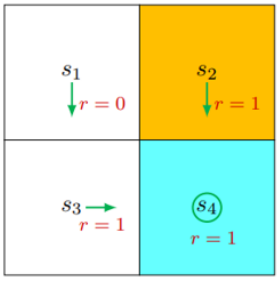


图 2.4.1 例子

$$\begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix} = \begin{bmatrix} 0.5(0)+0.5(-1) \\ 1 \\ 1 \\ 1 \end{bmatrix} + \gamma \begin{bmatrix} 0 & 0.5 & 0.5 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} v_\pi(s_1) \\ v_\pi(s_2) \\ v_\pi(s_3) \\ v_\pi(s_4) \end{bmatrix} \tag{40}$$

问题：为什么要求解状态价值？

回答：因为状态价值可以描述一个策略的好坏，我们期望强化学习可以找到设定任务下的最优策略，给定一个策略下求出对应状态价值的过程被称为策略评估，这是 RL 中的一个基本问题，是找到最优策略的基本方法。同时，它也有助于我们理解贝尔曼方程。

2.5 动作价值

上文介绍了状态价值，接下来介绍动作价值。从二者的定义上来区分：

- 状态价值：智能体从一个状态出发能够得到的期望 return。
- 动作价值：智能体从一个状态选择确定的一个动作，能够得到的期望 return。

问题：为什么引入动作价值？

回答：因为我们想知道在当前状态采取什么动作能够获得更高的奖励，这有助于我们找到最优策略。

动作价值的定义如下：

$$q_\pi(s,a) = \mathbb{E}[G_t \mid S_t = s, A_t = a] \tag{41}$$

- $q_\pi(s,a)$是状态-动作对(s,a)的函数。
- $q_\pi(s,a)$依赖于策略 π。

接下来推导动作价值和状态价值的关系，根据条件期望的性质可以得到，状态价值等于采取每一个动作的得到的动作价值×采取该动作的概率，全部求和：

$$\underbrace{\mathbb{E}[G_t \mid S_t = s]}_{v_\pi(s)} = \sum_a \underbrace{\mathbb{E}[G_t \mid S_t = s, A_t = a]}_{q_\pi(s,a)} \pi(a \mid s) \tag{42}$$

因此：

$$v_\pi(s) = \sum_a q_\pi(s,a)\pi(a \mid s) \tag{43}$$

比较式 43 与式 34，可以得到一些共同的部分，动作价值函数也就是如下，采取动作a获得立即奖励 r 的概率×奖励值 r+采取动作a跳到下一个状态 s' 的概率×该状态价值：

$$q_\pi(s,a)=\sum_r p(r\mid s,a)r+\gamma\sum_{s'}p(s'\mid s,a)v_\pi(s') \tag{44}$$

以上式子告诉我们，知道所有动作价值可以求解所有状态价值，知道所有状态价值可以求解所有动作价值。

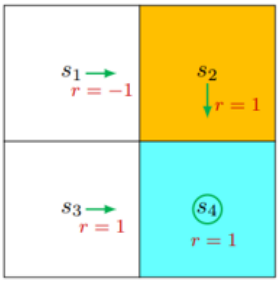


图 2.5.1 例子

例子中关于状态 s_1 的动作价值为，首先立即奖励为确定的-1，未来奖励为确定的 s_2 状态价值：

$$q_\pi(s_1,a_2)=-1+\gamma v_\pi(s_2) \tag{45}$$

问题：这个例子中其他动作的动作价值为 0 吗？

回答：并不为 0，动作价值是依赖于策略的，在当前策略 π 中，动作是确定的，其他动作概率为 0，在另一个策略 π_1 中，可能是随机策略，因此所有动作都会有其动作价值。

注意:

- 在我们想知道采取那种动作最好时，动作价值是一个非常重要的参考。
- 我们首先可以计算所有的状态价值，然后在计算动作价值。
- 我们也可以直接计算动作价值，在有模型或者无模型的条件下，利用一些算法，这在后面会介绍。

Chapter3 贝尔曼最优公式

3.1 策略改进

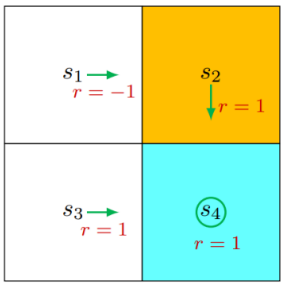


图 3.1.1 例子

在 3.1.1 的网格世界例子中，先列出所有状态的贝尔曼方程：

$$\begin{aligned} v_\pi(s_1) &= -1+\gamma v_\pi(s_2) \\ v_\pi(s_2) &= +1+\gamma v_\pi(s_4) \\ v_\pi(s_3) &= +1+\gamma v_\pi(s_4) \\ v_\pi(s_4) &= +1+\gamma v_\pi(s_4) \end{aligned} \tag{46}$$

设定折扣因子为 0.9，通过迭代法可以计算出所有状态价值：

$$v_\pi(s_4)=v_\pi(s_3)=v_\pi(s_2)=10,\ \ v_\pi(s_1)=8 \tag{47}$$

考虑状态 s_1 的所有动作价值：

$$\begin{aligned} q_\pi(s_1,a_1) &= -1+\gamma v_\pi(s_1)=6.2 \\ q_\pi(s_1,a_2) &= -1+\gamma v_\pi(s_2)=8 \\ q_\pi(s_1,a_3) &= 0+\gamma v_\pi(s_3)=9 \\ q_\pi(s_1,a_4) &= -1+\gamma v_\pi(s_1)=6.2 \\ q_\pi(s_1,a_5) &= 0+\gamma v_\pi(s_1)=7.2 \end{aligned} \tag{48}$$

问题：这个例子采取的策略显然不好，它进入了禁止区域，我们是否能改进这个策略呢？

回答：我们可以通过动作价值进行改进。

当前策略$\pi(a|s_1)$的数学形式如下：

$$\pi(a\,|\,s_1)=\begin{cases}1 & a=a_2\\ 0 & a\neq a_2\end{cases} \tag{49}$$

观察我们刚刚计算的状态 s_1 的所有动作价值，发现a_3的动作价值是最高的。因此，我们选择a_3作为一个新的策略：

$$\pi'(a\,|\,s_1)=\begin{cases}1 & a=a^*\\ 0 & a\neq a^*\end{cases} \tag{50}$$

其中$a^* = argmax_a\, q_\pi\,(s_1,a) = a_3$。

问题：为什么直接选择动作价值最大的动作就可以提高策略呢？

回答：直观上理解，动作价值代表一个动作的好坏，高的动作价值能够得到更大的 return。但实际上一次可能不会改进策略，这更像是迭代中的一个贪心策略，而后续的迭代算法会证明其最终会收敛到最优解。

3.2 最优策略及公式推导

在 3.1 节中，我们通过状态价值评估了一个策略的好或者坏，如果：

$$v_{\pi_1}(s)\geq v_{\pi_2}(s),\ \ for\ \text{all}\ s\in S \tag{51}$$

我们认为策略 π_1 要高于策略 π_2。

基于以上背景，给出最优策略的定义：如果一个策略 π^*对于任何一个状态的

状态价值都高于其他任何一个策略，称 π^*为最优策略。

首先直接给出贝尔曼最优公式的数学公式：

$$\begin{aligned} v(s) &= \max_{\pi} \sum_{a} \pi(a \mid s)(\sum_{r} p(r \mid s,a) + \gamma \sum_{s'} p(s' \mid s,a) v(s')),\ \forall s \in S \\ &= \max_{\pi} \sum_{a} \pi(a \mid s) q(s,a),\ \forall s \in S \end{aligned} \tag{52}$$

可以看出，这个公式里嵌入了一个优化过程，也就是求解最优策略 π，模型 p 是已知的，状态价值$v(s)$是可以被计算的，但是这些都依赖于确定的策略 π，接下来的问题是如何求解策略 π。

给出贝尔曼最优公式的矩阵形式：

$$v = \max_{\pi}(r_{\pi} + \gamma P_{\pi} v) \tag{53}$$

这里的 max 是对每个元素进行运算。

接下来，我们用一个例子解释如何对存在优化过程的公式进行求解。

$$x = \max_{a}(2x - 1 - a^2) \tag{54}$$

我们将优化过程和计算 x 过程分成两步走，首先确定是式子最大的a值，由于a平方大于 0，所以使这个式子最大，a只能等于 0，带入后求解$x = 2x - 1 - 0$，得到解为 1。

这个思路引入求解贝尔曼最优公式中。首先初始化一个$v(s)$值，然后在此基础上求解策略 π。

$$v(s) = \max_{\pi} \sum_{a} \pi(a \mid s) q(s,a) \tag{55}$$

再次引入一个数学例子解释如何求解$\max_{\pi} \sum_a \pi(a|s)\, q(s,a)$这一项。假设$q_i$属于实数，找到一组$c_i{}^*$，使得下式最大：

$$\max_{c_1,c_2,c_3}\ c_1 q_1 + c_2 q_2 + c_3 q_3, where\ c_1 + c_2 + c_3 = 1 \tag{56}$$

不失一般性，我们假设$q_3 \geq q_1, q_2$，此时最优解是$c_3{}^* = 1$ and $c_1{}^* = c_2{}^* = 0$。因为$q_3 = (c_1 + c_2 + c_3) q_3 = c_1 q_3 + c_2 q_3 + c_3 q_3 > c_1 q_1 + c_2 q_2 + c_3 q_3$，此时对于任意$c_i$，都要小于最优解。

受到上面的例子启发，我们知道在$q(s,a)$已知的情况下，能够使状态价值最大的策略 π^*也就是能够使$q(s,a)$最大的策略 π。

可以使用迭代求解方法求最大$q(s,a)$对应的策略 π，首先随机给一个初始值，选择使动作价值最大的策略，然后求解状态价值，更新动作价值，然后再次选择使动作价值最大的策略，循环迭代，最终可以收敛到最大$q(s,a)$。

$$\max_{\pi} \sum_{a} \pi(a \mid s) q(s,a) = \max_{a \in A(s)} q(s,a) \tag{57}$$

我们更新策略的方式为：

$$\pi(a|s)=\begin{cases}1 & a=a^* \\ 0 & a\neq a^*\end{cases} \tag{58}$$

其中，a^*是动作价值最大的那个动作。

$$a^*=\arg\max_a q(s,a) \tag{59}$$

3.3 公式求解及最优性

贝尔曼最优公式写成函数的形式：

$$f(v)=\max_\pi(r_\pi+\gamma P_\pi v) \tag{60}$$

然后，贝尔曼最优公式变为：

$$v=f(v) \tag{61}$$

引入一些概念：

- 不动点：一个 x 满足映射$f(x)=x$，称这个点为不动点。
- 压缩映射（压缩函数）：f是一个压缩函数，如果满足：

$$\|f(x_1)-f(x_1)\|\leq\gamma\|x_1-x_2\| \tag{62}$$

其中$\gamma\in(0,1)$。诸如$x=f(x)=0.5x, x=f(x)=Ax$（A 矩阵范数小于 1），都为压缩函数。

引入压缩映射定理：对于任何一个诸如$f(x)=x$的方程，如果f是一个压缩函数，则具有以下性质：

1. 存在性。存在一个不动点x^*，满足方程。
2. 唯一性。存在的不动点是唯一的。
3. 算法。利用$x_{k+1}=f(x_k)$，迭代循环计算，当$k\to\infty, x_k\to x^*$。

现在，我们利用上述理论计算贝尔曼最优公式：

$$v=f(v)=\max_\pi(r_\pi+\gamma P_\pi v) \tag{63}$$

可以证明是$f(v)$一个压缩函数，其中的 γ 小于 1 和状态转移矩阵 P 的范数都小于 1。可以知道一定存在一个唯一的最优解v^*，这个方程一定可以用迭代法求解出来。

v^*是贝尔曼最优方程的解，它满足：

$$v^*=\max_\pi(r_\pi+\gamma P_\pi v^*) \tag{64}$$

固定v^*后，可以得到策略 π^*：

$$\pi^*=\arg\max_\pi(r_\pi+\gamma P_\pi v^*) \tag{65}$$

因此满足：

$$v^* = r_{\pi^*} + \gamma P_{\pi^*} v^* \tag{66}$$

可以看出，贝尔曼最优方程实际上是一个特殊的贝尔曼方程，只不过多了一个最优化的过程和一个不动点迭代的求解思想。

而最优策略是确定的一个贪婪策略：

$$\pi^*(a \mid s) = \begin{cases} 1 & a = a^*(s) \\ 0 & a \neq a^*(s) \end{cases} \tag{67}$$

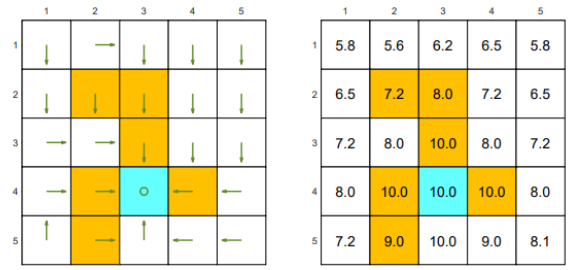
它告诉我们，最优策略实际上就是选择动作价值最大的那个动作。

3.4 最优策略性质

我们看一下贝尔曼最优公式，需要求解的变量是状态价值与策略，而已知的量是系统模型，我们通过改变系统模型分析其对最优策略的影响。

$$v(s) = \max_{\pi(s) \in \Pi(s)} \sum_{a \in \mathcal{A}} \pi(a \mid s) \left(\sum_{r \in \mathcal{R}} p(r \mid s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s' \mid s, a) v(s') \right) \tag{68}$$

例 1：γ 为 0.9。可以看出终点附近的状态价值都很高，而最优策略中，智能体甚至穿越了障碍物区域到达终点，因为智能体现在更注重未来奖励，它发现障碍物给予的惩罚要远小于直接到达终点的奖励，所以它宁愿冒着风险也要穿越。



(a) Baseline example: $r_{\text{boundary}} = r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$.

图 3.4.1 例 1

例 2：γ 为 0.5。很明显的转变是智能体已经不进入障碍物区域了，因为智能体现在更注重短期奖励，避免直接进入障碍物区域导致的惩罚。

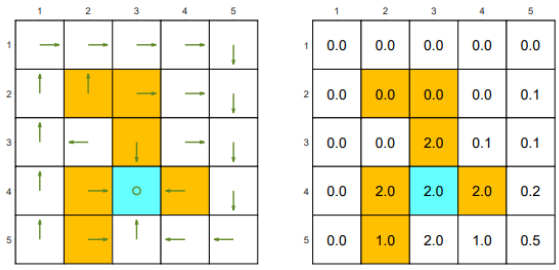


图 3.4.2.例 2

例 3：γ 为 0。智能体变得完全近视，只看立即奖励，不能够到达终点。

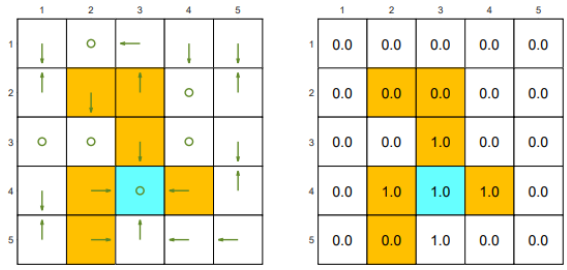


图 3.4.3 例 3

例 4：γ 为 0.9，障碍物惩罚为-10。可以看出，智能体不仅仅能够到达终点，而且避开了障碍物，达到了平衡。

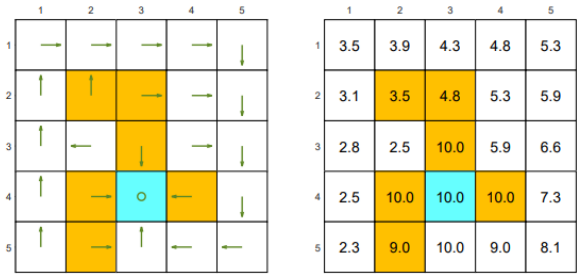


图 3.4.4 例 4

例 5：如果我们改变 r 为$ar+b$，所有的奖励值都进行线性变化，最后的最优策略是一样的，说明最优策略依赖于奖励的相对关系，而不是绝对值。

考虑一个马尔科夫决策过程满足状态价值计算式：

$$v^* = \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v^*) \tag{69}$$

变化奖励后，其状态价值具有以下变换：

$$v' = \alpha v^* + \frac{\beta}{1-\gamma}1 \tag{70}$$

可以看出，变换后的最优状态价值与变换之前的最优状态价值之间是线性的，单调的变化。

例 6：对比如下这两个策略，直观上可以看出，策略 2 是不好的，因为它存在绕路的情况，我们计算其状态价值也是小于策略 1 的，从数学上分析一下两者之间的区分。

关键在于折扣因子的引入，对于策略 1：

$$\text{return} = 1 + \gamma 1 + \gamma^2 1 + \cdots = 1/(1-\gamma) = 10 \tag{71}$$

对于策略 2：

$$\text{return} = 0 + \gamma 0 + \gamma^2 1 + \gamma^3 1 + \cdots = \gamma^2/(1-\gamma) = 8.1 \tag{72}$$

可以看出，由于策略 2 多绕了一步，导致奖励值多乘了一次折扣因子造成更多衰减，也就是说，return 的计算天然受到步数的影响，这本身也是一种贪婪。

（a）策略 1　　（b）策略 2

图 3.4.5 例 6

Chapter4 值迭代与策略迭代

4.1 值迭代算法

在 3 节中，我们已经给出了一种贝尔曼最优公式的求解方法，利用压缩映射原理和不动点迭代方法：

$$v_{k+1} = f(v_k) = \max_{\pi}(r_\pi + \gamma P_\pi v_k), \quad k = 1, 2, 3\ldots \tag{73}$$

这里的初始状态价值v_0是任意的。算法从v_0开始，然后更新策略 π，依次迭代，最终找到最优策略。

这个算法就叫做值迭代算法（Value iteration）。它对应了两个步骤：

Step1：策略更新（Policy update），这一步求解：

$$\pi_{k+1} = \arg\max(r_\pi + \gamma P_\pi v_k) \tag{74}$$

其中v_k是预先给定的。

Step2：值更新（Value update），这一步求解：

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \tag{75}$$

这里需要注意，v_k是我们定义的状态价值吗？仔细查看方程两边，发现左边角标是 k+1，右边是 k，这并是贝尔曼方程的形式，所以这里的v_k只是迭代中未收敛的一个值，而不是真正的状态价值。

针对 Step1 策略更新，我们的具体数学求解形式为：

$$\pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a) r + \gamma \sum_{s'} p(s'|s,a) v_k(s')\right)}_{q_k(s,a)}, \quad s \in \mathcal{S} \tag{76}$$

在 3.3 节中我们已经知道，这个优化问题的解满足下面这种形式：

$$\pi_{k+1}(a|s) = \begin{cases} 1 & a = a_k^*(s) \\ 0 & a \neq a_k^*(s) \end{cases} \tag{77}$$

其中$a_k^*(s) = arg\max_a q_k(a,s)$，$\pi_{k+1}$也叫做贪婪策略，它每次都会选择动作价值最大的那个动作。

针对 Step2 值更新，我们的具体数学求解形式为：

$$v_{k+1}(s)=\sum_a \pi_{k+1}(a\mid s)\underbrace{\left(\sum_r p(r\mid s,a)r+\gamma\sum_{s'} p(s'\mid s,a)v_k(s')\right)}_{q_k(s,a)}, s\in\mathcal{S} \tag{78}$$

因为我们知道贪婪策略π_{k+1}只选择动作价值最大的那一个动作，因此求和符号去掉，状态价值就等于一个动作的动作价值，上式简化为：

$$v_{k+1}(s)=\max_a q_k(a,s) \tag{79}$$

表 4.1.1 值迭代算法伪代码

伪代码：值迭代算法
Initialization：对于每一个(s,a)都已知系统模型$p(r\|s,a)$和$p(s'\|s,a)$。随机初始化v_0的值。 Aim：找到最优状态价值和最优策略。 While $\|v_k-v_{k+1}\|\geq\varepsilon$ 其中ε为设定的阈值 对于第 k 次迭代 For 每个状态$s\in S$ For 每个动作$a\in A(s)$ 计算动作价值 *q-value*：$q_k(s,a)=\sum_r P(r\|s,a)r+\gamma\sum_{s'}p(s'\|s,a)v_k(s')$ 求最大动作价值的动作$a_k{}^*(s)=arg\max_a q_k(a,s)$ Policy update：$\pi_{k+1}(a\|s)=1, if\ a=a_k{}^*$，其他动作概率为 0。 Value update：$v_{k+1}(s)=\max_a q_k(a,s)$ 计算$\|v_k-v_{k+1}\|$

用一个例子说明算法的运算流程。边界惩罚、障碍物惩罚为-1，终点奖励为+1，折扣因子为 0.9。

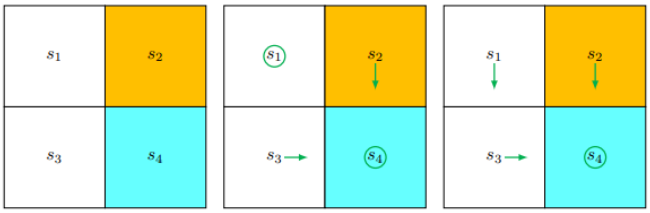


图 4.1.1 例 1

我们首先列出四个状态和其可能动作的 q-table。

表 4.1.2 q-table

q-table	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma v(s_1)$	$-1+\gamma v(s_2)$	$0+\gamma v(s_3)$	$-1+\gamma v(s_1)$	$0+\gamma v(s_1)$
s_2	$-1+\gamma v(s_2)$	$-1+\gamma v(s_2)$	$1+\gamma v(s_4)$	$0+\gamma v(s_1)$	$-1+\gamma v(s_2)$
s_3	$0+\gamma v(s_1)$	$1+\gamma v(s_4)$	$-1+\gamma v(s_3)$	$-1+\gamma v(s_3)$	$0+\gamma v(s_3)$
s_4	$-1+\gamma v(s_2)$	$-1+\gamma v(s_4)$	$-1+\gamma v(s_4)$	$0+\gamma v(s_3)$	$1+\gamma v(s_4)$

算法开始，第一次迭代 k=0，初始化$v_0(s_i)=0$。

表 4.1.3 k=0

q-table	a_1	a_2	a_3	a_4	a_5
s_1	-1	-1	0	-1	0
s_2	-1	-1	1	0	-1
s_3	0	1	-1	-1	0
s_4	-1	-1	-1	0	1

Step1：Policy update，选择动作价值最大的那个动作。

$$\pi_1(a_5\,|\,s_1)=1,\pi_1(a_3\,|\,s_2)=1,\pi_1(a_2\,|\,s_3)=1,\pi_1(a_5\,|\,s_4)=1 \tag{80}$$

策略可视化为例 4.1.1 中第二个子图，可以看出状态 23 已经是最优策略了，但是状态 1 原地不动，并不是最优的，我们需要继续进行。

Step2：Value update，状态价值等于最优动作价值。

$$v_1(s_1)=0,v_1(s_2)=1,v_1(s_3)=1,v_1(s_4)=1 \tag{81}$$

第二次迭代，k=1，状态价值为$v_1(s_1)=0,v_1(s_2)=1,v_1(s_3)=1,v_1(s_4)=1$。

表 4.1.4 k=1

q-table	a_1	a_2	a_3	a_4	a_5
s_1	$-1+\gamma 0$	$-1+\gamma 1$	$0+\gamma 1$	$-1+\gamma 0$	$0+\gamma 0$
s_2	$-1+\gamma 1$	$-1+\gamma 1$	$1+\gamma 1$	$0+\gamma 0$	$-1+\gamma 1$
s_3	$0+\gamma 0$	$1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$
s_4	$-1+\gamma 1$	$-1+\gamma 1$	$-1+\gamma 1$	$0+\gamma 1$	$1+\gamma 1$

Step1：Policy update，选择动作价值最大的那个动作。

$$\pi_2(a_3\,|\,s_1)=1,\pi_2(a_3\,|\,s_2)=1,\pi_2(a_2\,|\,s_3)=1,\pi_2(a_5\,|\,s_4)=1 \tag{82}$$

策略可视化为例 4.1.1 中第 3 个子图，可以看出状态 123 均为最优策略。

Step2：Value update，状态价值等于最优动作价值。

$$v_2(s_1)=\gamma 1,v_2(s_2)=1+\gamma 1,v_2(s_3)=1+\gamma 1,v_2(s_4)=1+\gamma 1 \tag{83}$$

我们可以继续往下迭代 k=2,3,4……，直到每轮解变化小于某个阈值停止迭代。上面这个例子比较简单，所以两次迭代已经收敛到最优解了，在复杂问题中需要更多迭代次数才可以收敛到最优解。

4.2 策略迭代算法

策略迭代算法同样有两个步骤。首先随机初始化策略 π_0。

Step1：策略评估（Policy evaluation，PE），这一步是计算该策略下的状态价值。

$$v_{\pi_k}=r_{\pi_k}+\gamma P_{\pi_k}v_{\pi_k} \tag{84}$$

注意这时的$v_{\pi k}$是确定策略下的状态价值，而不是迭代中的某个值。

Step2：策略改进（Policy improvement，PI），对每个状态求解能够使状态价

值达到最大的策略。

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k}) \tag{85}$$

算法的过程如下：

$$\pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots \tag{86}$$

问题 1：在策略评估步骤中，如何通过求解贝尔曼方程得到状态价值？

我们之前求解贝尔曼方程有两个解法。

- 矩阵解法：

$$v_{\pi_k} = (I - \gamma P_{\pi_k})^{-1} r_{\pi_k} \tag{87}$$

- 迭代解法（常用）：

$$v_{\pi_k}^{(j+1)} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k}^{(j)}, \quad j = 0,1,2,... \tag{88}$$

所以在策略评估 Step1 中已经嵌套了一个小的迭代循环，用于求解当前策略下的准确的状态价值，外部套着一个大的策略迭代循环。

问题 2：在策略改进步骤中，如何保新的策略一定更好？

由策略改进的公式可以进行证明，新的策略的状态价值一定高于改进前策略。

问题 3：如何保证迭代算法最终会得到最优策略？

从迭代过程中我们知道状态价值会不断增加，同时从最优化理论可以证明，这里收敛到的状态价值就是全局最优的。

$$v_{\pi_0} \le v_{\pi_1} \le v_{\pi_2} \le \cdots \le v_{\pi_k} \le \cdots \le v^* \tag{89}$$

问题 4：策略迭代算法与值迭代算法有什么关系？

实际上，策略迭代涉及到值迭代算法，由于值迭代算法收敛，所以策略迭代也会收敛。它们是截断策略迭代算法的两个极端。

针对 Step1 策略评估，我们的具体数学求解形式为：

$$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a|s)\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}^{(j)}(s')\right) \tag{90}$$

注意这里面，我们首先确定了第 k 次迭代的策略 π_k，然后对 v_k 进行迭代求解，j 是迭代次数，直到 v_k 收敛。也就是第 k 次策略评估里面，嵌套了一次迭代求解。

针对 Step2 策略改进，我们的具体数学求解形式为：

$$\pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s) \underbrace{\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')\right)}_{q_{\pi_k}(s,a)}, \quad s \in \mathcal{S} \tag{91}$$

当 Step1 迭代求解出所有状态的状态价值后，我们求解出所有状态的动作价值，然后进行贪婪选择，选择动作价值最高的那个动作。

$$a_k^*(s) = \arg\max_a q_{\pi_k}(a,s) \tag{92}$$

更新后的策略是：

$$\pi_{k+1}(a \mid s) = \begin{cases} 1, & a = a_k^*(s), \\ 0, & a \neq a_k^*(s). \end{cases} \tag{93}$$

表 4.2.1 策略迭代算法伪代码

伪代码：策略迭代算法
Initialization：对于每一个(s,a)都已知系统模型$p(r\|s,a)$和$p(s'\|s,a)$。随机初始化策略π_0。 Aim：找到最优状态价值和最优策略。 While 策略还没有收敛 对于第 k 次迭代 PE： Initialization：随机初始化$v_{\pi_k}{}^{(0)}$ While 状态价值还没有收敛 For 每个状态$s \in S$ 计算$v_{\pi_k}^{(j+1)}(s) = \sum_a \pi_k(a\|s)\left(\sum_r p(r\|s,a)r + \gamma \sum_{s'} p(s'\|s,a)\, v_{\pi_k}^{(j)}(s')\right)$ PI： For 每个状态$s \in S$ For 每个动作$a \in A(s)$ 计算动作价值：$q_{\pi_k}(s,a) = \sum_r p(r\|s,a)r + \gamma \sum_{s'} p(s'\|s,a) v_{\pi_k}(s')$ 求最大动作价值的动作$a_k{}^*(s) = arg\max_a q_{\pi k}(a,s)$ $\pi_{k+1}(a\|s) = 1, if\ a = a_k{}^*$，其他动作概率为 0。 判断策略是否收敛

用一个例子说明算法的运算流程。边界惩罚、障碍物惩罚为-1，终点奖励为+1，折扣因子为 0.9。每个网格有三个动作，向左、向右或不动。

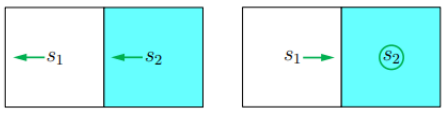


图 4.2.1 例 1

算法开始，第一次迭代 k=0，执行 step1 策略评估。初始化策略为例 1 左图，计算各状态价值，因为例子比较简单，直观上就可以直接写出，如果环境比较复杂，采用迭代法进行求解。

$$v_{\pi_0}(s_1) = -1 + \gamma v_{\pi_0}(s_1), v_{\pi_0}(s_2) = 0 + \gamma v_{\pi_0}(s_1). \tag{94}$$

法 1：直接求解。

$$v_{\pi_0}(s_1) = -10, \quad v_{\pi_0}(s_2) = -9. \tag{95}$$

法 2：迭代法求解，初始状态价值为 0。

$$\begin{aligned}
&\begin{cases} v_{\pi_0}^{(1)}(s_1) = -1 + \gamma v_{\pi_0}^{(0)}(s_1) = -1, \\ v_{\pi_0}^{(1)}(s_2) = 0 + \gamma v_{\pi_0}^{(0)}(s_1) = 0, \end{cases} \\
&\begin{cases} v_{\pi_0}^{(2)}(s_1) = -1 + \gamma v_{\pi_0}^{(1)}(s_1) = -1.9, \\ v_{\pi_0}^{(2)}(s_2) = 0 + \gamma v_{\pi_0}^{(1)}(s_1) = -0.9, \end{cases} \\
&\begin{cases} v_{\pi_0}^{(3)}(s_1) = -1 + \gamma v_{\pi_0}^{(2)}(s_1) = -2.71, \\ v_{\pi_0}^{(3)}(s_2) = 0 + \gamma v_{\pi_0}^{(2)}(s_1) = -1.71, \end{cases} \\
&\cdots\cdots
\end{aligned} \tag{96}$$

进行 Step2 策略改进。已知状态价值，求解所有的动作价值，表格如下：

表 4.2.2 动作价值表

$q_{\pi_k}(s,a)$	a_ℓ	a_0	a_r
s_1	$-1+\gamma v_{\pi_k}(s_1)$	$0+\gamma v_{\pi_k}(s_1)$	$1+\gamma v_{\pi_k}(s_2)$
s_2	$0+\gamma v_{\pi_k}(s_1)$	$1+\gamma v_{\pi_k}(s_2)$	$-1+\gamma v_{\pi_k}(s_2)$

带入 Step1 得到的值后：

表 4.2.3 动作价值表

$q_{\pi_0}(s,a)$	a_ℓ	a_0	a_r
s_1	-10	-9	-7.1
s_2	-9	-7.1	-9.1

通过贪婪选择，找到最优动作价值对应的那个动作，作为新的策略。改进后的策略为：

$$\pi_1(a_r \mid s_1) = 1, \quad \pi_1(a_0 \mid s_2) = 1. \tag{97}$$

可以看到通过一次迭代我们就找到了最优策略，在复杂问题中，需要更多次迭代即可收敛。

下面是一个比较复杂的例子，我们可以发现一些有趣的现象。在策略优化过程中，靠近目标点附近的策略是先得到最优的，然后在慢慢向外扩散，最终整个网格都收敛到最优，直观上来看，如果我们想最终达到终点，就算前面策略再好，靠近终点的策略不好，智能体也不会到达终点，所以靠近终点的策略是最好确定的。

另外，在最优化理论中，这个过程有点类似于路径寻优，我们也可以从折扣因子中进行思考，因为靠近终点的状态的 return 损失最少，所以也最好确定，而距离终点越远，奖励值损失多，所以也最难确定，这个过程也有点类似

于梯度下降的过程。

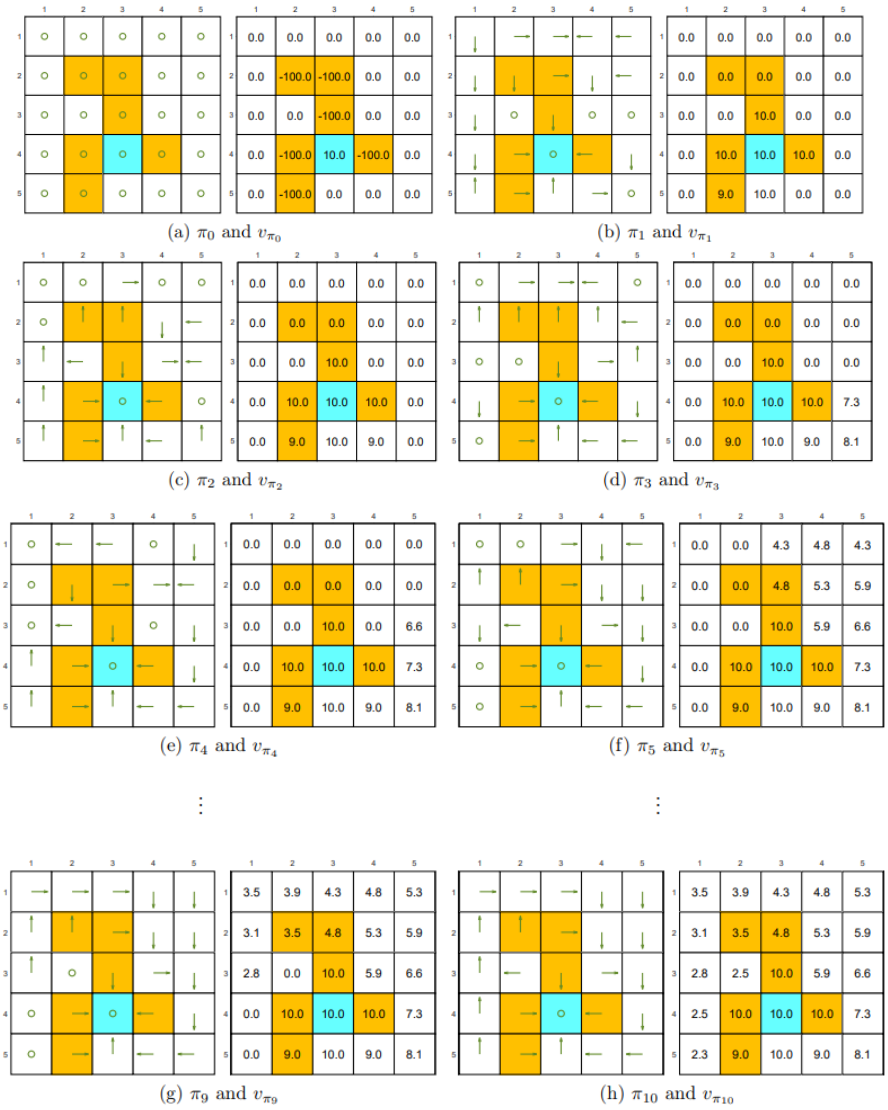


图 4.2.2 例 2

4.3 截断策略迭代算法

截断策略迭代算法（Truncated policy iteration algorithm）实际上不是一种新算法，它是值迭代算法和策略迭代算法的一种更普遍的形式，值迭代算法和策略迭代算法是它的两种极端情况。首先我们回顾一下值迭代和策略迭代的过程：

Policy iteration：从随机策略开始

- PE：

$$v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \tag{98}$$

- PI：

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k}) \tag{99}$$

Value iteration：从随机状态价值开始

- PU：

$$\pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_k) \tag{100}$$

- VU：

$$v_{k+1} = r_{\pi_{k+1}} + \gamma P_{\pi_{k+1}} v_k \tag{101}$$

对比一下这两个算法的过程。

$$\begin{aligned} &\text{Policy iteration: } \pi_0 \xrightarrow{PE} v_{\pi_0} \xrightarrow{PI} \pi_1 \xrightarrow{PE} v_{\pi_1} \xrightarrow{PI} \pi_2 \xrightarrow{PE} v_{\pi_2} \xrightarrow{PI} \dots \\ &\text{Value iteration: } \quad u_0 \xrightarrow{PU} \pi_1' \xrightarrow{VU} u_1 \xrightarrow{PU} \pi_2' \xrightarrow{VU} u_2 \xrightarrow{PU} \dots \end{aligned} \tag{102}$$

我们利用表格更详细的对比这两个算法。第一步时，策略迭代给出随机策略 0，而值迭代还未执行；第二步时，策略迭代依据策略 0，通过迭代计算出状态价值，为了方便对比，我们强行让值迭代的随机状态价值等于策略 0 的状态价值；第三步时，两个算法的操作一致，找到动作价值最高的动作作为更新后的策略；第四步时，两个算法出现了不同，策略迭代利用贝尔曼方程求解出状态价值，值迭代的公式与贝尔曼公式很像，但不是贝尔曼公式，它利用上一步迭代值计算了状态价值；第五步以后自然就不一样了。

	Policy iteration algorithm	Value iteration algorithm	Comments
1) Policy:	π_0	N/A	
2) Value:	$v_{\pi_0} = r_{\pi_0} + \gamma P_{\pi_0} v_{\pi_0}$	$v_0 \doteq v_{\pi_0}$	
3) Policy:	$\pi_1 = \arg\max_\pi(r_\pi + \gamma P_\pi v_{\pi_0})$	$\pi_1 = \arg\max_\pi(r_\pi + \gamma P_\pi v_0)$	The two policies are the same
4) Value:	$v_{\pi_1} = r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}$	$v_1 = r_{\pi_1} + \gamma P_{\pi_1} v_0$	$v_{\pi_1} \geq v_1$ since $v_{\pi_1} \geq v_{\pi_0}$
5) Policy:	$\pi_2 = \arg\max_\pi(r_\pi + \gamma P_\pi v_{\pi_1})$	$\pi_2' = \arg\max_\pi(r_\pi + \gamma P_\pi v_1)$	
⋮	⋮	⋮	⋮

图 4.3.1 算法对比

从上面的对比中，我们注意到第四步时，两个算法有明显的区别，我们单独进行分析。

$$\begin{aligned} v_{\pi_1}^{(0)} &= v_0 \\ v_{\pi_1}^{(1)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(0)} \to v_1 \to VI \\ v_{\pi_1}^{(2)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(1)} \\ &\vdots \\ v_{\pi_1}^{(j)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(j-1)} \to \bar{v}_1 \to TPI \\ &\vdots \\ v_{\pi_1}^{(\infty)} &= r_{\pi_1} + \gamma P_{\pi_1} v_{\pi_1}^{(\infty)} \to v_{\pi_1} \to PI \end{aligned} \tag{103}$$

可以发现，值迭代类似于雅可比迭代，它在每次迭代中，状态价值基于上一个策略得到的旧值进行更新（也就是只进行一步值更新），然后立即更新策略。这种方式收敛往往比较慢。

策略迭代类似于高斯-赛德尔迭代，它采用嵌套更新，在每次迭代中，状态价值的更新基于当前策略条件下，迭代中已计算的新值（值更新进行多步迭代，得到更准确的值），然后更新策略。这种方式能够加速收敛。

截断策略迭代是策略迭代的变种，在策略迭代中只进行 K 次值更新，而不是无穷次，减少了策略评估的计算量，使得实际可行，但可能会增加外部循环次数。

Chapter5 蒙特卡洛方法

5.1 蒙特卡洛引入

之前我们学习到的 Value iteration 算法和 Policy iteration 算法都是基于已知 model 的算法，它属于动态规划的理论。但实际情况中，系统的模型往往是未知的，怎么在没有模型的情况下估计系统的一些参量呢？我们的基本思想是蒙特卡洛估计方法。

举一个抛硬币的例子。假设抛硬币的结果用一个随机变量 X 来表示。如果硬币正面朝上，则 X=+1，如果硬币反面朝上，则 X=-1，我们的目标是估计 X 的期望值。

- 法 1：Model-based

假设抛硬币的概率模型已知：

$$p(X=1)=0.5,\quad p(X=-1)=0.5 \tag{104}$$

利用期望的定义公式可以计算：

$$\mathbb{E}[X]=\sum_x xp(x)=1\times 0.5+(-1)\times 0.5=0 \tag{105}$$

这种方法非常简单，但问题是非常精确的概率模型很难得到。

- 法 2：Model-free

我们抛 N 次硬币，记录每次结果，当抛的次数足够多时：

$$\mathbb{E}[X]\approx \bar{x}=\frac{1}{N}\sum_{j=1}^{N}x_j \tag{106}$$

也就是利用不停的实验，在样本数量足够多的时候，能够用样本的统计量（一阶原点矩是期望的一个无偏估计量）估计总体的统计量，这就是蒙特卡洛估计方法的思想（大数定理证明）。

5.2 MC-basic 算法

MC-basic 是一个 model-free 的算法，对比于策略迭代算法，我们主要解决的问题是如何将策略迭代变为 model-free，这需要引入蒙特卡洛估计方法。

我们回顾一下策略迭代算法的步骤。

$$\begin{cases} \text{PE:}\ v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{PI:}\ \pi_{k+1} = \arg\max_{\pi}(r_\pi + \gamma P_\pi v_{\pi_k}) \end{cases} \tag{107}$$

其中：

$$\pi_{k+1}(s) = \arg\max_{\pi} \sum_a \pi(a|s)\left(\sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s')\right), \quad s \in \mathcal{S} \tag{108}$$

可以看出，当模型未知时，$q_{\pi_k}(s,a)$是未知的。我们从两种计算$q_{\pi_k}(s,a)$的方法思考：

- 法 1：已知模型求解。

$$q_{\pi_k}(s,a) = \sum_r p(r|s,a)r + \gamma \sum_{s'} p(s'|s,a) v_{\pi_k}(s') \tag{109}$$

- 法 2：用原始定义。

$$q_{\pi_k}(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a] \tag{110}$$

我们发现，原始定义方法是不依赖于模型的，在式 110 中只需要求出期望即可，这与我们在 5.1 节引入的蒙特卡洛估计方法不谋而合。

具体来说，我们从状态-动作(s,a)开始，遵循当前策略 π，完成一回合产生一个轨迹，这个回合得到的 return 是 $g(s,a)$，而 $g(s,a)$就是 G_t 的一个采样值。我们从当前状态-动作对开始，进行足够多的回合，收集到足够多的 $g(s,a)$序列，然后我们就可以利用样本 $g(s,a)$的平均值计算 G_t 的期望。

$$q_{\pi_k}(s,a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N}\sum_{i=1}^{N} g^{(i)}(s,a) \tag{111}$$

在强化学习中，样本有特殊的一个名称——经验（Experience）。在有模型的时候，我们不需要样本数据，直接通过具体的概率模型计算期望，这种方式有两个问题：（1）很多情况下，我们并不知道复杂问题的模型，或者无法知道问题的模型。（2）就算已知问题的模型，但也许它的计算维度非常高，传统方法无法解决。在无模型时，我们就需要样本数据，通过大量采样，用样本的表现出来的性质来作为总体的性质。

表 5.2.1 MC-basic 算法伪代码

伪代码：MC-basic 算法

Initialization：随机初始化策略π_0。
Aim：找到最优状态价值和最优策略。
While 价值估计还没有收敛
  For 每个状态$s \in S$
  For 每个动作$a \in A(s)$
  从当前状态-动作对(s,a)出发，获得一系列经验。

MC-basic 策略评估步骤：

动作价值等于从一系列经验的平均值。

策略改进步骤：

求最大动作价值的动作$a_k{}^*(s) = arg\max_a q_{\pi k}(a,s)$

$\pi_{k+1}(a|s) = 1, if\ a = a_k{}^*$，其他动作概率为 0。

判断策略是否收敛

总结：MC-basic 不是新算法，它只是策略迭代算法的一个变种，将其中依赖于模型的部分替换为利用样本估计。Model-free 算法是建立在 Model-based 算法的基础上，想要理解无模型算法必须先理解有模型算法。虽然 MC-basic 算法解决了无模型问题，但它实际的效率是非常低的，后面我们会介绍更有效的算法。为什么我们不估计状态价值，而是计算动作价值呢？虽然状态价值可以计算出动作价值，但它的计算过程是依赖于模型的，所以不如直接计算动作价值。

5.3 MC-basic 算法用例

我们用一个例子说明 MC-basic 算法的过程。例 1 展示了一个初始策略，边界惩罚、障碍物惩罚为-1，终点奖励为+1，折扣因子为 0.9。

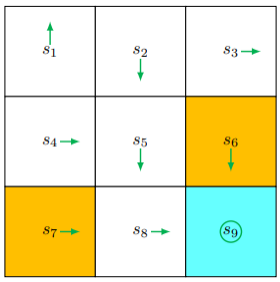


图 5.3.1 例 1

Step1：策略评估。由于策略是确定的，每个状态只执行一个动作，所以每个状态-动作对只有一条轨迹，所以需要采样一条轨迹即可。如果策略是随机的，那么就不能只采样一条了。

给出关于状态 1 的所有动作价值。

从状态-动作对(s_1,a_1)开始，轨迹和动作价值的计算是：

$$\begin{aligned} &s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots \\ &q_{\pi_0}(s_1,a_1) = -1 + \gamma(-1) + \gamma^2(-1) + \dots \end{aligned} \tag{112}$$

从状态-动作对(s_1,a_2)开始，轨迹和动作价值的计算是：

$$\begin{aligned} &s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} \dots \\ &q_{\pi_0}(s_1,a_2) = 0 + \gamma 0 + \gamma^2 0 + \gamma^3(1) + \gamma^4(1) + \dots \end{aligned} \tag{113}$$

从状态-动作对(s_1,a_3)开始，轨迹和动作价值的计算是：

$$
\begin{aligned}
&s_1 \xrightarrow{a_3} s_4 \xrightarrow{a_2} s_5 \xrightarrow{a_3} \dots \\
&q_{\pi_0}(s_1, a_3) = 0 + \gamma 0 + \gamma^2 0 + \gamma^3 (1) + \gamma^4 (1) + \dots
\end{aligned} \tag{114}
$$

从状态-动作对(s_1,a_4)开始，轨迹和动作价值的计算是：

$$
\begin{aligned}
&s_1 \xrightarrow{a_4} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots. \\
&q_{\pi_0}(s_1, a_4) = -1 + \gamma(-1) + \gamma^2(-1) + \dots
\end{aligned} \tag{115}
$$

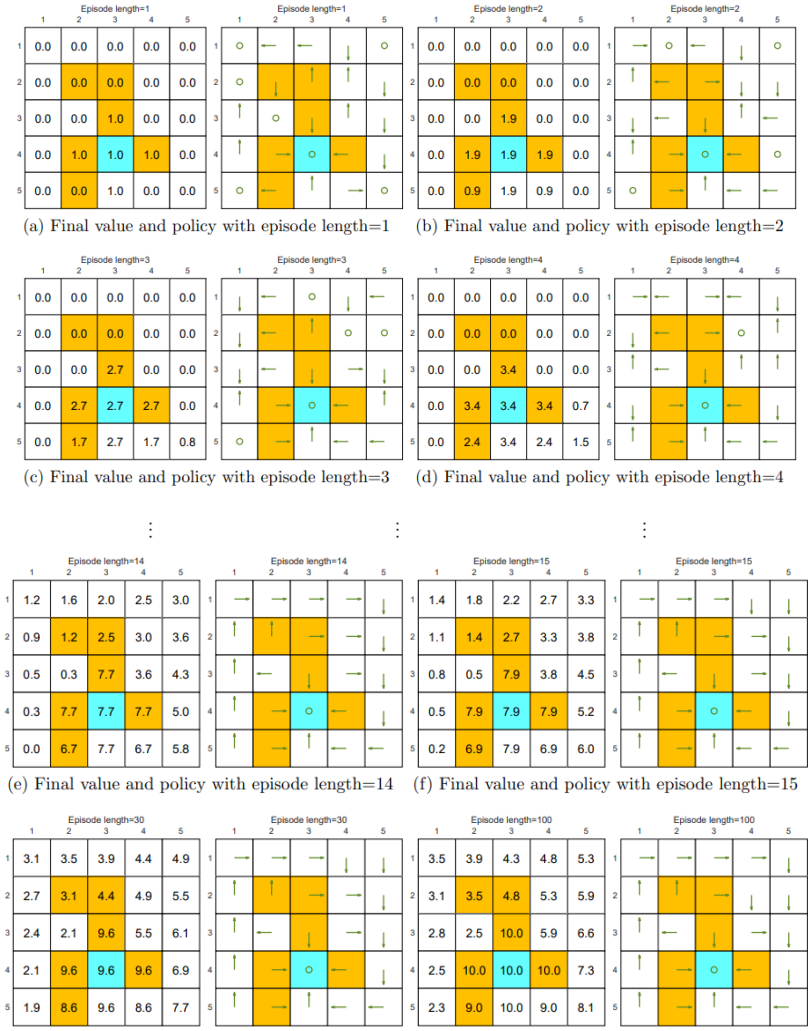
从状态-动作对(s_1,a_5)开始，轨迹和动作价值的计算是：

$$
\begin{aligned}
&s_1 \xrightarrow{a_5} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots. \\
&q_{\pi_0}(s_1, a_5) = 0 + \gamma(-1) + \gamma^2(-1) + \dots
\end{aligned} \tag{116}
$$

Step2：策略改进。通过观察动作价值，我们将最大动作价值的那个动作作为新的策略，最终策略更新为：

$$
\pi_1(a_2 \mid s_1) = 1 \quad \text{or} \quad \pi_1(a_3 \mid s_1) = 1. \tag{117}
$$

在上面这个简单的例子中，一步就找到了最优策略。下面我们再分析一个例子，研究轨迹长度对 MC-basic 算法的影响。



(a) Final value and policy with episode length=1
(b) Final value and policy with episode length=2
(c) Final value and policy with episode length=3
(d) Final value and policy with episode length=4
(e) Final value and policy with episode length=14
(f) Final value and policy with episode length=15

图 5.3.2 例 2

我们从 length=1 开始逐渐增加 length=15，可以看到在长度为 1 时，只有终点附近的状态价值不为 0，说明只有终点附近看到了终点，而其他网格为 0。增加长度到 4，发现终点附近的状态价值逐渐变高，而距离终点比较远的网格依旧为 0。直到长度到 15 时，所有网格才均为最优策略，而长度超过 15，策略就没有变化了。受到这个现象的启示，我们知道轨迹的长度不能无限长，也不能很短，适合解决问题的长度是最好的。

5.4 MC-exploring starts 算法

在 5.3 节我们介绍了 MC-basic 算法，并且指出了 MC-basic 算法虽然简单，但其效率非常低，本节将介绍如何对 MC-basic 算法进行改进。

考虑一个网格世界的例子，在随机策略下，我们从某个状态-动作对开始，产生轨迹：

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \tag{118}$$

我们引入访问的概念。如果某个状态-动作对在这个轨迹中出现，我们称这个状态-动作对被访问。MC-basic 算法使用的是初始访问方法（Initial-visit method），也就是只计算初始状态-动作对的 return，我们指出一个问题是，它没有完全利用轨迹中的数据。

通过观察轨迹可以发现，从初始状态-动作对开始，后续的每一个状态-动作对都可以视为以该状态-动作对开始的轨迹，因为马尔科夫的无记忆性，后续状态并不受前面状态的影响。

$$\begin{aligned}
&s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \\
&\quad\quad s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \\
&\quad\quad\quad\quad s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \\
&\quad\quad\quad\quad\quad\quad s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \\
&\quad\quad\quad\quad\quad\quad\quad\quad s_5 \xrightarrow{a_1} \dots
\end{aligned} \tag{119}$$

可以看出，后续的每一个状态-动作对都可以视作从该状态-动作开始的轨迹，而 MC-basic 算法并没有利用到后面的信息，而是仅仅使用初始状态的一次信息。

这种数据利用有两种方法，一种是 first-visit 方法，我只对该轨迹中出现的第一次状态-动作对进行利用，重复出现以第一次出现为主。另一种是 every-visit 方法，顾名思义，它对轨迹每个出现的状态-动作都进行使用。

除去对数据利用效率的改进，我们也可以对策略更新步骤进行改进。这部分可以参考值迭代和策略迭代算法的区别，我们在 MC-basic 算法中要收集到所有的状态-动作对才更新策略，而改进的方法是，我们只收集到部分轨迹就立马更

新策略（更新出现的所有动作价值，其他未出现的按照上一时刻的值），那么它的效率会提升。

这种方法为什么可行呢？可以参考 4.3 节截断策略迭代算法的思想，我们通过一个轨迹包含部分状态-动作对计算出的动作价值固然有误差，但不影响我们后续策略的收敛，而且相对于收集到全部更新，这种方式显然更有效率。

上述的几种算法或改进方法都可以归为普遍策略迭代算法（Generalized Policy iteration）的概念。这不是一个具体的算法，而是一类算法，它是策略评估与策略更新不停交互进行框架的一个普遍统称。

表 5.4.1 MC-exploring starts 算法伪代码

伪代码：MC-exploring starts 算法
Initialization：随机初始化策略π_0。 Aim：找到最优状态价值和最优策略。 For each episode Episode generation：在当前策略下，随机选择一个(s,a)出发，并且尽量保证所有(s,a)能被访问。对产生的轨迹进行切割，按照轨迹长度 T：s_0，a_0，r_0，……，s_{T-1}，a_{T-1}，r_{T-1}。 PI 和 PE： Initialization：初始化 g 很小的数。 For 轨迹中切割下来的每一步，从后往前推，$t=T-1,T-2,\ldots,0$。 计算$g \leftarrow \gamma g + r_{t+1}$ 使用 first-visit 方法： If (s,a)没有重复出现 $Returns(s_t,a_t) \leftarrow Returns(s_t,a_t) + g$ $q(s_t,a_t) = \text{average}(Returns(s_t,a_t))$ $\pi(a\|s_t) = 1 \text{ if } a = arg\max_a q(s_t,a)$ 判断策略是否收敛

从上面算法的过程，我们会发现一个致命的问题，我们能够利用单次轨迹就更新策略的前提是，这条轨迹能够遍历到尽可能多的状态-动作对，不然在更新策略到时候，有些最优动作可能会被漏掉。但实际情况是我们不可能在随机环境中获得这一条完美的轨迹，所以这只是理论下表现优异的算法，但实际应用很困难，为了解决 exploring starts 的问题，下文即将介绍 soft-policy 方法。

5.5 MC-epsilon greedy 算法

为了解决 exploring starts 的问题，我们引入 soft-policy 方法。Policy 有两种方式，一种是确定性的，包括 greed-policy，一种是随机性的，包括 soft-policy。

soft 的概念是，对于每一个动作，我们都会有机会去选择它，即动作是探索性的，也就是说从一个或几个出发，在这一段无限长的轨迹中，每个动作都有机会执行，在大量的实验中就能确保所有状态-动作对被覆盖。

soft-Policy 具体的实现之一是 ε-greed 策略，其数学表示如下：

$$\pi(a\,|\,s)=\begin{cases}1-\dfrac{\varepsilon}{|\mathcal{A}(s)|}(|\mathcal{A}(s)|-1), & \text{for the greedy action}\\ \dfrac{\varepsilon}{|\mathcal{A}(s)|}, & \text{for the other}\,|\mathcal{A}(s)|-1\text{actions}\end{cases}\tag{120}$$

其中 ε 取值范围[0,1]，是每个状态的可执行动作数量。当 ε 为 0 时，策略退化为贪婪策略，当 ε 为 1 时，策略变为均匀分布策略。

ε-greed 的思想基于以下现象，前几个算法在动作选择时，只选择动作价值最大的那个动作。但考虑到在策略改进前，我们需要通过从某些状态-动作对出发尽可能遍历到所有状态-动作，然后更新动作价值，问题是（1）确定性策略很难遍历到所有状态-动作对，因为策略往往只选择动作价值高的动作。（2）部分状态-动作对没有被遍历到，导致策略没有更新到最优策略，而是局部最优，这样动作价值会计算错，进而一错再错，陷入循环。因此 ε-greed 的思想是，保留非最优动作的选择概率，同时确保最优动作概率最大，这样在轨迹中有几率遍历到全部状态-动作对，而不只是策略认为的当前局部最优动作。

经过上面的分析，我们已经很清楚 ε-greed 如何改进 MC-basic 算法框架了，在策略更新环节，我们将其他动作的概率从 0 修改为：

$$\pi_{k+1}(a\,|\,s)=\begin{cases}1-\dfrac{|\mathcal{A}(s)|-1}{|\mathcal{A}(s)|}\varepsilon, & a=a_k^*,\\ \dfrac{1}{|\mathcal{A}(s)|}\varepsilon, & a\neq a_k^*.\end{cases}\tag{121}$$

MC-epsilon greedy 算法的过程与 MC-exploring starts 算法几乎一致，只在策略更新环节有微微的改动，它不需要 exploring starts 环节，但同样需要用 ε-greed 访问到所有状态-动作对，从而提高动作价值的计算效率。

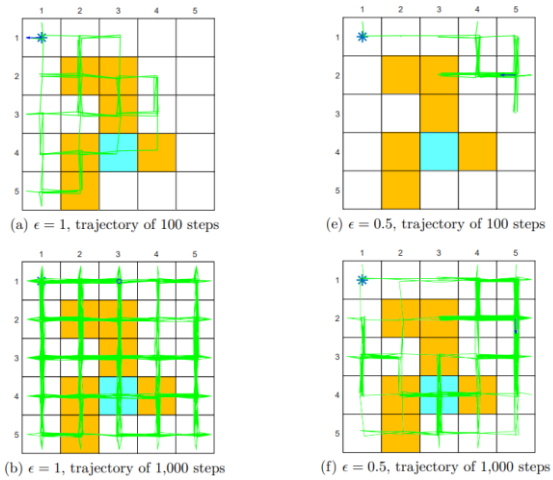
表 5.4.1 MC-epsilon greedy 算法伪代码

伪代码：MC-epsilon greedy 算法
Initialization：随机初始化策略π_0。 Aim：找到最优状态价值和最优策略。 For each episode Episode generation：在当前策略下，随机选择一个(s,a)出发，并且尽量保证所有(s,a)能被访问。对产生的轨迹进行切割，按照轨迹长度 T：s_0，a_0，r_0，……，s_{T-1}，a_{T-1}，r_{T-1}。

伪代码：MC-epsilon greedy 算法
PI 和 PE： Initialization：初始化 g 很小的数。 For 轨迹中切割下来的每一步，从后往前推，t=T-1,T-2,…,0。 计算$g \leftarrow \gamma g + r_{t+1}$ 使用 every-visit 方法： If (s,a)没有重复出现 $Returns(s_t, a_t) \leftarrow Returns(s_t, a_t) + g$ $q(s_t, a_t) = \text{average}(Returns(s_t, a_t))$ $a^* = \arg\max_a q(s_t, a)$ $\pi(a\|s_t) = \begin{cases} 1 - \frac{\|\mathcal{A}(s_t)\|-1}{\|\mathcal{A}(s_t)\|}\varepsilon, a = a^* \\ \frac{1}{\|\mathcal{A}(s_t)\|}\varepsilon, a \neq a^* \end{cases}$
判断策略是否收敛

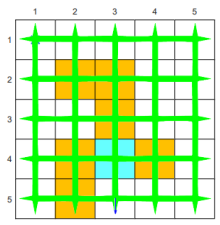
5.6 MC-epsilon greedy 算法用例

用一些例子分析 MC-epsilon greedy 性质。在例 1 中，我们设置 0.5 和 1 两个探索率，通过不断增大步数，分析状态-动作的覆盖情况。对于 epsilon=1 的情况，我们发现它在 10000 步已经覆盖到全部状态-动作对了，并且每个状态-动作对的出现次数都差不多。对于 epsilon=0 的情况，我们发现它在 10000 步虽然也覆盖了全部的状态-动作对，但是覆盖次数差异特别大，有些对出现次数很多，有些对出现次数很少。

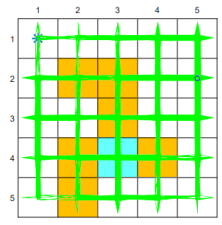


(a) $\epsilon = 1$, trajectory of 100 steps (e) $\epsilon = 0.5$, trajectory of 100 steps

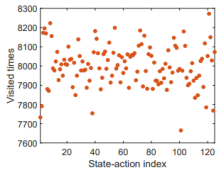
(b) $\epsilon = 1$, trajectory of 1,000 steps (f) $\epsilon = 0.5$, trajectory of 1,000 steps



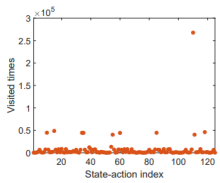
(c) $\epsilon = 1$, trajectory of 10,000 steps



(g) $\epsilon = 0.5$, trajectory of 10,000 steps



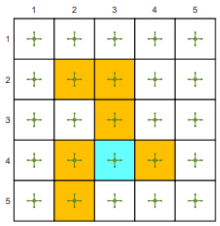
(d) $\epsilon = 1$, number of times each action is visited within 1 million steps



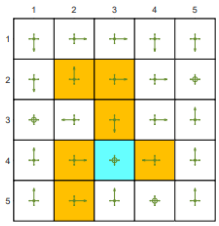
(h) $\epsilon = 0.5$, number of times each action is visited within 1 million steps

图 5.6.1 例 1

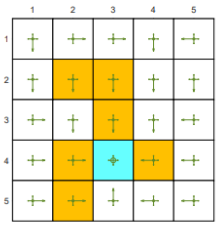
在例 2 中，我们设置单轮的步数为一百万步，每轮后都进行策略改进，可以发现，在两次迭代后，我们就已经找到了最优策略。



(a) Initial policy



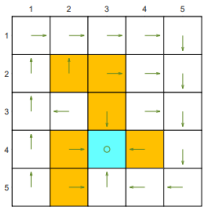
(b) After the first iteration



(c) After the second iteration

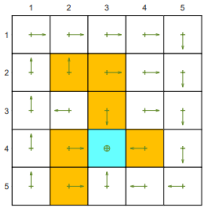
图 5.6.2 例 2

在例 3 中，我们观察不同的探索率下最优策略的状态价值，可以看出随着探索率的增大，最优策略的状态价值是不断减小的。



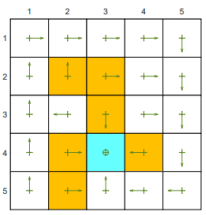
3.5	3.9	4.3	4.8	5.3
3.1	3.5	4.8	5.3	5.9
2.8	2.5	10.0	5.9	6.6
2.5	10.0	10.0	10.0	7.3
2.3	9.0	10.0	9.0	8.1

(a) A given ϵ-greedy policy and its state values: $\epsilon = 0$



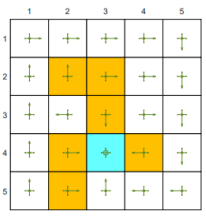
0.4	0.5	0.9	1.3	1.4
0.1	0.0	0.5	1.3	1.7
0.1	-0.4	3.4	1.4	1.9
-0.1	3.4	3.3	3.7	2.2
-0.3	2.6	3.7	3.1	2.7

(b) A given ϵ-greedy policy and its state values: $\epsilon = 0.1$



-2.2	-2.4	-2.1	-1.7	-1.8
-2.5	-3.0	-3.3	-2.3	-2.0
-2.3	-3.3	-2.5	-2.8	-2.2
-2.5	-2.5	-2.8	-2.0	-2.4
-2.8	-3.2	-2.1	-2.3	-2.2

(c) A given ϵ-greedy policy and its state values: $\epsilon = 0.2$

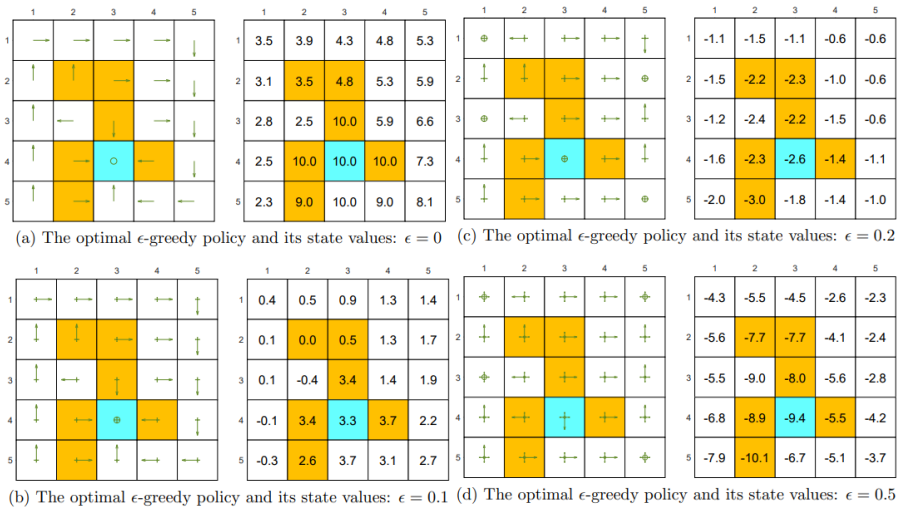


-8.0	-9.0	-8.4	-7.2	-7.8
-8.7	-10.8	-12.4	-9.6	-8.9
-8.3	-12.3	-15.3	-12.3	-10.5
-9.7	-15.5	-17.0	-14.4	-12.2
-10.9	-16.7	-15.2	-14.3	-12.4

(d) A given ϵ-greedy policy and its state values: $\epsilon = 0.5$

图 5.6.3 例 3

从例 4 中可以看出，在探索率为 0 时，我们的得到策略是最优策略。随着探索率逐渐增大，我们得到的策略距离最优策略越来越不一致，在探索率等于 0.5 时，几乎就不是最优策略了。



(a) The optimal ϵ-greedy policy and its state values: $\epsilon = 0$ (c) The optimal ϵ-greedy policy and its state values: $\epsilon = 0.2$

(b) The optimal ϵ-greedy policy and its state values: $\epsilon = 0.1$ (d) The optimal ϵ-greedy policy and its state values: $\epsilon = 0.5$

图 5.6.4 例 4

从例子中我们可以总结出 ε-greed 的优缺点：

- 优点：ε-greed 策略具有强大的探索能力，这样就不需要 exploring starts 严苛的覆盖所有状态-动作对的条件限制。
- 缺点：ε-greed 策略损失了最优性，因为我们总会考虑非最优的动作，但实际上我们可以实现探索与收敛的兼顾，比如 ε 衰减方法。

Chapter6 随机近似与随机梯度下降

6.1 均值估计问题回顾

在 5 章中，我们介绍了第一个 model-free 的算法，它的核心思想就是在无模型条件下，通过试验获得一系列样本，用样本的均值估计期望，在样本足够多时，样本的均值就收敛到期望。

$$\mathbb{E}[X] \approx \bar{x} = \frac{1}{N}\sum_{j=1}^{N} x_j. \tag{122}$$

我们非常关注均值的原因在于，除去计算动作价值期望，还有很多其他涉及到期望计算的部分，在无模型条件下，我们需要通过这种采样的手段，建立未知的环境模型，这是一种最基本的方法，也是强化学习的核心思想——交互，所以在后续的算法或方法中，我们的思考方向就是更高效率利用样本，获取更高质量样本，提高样本计算效率等等，让样本表现出来的性质更贴合真实的未知的模型。

针对式 122，实际上有两种计算方法：

- 当我们收集到所有的样本后，统一计算均值，但在样本收集的时候，我们什么事都不做，这显然浪费了时间。
- 我们收集到样本就进行计算，每多一个样本就重新计算一次，这种方法称为增量式或迭代式，效率明显更高。

我们推导一下方法 2 的数学表达式。不失一般性，设 k 个数据的平均值为：

$$w_{k+1} = \frac{1}{k}\sum_{i=1}^{k} x_i, \quad k = 1, 2, \ldots \tag{123}$$

设 k-1 个数据的平均值为：

$$w_k = \frac{1}{k-1}\sum_{i=1}^{k-1} x_i, \quad k = 2, 3, \ldots \tag{124}$$

显然 k 个数据的计算是建立在 k-1 个数据的基础上，式 122 进行变换：

$$w_{k+1} = \frac{1}{k}\sum_{i=1}^{k} x_i = \frac{1}{k}\left(\sum_{i=1}^{k-1} x_i + x_k\right) = \frac{1}{k}((k-1)w_k + x_k) = w_k - \frac{1}{k}(w_k - x_k) \tag{125}$$

于是，我们就得到了增量表达式：

$$w_{k+1} = w_k - \frac{1}{k}(w_k - x_k) \tag{126}$$

这种方法的优点是我不用等全部数据收集到再计算均值，而是有一个就可以计算一次，虽然前期的均值计算误差比较大，但有总比没有强，随着样本的量增大，这个均值会越来越接近期望，在这个过程中，我们可以将这些均值用在其他任务中。

而且，更一般的，有如下表达式：

$$w_{k+1} = w_k - \alpha_k(w_k - x_k) \tag{127}$$

当$\alpha_k = \frac{1}{k}$时，这个式子就是我们刚刚推到的增量表达式。值得注意的是，在$\alpha_k \neq \frac{1}{k}$时，若满足一些条件时，这个表达式也能逐渐收敛到期望值。这看起来奇怪，因为它不是严格按平均值的定义计算每个数据的算术平均。关键在于增量形式的本质是一种“逐步调整”机制：它通过一个学习率α_k来平衡新样本和旧估计，每一步都根据新数据微调当前值，而不是完全依赖所有历史数据的简单平均。如果新数据波动大，学习率可以控制调整幅度，避免估计值过度偏向最新样本。这样，在长期过程中，估计值会综合考虑历史数据的整体趋势，通过可伸缩的调整稳定地逼近期望，类似于一种平滑的移动平均，而不是僵硬的算术计算。

6.2 RM 算法介绍

前几节我们提到的随机近似（Stochastic approximation）理论，它指的是随机迭代算法这一大类，随机指的是算法涉及到对随机过程的采样，这算法普遍应用于方程求解或优化算法，当然也有其他算法可以进行方程求解或优化问题，比如梯度下降（gradient descent）等。SA 算法的优点是它不需要知道这个方程或目标函数的表达式，就可以进行求解。

Robbins-Monro（RM）算法是随机近似领域非常具有开创性的一个工作，我们常用的 SGD 是 RM 算法的一种特殊情况，我们前面介绍的均值估计算法也是 RM 算法的一种特殊形式。

假设我们要求解如下形式方程的根：

$$g(w)=0 \tag{128}$$

虽然有很多问题的数学形式与式 128 不同，但我们可以通过转换，将其变为求解式 128，例如求解 $J(w)$函数的最小值，我们可以转换成：

$$g(w) \doteq \nabla_w J(w)=0 \tag{129}$$

大多数情况下，如果函数导数存在并且只存在一个最优点，那么这个方程是可以求解的。或者求解：

$$g(w)=c \rightarrow g(w)-c=0 \tag{130}$$

也可以转换成同样的形式。

那么如何求解式 128 的根呢？如果我们已知函数的表达式，自然就会非常多的方法去求解。但现在的问题是，在不知道函数表达式的情况下，如何求解根，RM 算法可以解决这个问题，具有代表性的应用就是神经网络，我们预先不知道函数表达式，但我们可以通过持续调整神经元权重，最终找到一组参数使函数输出为最小。

RM 算法的数学表达式如下：

$$w_{k+1}=w_k-a_k\tilde{g}(w_k,\eta_k), \quad k=1,2,3,\ldots \tag{131}$$

其中w_k是第 k 次对根的估计值。$\tilde{g}(w_k,\eta_k)=g(w_k)+\eta_k$是带噪声函数输出的观测值。$a_k$是一个正的系数。

算法的迭代流程为：首先预设一个w_0，输入给函数，得到观测值，然后通过式 131 计算出w_1，继续输入给函数，依次迭代循环，最终可以收敛到方程的根。

6.3 RM 算法收敛性及应用

在 RM 算法的收敛性进行一个证明。首先给出 RM 定理：在 RM 算法中，如果满足:

1) 对于所有的w，满足

$$0<c_1 \le \nabla_w g(w) \le c_2 \tag{132}$$

2) 满足

$$\sum_{k=1}^{\infty} a_k=\infty \ \ and \ \ \sum_{k=1}^{\infty} a_k^2<\infty \tag{133}$$

3) 满足

$$\mathbb{E}[\eta_k \mid \mathcal{H}_k]=0 \ \ and \ \ \mathbb{E}[\eta_k^2 \mid \mathcal{H}_k]<\infty \tag{134}$$

其中 $\mathcal{H}_k=\{w_k,w_{k-1},\ldots\}$。如果满足以上三个条件，$w_k$会依概率收敛到方程的

解 w^*。

针对条件 1），需要保证$g(w)$导数严格大于 0，但又非无限大，也就是$g(w)$函数必须严格递增。如果$g(w)$是用来求解一个优化问题，那么$g(w)$导数就是优化函数的二阶导，这个条件要求优化问题是凸优化。

针对条件 2），我们需要保证a_k在 k 趋于无穷时收敛到 0，但收敛速度不能太快。

首先，当a_k在 k 趋于无穷时收敛到 0 时，

$$w_{k+1} - w_k = -a_k \tilde{g}(w_k, \eta_k), \tag{135}$$

等式右边趋于 0，左边也趋于 0，确保后期根收敛，至于为什么不是正号，而是负号，可以参考自动控制原理中的负反馈抑制作用，这里的效果也是类似的。当w_k收敛到根，方程解趋于 0，此时观测值由噪声支配。

当满足收敛条件后，我们写出每个w_k的式子并加在一起：

$$w_2 - w_1 = -a_1 \tilde{g}(w_1, \eta_1),\ \ w_3 - w_2 = -a_2 \tilde{g}(w_2, \eta_2),\ \ \ldots \tag{136}$$

$$w_1 - w_\infty = \sum_{k=1}^{\infty} a_k \tilde{g}(w_k, \eta_k) \tag{137}$$

当 k 趋于无穷时，w_k收敛到根，因为a_k求和有界，所以等式右边有界，因此左边也有界，因此根与w_1的距离总是有限的，它告诉我们无论w_1怎么选，这个算法总会收敛。

常见的a_k取值比如 6.1 节的 $1/k$，但现实中我们往往会选择一个较小的常数，而不是一个变数，因为我们希望在后续进来的数据依旧发挥用处，而不是被 $1/k$ 稀释掉。

针对条件 3），需要保证在迭代解中中依照一个概率分布中进行独立同分布采样（iid）后，样本满足期望为 0，方差有限。

回顾 6.1 节的均值估计算法的式子为：

$$w_{k+1} = w_k + \alpha_k (x_k - w_k) \tag{138}$$

a_k取值为 $1/k$，此时能够简单的证明这个式子是收敛的，因为它代表着均值的计算。除去 $1/k$，我们其他取值也可以收敛，正如前文所说，这就是一个特殊的 RM 算法。

考虑一个函数，我们的目标是估计总体的期望，转化为方程形式，w为估计值，可得函数：

$$g(w) \doteq w - \mathbb{E}[X] \tag{139}$$

对期望的观测值是从样本得到的x，进而函数的观测值为：

$$\tilde{g}(w, x) \doteq w - x \tag{140}$$

为了凑出 RM 算法的标准形式，找出噪声的组成：

$$\begin{aligned}\tilde{g}(w,\eta) &= w - x = w - x + \mathbb{E}[X] - \mathbb{E}[X] \\ &= (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta\end{aligned} \tag{141}$$

我们就得到了 RM 算法的形式：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k) \tag{142}$$

这个形式就是我们之前推导出的均值估计算法，因此均值估计算法的收敛性同 RM 算法的收敛性。

（补充）Dvoretzky 定理：考虑一个随机过程

$$w_{k+1} = (1 - \alpha_k) w_k + \beta_k \eta_k \tag{143}$$

其中$\alpha\beta\eta$均为随机变量序列，同时$\alpha\beta$在迭代过程中保持非负。若满足：

- $$\sum_{k=1}^{\infty} \alpha_k = \infty, \sum_{k=1}^{\infty} \alpha_k^2 < \infty; \sum_{k=1}^{\infty} \beta_k^2 < \infty \quad uniformly\ w.p.1; \tag{144}$$
- $$\mathbb{E}[\eta_k \mid \mathcal{H}_k] = 0 \ \text{ and } \ \mathbb{E}[\eta_k^2 \mid \mathcal{H}_k] \le C \quad w.p.1 \tag{145}$$

则w_k依概率收敛导 0。这个定义是比 RM 定理更普遍的定理，它也可以用来分析均值估计算法的收敛性，同时它也可以用来分析未来即将介绍的 Q-learing 和 TD-learning。

6.4 SGD 算法介绍

SGD 算法是在机器学习领域和强化学习领域广泛使用的算法之一，它是 RM 算法的一种特殊情况，而均值估计算法又是它的一种特殊情况。

定义一个优化问题的函数表达式如下：

$$\min_w J(w) = \mathbb{E}[f(w, X)] \tag{146}$$

其中w是被优化的变量。X 是一个随机变量。我们的目标是找到最优的w使函数$J(w)$的值最小。

对上面这个优化问题的求解方法有如下几种，每一个方法都是依次递进的。

- 方法 1：梯度下降（gradient descent，GD）

$$w_{k+1} = w_k - \alpha_k \nabla_w \mathbb{E}[f(w_k, X)] = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] \tag{147}$$

我们将优化问题转换为求导数为 0 的问题。这里的期望符号可以提进去。这个方法最致命的问题是，它需要求期望，除非我们知道问题模型，否则没有办法计算，因此需要换个方法。

- 方法 2：批次梯度下降（batch gradient descent，BGD）

$$\mathbb{E}[\nabla_w f(w_k, X)] \approx \frac{1}{n}\sum_{i=1}^{n}\nabla_w f(w_k, x_i) \tag{148}$$

$$w_{k+1} = w_k - \alpha_k \frac{1}{n}\sum_{i=1}^{n}\nabla_w f(w_k, x_i) \tag{149}$$

BGD 的思想与均值估计的思想是一致的，我们通过样本均值来估计总体期望，只要样本足够多，样本均值就趋近于总体期望，但这个方法需要很多样本进行迭代才能减少误差。

- 方法 3：随机梯度下降（stochastic gradie descent，SGD）

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) \tag{150}$$

相比于 GD 方法，我们将需要模型的那部分替换为采样。相比于 BGD 需要很多样本来计算，这里我们用一个采样的x_k的值来替代，这会存在误差，但不影响算法收敛。

6.5 SGD 算法收敛性及应用

我们举一个优化问题的例子：

$$\min_w J(w) = \mathbb{E}[f(w, X)] = \mathbb{E}\left[\frac{1}{2}\| w - X \|^2\right] \tag{151}$$

$f(w, X)$的梯度为：

$$\nabla_w f(w, X) = w - X \tag{152}$$

- 求这个优化问题的解。

$$\begin{aligned}&\min_w J(w) = \mathbb{E}[f(w, X)] \to \nabla_w \mathbb{E}[f(w, X)] = 0 \to \\ &\mathbb{E}[\nabla_w f(w, X)] = 0 \to \mathbb{E}[w - X] = 0 \to w - \mathbb{E}[X] = 0 \to w = \mathbb{E}[X]\end{aligned} \tag{153}$$

- 用 GD 算法求解这个优化问题的解。

$$\begin{aligned}w_{k+1} &= w_k - \alpha_k \nabla_w J(w_k) = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] \\ &= w_k - \alpha_k \mathbb{E}[w_k - X]\end{aligned} \tag{154}$$

- 用 SGD 算法求解这个优化问题的解。

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) = w_k - \alpha_k (w_k - x_k) \tag{155}$$

我们下面证明一下 SGD 的收敛性。从 GD 到 SGD，我们有如下式子：

$$w_{k+1} = w_k - \alpha_k \mathbb{E}[\nabla_w f(w_k, X)] \to w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) \tag{156}$$

其中$\nabla_w f(w_k, x_k)$是对样本进行采样得到的梯度，而不是基于模型。

我们之前已经说明了 SGD 是一个特殊的 RM 算法，既然如此，可以利用 RM 的性质去证明 SGD 的收敛性。一个 SGD 算法的目标是最小化如下函数：

$$J(w) = \mathbb{E}[f(w, X)] \tag{157}$$

将这个最小化问题转换为导数求根问题：

$$\nabla_w J(w) = \mathbb{E}[\nabla_w f(w, X)] = 0 \tag{158}$$

为了求这个方程，令：

$$g(w) = \nabla_w J(w) = \mathbb{E}[\nabla_w f(w, X)] \tag{159}$$

现在，我们的算法目标就是找到$g(w) = 0$的根。自然，我们用 RM 算法来求解这个方程，令：

$$\begin{aligned}\tilde{g}(w,\eta) &= \nabla_w f(w,x) \\ &= \underbrace{\mathbb{E}[\nabla_w f(w,X)]}_{g(w)} + \underbrace{\nabla_w f(w,x) - \mathbb{E}[\nabla_w f(w,X)]}_{\eta}\end{aligned} \tag{160}$$

有了观测值，写出 RM 求解方程：

$$w_{k+1} = w_k - a_k \tilde{g}(w_k, \eta_k) = w_k - a_k \nabla_w f(w_k, x_k) \tag{161}$$

我们发现这就是 SGD 的形式，也就是 SGD 是一个特殊的 RM 算法。我们只要保证 SGD 满足 RM 的条件，即可证明收敛性。

下面给出 SGD 的收敛性定理：

1) $0 < c_1 \le \nabla_w^2 f(w, X) \le c_2$（同 RM，RM 是 g 的一阶导）

2) $\sum_{k=1}^{\infty} a_k = \infty$ and $\sum_{k=1}^{\infty} a_k^2 < \infty$

3) $\{x_k\}_{k=1}^{\infty}\ is\ iid$（独立同分布）

6.6 SGD 算法有趣的性质

SGD 将确定梯度替换为随机梯度，我们不由自主的思考一个问题，SGD 的收敛是否也是缓慢或随机进行的？为了回答这个问题，考虑随机梯度和确定梯度之间的相对误差：

$$\delta_k \doteq \frac{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}{|\mathbb{E}[\nabla_w f(w_k, X)]|} \tag{162}$$

首先假设w^*为最优解，注意到：

$$\mathbb{E}[\nabla_w f(w^*), X)] = 0 \tag{163}$$

我们将式 162 写为：

$$\delta_k = \frac{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}{|\mathbb{E}[\nabla_w f(w_k, X)] - \mathbb{E}[\nabla_w f(w^*, X)]|} \tag{164}$$

对分母，由拉格朗日中值定理得：

$$\delta_k = \frac{|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]|}{|\mathbb{E}[\nabla_w^2 f(\tilde{w}_k, X)(w_k - w^*)]|} \tag{165}$$

其中$\tilde{w}_k \in [w_k, w^*]$。

假设f是严格凸的，它满足：

$$\nabla_w^2 f \geq c > 0 \tag{166}$$

我们将式 165 的分母进行变换，由于$(w_k - w^*)$不是随机变量，因此提出期望符号；绝对值符号分别提到两边，由式 166 可得：

$$\begin{aligned} \left|\mathbb{E}[\nabla_w^2 f(\tilde{w}_k, X)(w_k - w^*)]\right| &= \left|\mathbb{E}[\nabla_w^2 f(\tilde{w}_k, X)](w_k - w^*)\right| \\ &= \left|\mathbb{E}[\nabla_w^2 f(\tilde{w}_k, X)]\right|\left|(w_k - w^*)\right| \geq c\,|\,w_k - w^*\,| \end{aligned} \tag{167}$$

式 165 具有如下不等式：

$$\delta_k \leq \frac{\left|\nabla_w f(w_k, x_k) - \mathbb{E}[\nabla_w f(w_k, X)]\right|}{c\,|\,w_k - w^*\,|} \tag{168}$$

注意到：

$$\delta_k \leq \frac{|\overbrace{\nabla_w f(w_k, x_k)}^{\text{stochastic gradient}} - \overbrace{\mathbb{E}[\nabla_w f(w_k, X)]}^{\text{true gradient}}|}{\underbrace{c\,|\,w_k - w^*\,|}_{\text{distance to the optimal solution}}} \tag{169}$$

这里的结论是当w_k距离解较远时，随机梯度和确定梯度的方向是一致的，因为我们能通过实际的梯度变化确定大致方向。当w_k距离解较近时，随机梯度由于其动量特性，就像二阶系统震荡响应一样，会在最优解附近震荡，此时的方向是不确定的。

关于 SGD 还有一个很有趣的问题，在我们上述将的 SGD 算法中，我们都涉及了随机变量，但是很多其他类型的公式介绍中，SGD 并没有随机变量的情况。为什么会出现这种情况，用一个例子解释。

考虑如下优化问题：

$$\min_w J(w) = \frac{1}{n}\sum_{i=1}^{n} f(w, x_i), \tag{170}$$

其中x_i为一个随机数集合，这个集合不是任何随机变量的采样。求解这个问题的 GD 算法的计算式为：

$$w_{k+1} = w_k - \alpha_k \nabla_w J(w_k) = w_k - \alpha_k \frac{1}{n}\sum_{i=1}^{n} \nabla_w f(w_k, x_i) \tag{171}$$

假设这个x_i集合非常庞大，我们每次只取一个数。因此可以用如下迭代算法;

$$w_{k+1} = w_k - \alpha_k \nabla_w f(w_k, x_k) \tag{172}$$

看到上面这个式子，我们想知道式 172 是不是 SGD 算法？同时，我们想知道x_i集合需不需要排列，需不需要重复使用，需不需要采样呢？

假设 X 是一个定义在x_i集合上的随机变量，它取集合中任何一个数的概率都

一样：

$$p(X=x_i)=1/n \tag{173}$$

于是，式 169 的优化问题自然就变成了一个与随机变量有关的函数：

$$\min_{w} \ J(w)=\frac{1}{n}\sum_{i=1}^{n} f(w,x_i)=\mathbb{E}[f(w,X)] \tag{174}$$

也就是说，求解刚刚那个优化问题的算法自然就是 SGD 算法，也就是说有的问题没有随机的变量引入，但我们

依旧可以人为创造一个平均采样的模型，然后利用 SGD 算法进行求解，同时集合的抽取是随机的。

6.7 SGD 算法对比

在本节我们对常见的 BGD、MBGD 和 SGD 算法进行对比。同样，我们定义一个优化问题$J(w)=\mathbb{E}[f(w,X)]$和一个 X 的随机采样序列$\{x_i\}_{i=1}^{n}$，三个算法的迭代公式如下：

$$\begin{aligned} w_{k+1}&=w_k-\alpha_k\frac{1}{n}\sum_{i=1}^{n}\nabla_w f(w_k,x_i),(\text{BGD}) \\ w_{k+1}&=w_k-\alpha_k\frac{1}{m}\sum_{j\in\mathcal{I}_k}\nabla_w f(w_k,x_j),(\text{MBGD}) \\ w_{k+1}&=w_k-\alpha_k\nabla_w f(w_k,x_k).(\text{SGD}) \end{aligned} \tag{175}$$

- BGD 将进行大量采样，然后将所有采样的值一起计算均值。MBGD 则是进行大量采样，选择一部分的采样值一起计算均值。SGD 则是随机抽取$\{x_i\}_{i=1}^{n}$中的值作为均值。
- 相比于 SGD，MBGD 的随机性更小，因为它用了很多的样本值，而不只是一个。
- 相比与 BGD，MBGD 不需要使用全部的样本值，使得它更有效，弹性更大。
- 如果采样数量为 1，MBGD 变为 SGD。
- 如果采样数量为 n，MBGD 不一定是 BGD，因为 BGD 是全部的样本，MBGD 是采样 n 个样本，可能存在重复采样的情况。

Chapter7 时序差分方法

7.1 时序差分引入

在 5 章中我们介绍了第一个 model-free 的蒙特卡洛方法，在本章我们将介绍第二个 model-free 的时序差分（temporal-difference，TD）方法，与蒙特卡洛相比，TD 方法是一种增量式方法，它基于随机近似理论，比蒙特卡洛更具有优势。

我们回顾一下 RM 算法的求解过程，首先考虑一个均值估计问题：

$$w = \mathbb{E}[X] \tag{176}$$

令$g(w) = w - \mathbb{E}[X]$，转为求解以下方程的根：

$$g(w) = 0 \tag{177}$$

由于我们只能观测到样本 x，观测值为：

$$\tilde{g}(w,\eta) = w - x = (w - \mathbb{E}[X]) + (\mathbb{E}[X] - x) \doteq g(w) + \eta \tag{178}$$

写出迭代方程：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k (w_k - x_k) \tag{179}$$

在上面的均值估计问题中，我们引入函数$v(X)$，求解过程不变，只需要修改部分公式：

$$\begin{aligned} g(w) &= w - \mathbb{E}[v(X)] \\ \tilde{g}(w,\eta) &= w - v(x) = (w - \mathbb{E}[v(X)]) + (\mathbb{E}[v(X)] - v(x)) \doteq g(w) + \eta \end{aligned} \tag{180}$$

迭代方程为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k [w_k - v(x_k)] \tag{181}$$

在函数的基础上，我们再引入一些随机变量 R 和参数 γ，同样的求解过程不变，只需要修改部分公式：

$$w = \mathbb{E}[R + \gamma v(X)] \tag{182}$$

$$\begin{aligned} g(w) &= w - \mathbb{E}[R + \gamma v(X)] \\ \tilde{g}(w,\eta) &= w - [r + \gamma v(x)] \\ &= (w - \mathbb{E}[R + \gamma v(X)]) + (\mathbb{E}[R + \gamma v(X)] - [r + \gamma v(x)]) \\ &\doteq g(w) + \eta. \end{aligned} \tag{183}$$

迭代方程为：

$$w_{k+1} = w_k - \alpha_k \tilde{g}(w_k, \eta_k) = w_k - \alpha_k [w_k - (r_k + \gamma v(x_k))] \tag{184}$$

是不是看着很熟悉，其实这就是状态价值的计算公式。

7.2 时序差分算法

TD 方法通常指的是 RL 算法的一类，它也可以特指估计状态价值的一种特殊的算法。首先，我们定义在策略 π 下产生的经验序列：

$$(s_0, r_1, s_1, \ldots, s_t, r_{t+1}, s_{t+1}, \ldots) \text{ or } \{(s_t, r_{t+1}, s_{t+1})\}_t \tag{185}$$

TD 算法的迭代方程如下：

$$\begin{aligned} v_{t+1}(s_t) &= v_t(s_t) - \alpha_t(s_t)\left[v_t(s_t) - \left[r_{t+1} + \gamma v_t(s_{t+1})\right]\right] && (1) \\ v_{t+1}(s_t) &= v_t(s_t), \forall s \neq s_t && (2) \end{aligned} \tag{186}$$

- 其中$v_t(s_t)$是在对策略 π 下第 t 时刻对状态 s_t 的状态价值$v_\pi(s_t)$的估计。$\alpha_t(s_t)$是的第 t 时刻状态 s_t 的学习率。
- 在迭代时，只有状态 s_t 的状态价值更新，其他状态保持不变。
- 解释一下，首先 s_t 代表智能体在 t 时刻时的状态。（1）式就是告诉我们，对 t 时刻智能体的状态的状态价值的估计，（2）式告诉我们，在更新 s_t 的状态价值的时候，其他状态价值保持不变，这个公式一般都是隐含的。

对式（185）的（1）进行详细解释：

$$\underbrace{v_{t+1}(s_t)}_{new\ estimate} = \underbrace{v_t(s_t)}_{current\ estimate} - \alpha_t(s_t)\left[\overbrace{v_t(s_t) - \left[\underbrace{r_{t+1} + \gamma v_t(s_{t+1})}_{TD\ \text{target}}\right]}^{TD\ \text{error}}\right] \tag{187}$$

上式可以总结为新的估计等于旧的估计加上学习率乘于一个修正项。

- 其中：

$$\bar{v}_t \doteq r_{t+1} + \gamma v(s_{t+1}) \tag{188}$$

被称为 TD target。原因在于，我们的式 187 实际上就是让我们的估计值越来越接近 TD target，证明如下：

$$\begin{aligned}
&v_{t+1}(s_t) = v_t(s_t) - \alpha_t(s_t)\left[v_t(s_t) - \bar{v}_t\right] \\
&\Rightarrow v_{t+1}(s_t) - \bar{v}_t = v_t(s_t) - \bar{v}_t - \alpha_t(s_t)\left[v_t(s_t) - \bar{v}_t\right] \\
&\Rightarrow v_{t+1}(s_t) - \bar{v}_t = [1 - \alpha_t(s_t)][v_t(s_t) - \bar{v}_t] \\
&\Rightarrow |v_{t+1}(s_t) - \bar{v}_t| = |1 - \alpha_t(s_t)||v_t(s_t) - \bar{v}_t|
\end{aligned} \tag{189}$$

因为学习率是一个小于 1 的数，因此：

$$0 < 1 - \alpha_t(s_t) < 1 \tag{190}$$

因此，每一次迭代，就会距离 TD target 更近。

$$|v_{t+1}(s_t) - \bar{v}_t| \le |v_t(s_t) - \bar{v}_t| \tag{191}$$

其中：

$$\delta_t \doteq v(s_t) - [r_{t+1} + \gamma v(s_{t+1})] = v(s_t) - \bar{v}_t \tag{192}$$

被称为 TD error。原因在于两个时间步之间是有差异的，而它描述了当前步的估计值与策略 π 下真实值之间的距离。我们假设式 189 都是确定策略的值：

$$\delta_{\pi,t} \doteq v_\pi(s_t) - [r_{t+1} + \gamma v_\pi(s_{t+1})] \tag{193}$$

计算其期望值，我们发现这就是贝尔曼公式：

$$\mathbb{E}[\delta_{\pi,t} \mid S_t = s_t] = v_\pi(s_t) - \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s_t\right] = 0 \tag{194}$$

如果$v_t = v_\pi$，那么 TD error 就是 0。因此，如果 TD error 不为 0，它就是描

述了v_t与v_π之间的差异。同时，TD error 也是一种新的信息，如果我在当前有一个估计值v_t，这个估计值是不准确的，然后我采样了一个新的经验，我将其与我的估计进行计算，计算出一个差异值，说明当前估计不准确，我就可以用这个经验来更新我的估计，作为一种新的信息。

注意：

- TD 算法能够在给定策略的条件下，估计状态价值。
- TD 算法不能估计动作价值。
- TD 算法不能找到最优策略。

7.3 时序差分算法收敛性

在 7.2 节我们介绍了 TD 算法，并给出 TD 算法的直观解释，它用来估计给定策略下的状态价值，但它在数学上究竟干了一些什么事，实际上就是求解了给定策略下的贝尔曼方程。

首先定义一种贝尔曼公式的新形式：

$$v_\pi(s) = \mathbb{E}\left[R + \gamma G \mid S = s\right], \quad s \in \mathcal{S} \tag{195}$$

因为期望的叠加性，将未来趋势部分展开：

$$\mathbb{E}[G \mid S = s] = \sum_a \pi(a \mid s) \sum_{s'} p(s' \mid s, a) v_\pi(s') = \mathbb{E}[v_\pi(S') \mid S = s] \tag{196}$$

其中S'就是跳转的下一个状态。我们将式 195 写成更简洁的形式：

$$v_\pi(s) = \mathbb{E}\left[R + \gamma v_\pi(S') \mid S = s\right], \quad s \in \mathcal{S} \tag{197}$$

式 197 是贝尔曼方程的另一种表达，这个方程也叫做贝尔曼期望方程。然后，我们利用 RM 算法求解式 197，定义函数;

$$g(v(s)) = v(s) - \mathbb{E}\left[R + \gamma v_\pi(S') \mid s\right] \tag{198}$$

转换为方程根问题：

$$g(v(s)) = 0 \tag{199}$$

此时我们只能观测奖励、状态，观测值为：

$$\begin{aligned} \tilde{g}(v(s)) &= v(s) - \left[r + \gamma v_\pi(s')\right] \\ &= \underbrace{\left(v(s) - \mathbb{E}\left[R + \gamma v_\pi(S') \mid s\right]\right)}_{g(v(s))} + \underbrace{\left(\mathbb{E}\left[R + \gamma v_\pi(S') \mid s\right] - \left[r + \gamma v_\pi(s')\right]\right)}_{\eta} \end{aligned} \tag{200}$$

最后，给出 RM 算法的迭代方程：

$$\begin{aligned} v_{k+1}(s) &= v_k(s) - \alpha_k \tilde{g}(v_k(s)) \\ &= v_k(s) - \alpha_k (v_k(s) - [r_k + \gamma v_\pi(s_k')]) \end{aligned} \tag{201}$$

其中$v_k(s)$是对状态价值在第 k 步的估计，r_k，s_k'是样本的在第 k 步的观测

值。在式 201 中，我们需要关注两个需求：

1) 我们必须有关于$\{(s,r,s')\}$的很多经验，与 MC 的回合不同的是，我需要从同一个状态出发，获得奖励，跳转到下个状态，重复这个过程。
2) 这个式子中，需要知道下一个状态的状态价值。

针对问题 1，我们不去针对一个状态不停的采样，而是随机从一个状态出发，获得一个轨迹，然后在轨迹中获取我们想要的那个状态，如果有，那就更新，如果没有，那就不更新。

针对问题 2，我们将$v_\pi(s_k')$替换为$v_k(s_k')$，同样对下一个状态的状态价值进行估计。

TD 算法的收敛性定理与 RM 算法基本一致，需要注意的是：

- TD 算法解决的是给定策略下状态价值估计的问题。
- TD 算法的学习率通常选择一个较小的常数。因为我们不希望后面进来的样本的重要性被忽略。

现在我们给出之前讲过的蒙特卡洛方法和 TD 方法的对比表。

表 7.3.1 对比表

特性	蒙特卡洛方法	时序差分方法
核心机制	使用实际回报 G_t 作为更新目标	使用估计的 TD 目标作为更新目标，引入偏差
更新时机	离线：必须等到回合结束才能更新	在线：每一步都可以用经验立即更新
更新方法	价值估计更需要依赖于历史估计	直接估计价值
偏差/方差	依赖于一整条轨迹的随机动作和状态转移，随机性大，方差高。但它是真实回报的无偏估计	TD 目标只依赖单步采样，方差显著低于 MC。但它是有偏估计
收敛性	对函数近似不稳定	通常比 MC 更高效，在函数近似下也常能收敛
环境要求	必须存在有终止状态的回合制任务	可以应用于连续无终止的任务
初值敏感	不敏感。因为更新使用的是实际回报，与初始值无关	敏感。最终的估计值会受到初始估计值的影响
效率	较低，需要等待整个回合，数据利用效率低	较高，数据立即被用于更新，学习更快
比喻	考完才知道全部结果	学一点，考一点，及时反馈

7.4 Sarsa 算法

前面介绍的 TD 算法只能估计给定策略下的状态价值，而我们接下来介绍的 Sarsa 算法能够直接估计动作价值，我们可以将这个过程认为是 PE，再结合 PI 过程，我们就能求解最优策略了。

首先假设我们有一系列经验$\{(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})\}_t$。

为了求解动作价值，Sarsa（State-action-reward-state-action）算法的迭代公式为：

$$\begin{aligned} q_{t+1}(s_t,a_t) &= q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - [r_{t+1} + \gamma q_t(s_{t+1},a_{t+1})]\right], \\ q_{t+1}(s,a) &= q_t(s,a), \quad \forall (s,a) \neq (s_t,a_t), \end{aligned} \tag{202}$$

可以发现这个公式与 TD 算法形式一致，只是将对 s 的状态价值估计替换成了对（s,a）的动作价值估计。

Sarsa 算法与 TD 算法一样，都在求解贝尔曼方程，但是方程的形式不一样。Sarsa 解决的是：

$$q_\pi(s,a) = \mathbb{E}\left[R + \gamma q_\pi(S', A') \mid s, a\right] \tag{203}$$

因为这里的策略是给定的，所以下一个状态是确定的。

Sarsa 算法的收敛性定理与 TD 算法一致。它告诉我们如何在给定策略的条件下，估计动作价值，与 PI 过程结合后，即可求解最优策略。不过我们经常看到的 Sarsa 算法一般都默认结合了 PI 过程，是一个完整的求解最优策略的算法。

表 7.4.1 Sarsa 算法伪代码

伪代码：Sarsa 算法
For each episode If 当前 s_t 不是终点状态 在策略 π_t 下收集一系列经验$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$。 Update q-value： $q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)[q_t(s_t,a_t) - [r_{t+1} + \gamma q_t(s_{t+1},a_{t+1})]]$ Update Policy： $a^* = arg\max_a q(s_t,a)$ $\pi(a\|s_t) = \begin{cases} 1 - \frac{\|\mathcal{A}(s_t)\| - 1}{\|\mathcal{A}(s_t)\|}\varepsilon, a = a^* \\ \frac{1}{\|\mathcal{A}(s_t)\|}\varepsilon, a \neq a^* \end{cases}$ 判断策略是否收敛

关于 Sarsa 算法：

- 相比于 MC 需要收集到大量数据才更新动作价值，Sarsa 算法只要收集到一个数据就立马更新策略，这种思想基于更普遍的策略迭代。
- 同时，Sarsa 保留了 ε-greed 思想，以平衡探索和利用。
- Sarsa 算法的核心观点很简单，解决给定策略下贝尔曼公式的求解问题。

我们用网格例子应用一下 Sarsa 算法，与以往的任务不同，我们这次设定的

任务是从一个设定的起点开始，找到到终点的最优策略。

从例 1 中左图可以看出，不是所有的状态都是最优策略。从右图可以看出每个回合的总奖励值被频繁的使用。

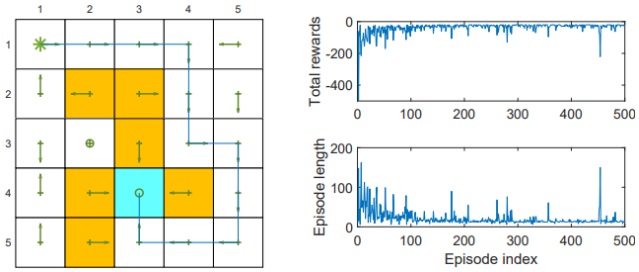


图 7.4.1 例 1

7.5 Expected Sarsa 和 n-step Sarsa 算法

Expected Sarsa 算法的迭代方程如下：

$$\begin{aligned} q_{t+1}(s_t,a_t) &= q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - (r_{t+1} + \gamma \mathbb{E}[q_t(s_{t+1},A)])\right], \\ q_{t+1}(s,a) &= q_t(s,a), \quad \forall (s,a) \neq (s_t,a_t), \end{aligned} \tag{204}$$

其中：

$$\mathbb{E}[q_t(s_{t+1},A)]) = \sum \pi_t(a \mid s_{t+1}) q_t(s_{t+1},a) \doteq v_t(s_{t+1}) \tag{205}$$

相比于 Sarsa，它的 TD target 改变成了式 205，因此它减少了随机变量，使得不需要那么多经验，从而减少了计算量、

从数学上看，Expected Sarsa 同样通过随机近似求解了一个贝尔曼方程，其形式如下：

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a\right] \tag{206}$$

它求解了这个贝尔曼方程：

$$q_\pi(s,a) = \mathbb{E}\left[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a\right] \tag{207}$$

n-step Sarsa 算法结合了 Sarsa 算法和 MC 算法的特点，我们通过下面可以直观看出这三个算法的区别：

$$\begin{aligned} G_t^{(1)} &= R_{t+1} + \gamma q_\pi(S_{t+1},A_{t+1}), && \to \text{Sarsa} \\ G_t^{(2)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 q_\pi(S_{t+2},A_{t+2}), \\ &\vdots \\ G_t^{(n)} &= R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n q_\pi(S_{t+n},A_{t+n}), && \to \text{n-step Sarsa} \\ &\vdots \\ G_t^{(\infty)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots && \to \text{MC} \end{aligned} \tag{208}$$

显然，MC 算法通过计算整个轨迹得到了精确的动作价值。Sarsa 算法只计算一步。n-step Sarsa 算法则是平衡了计算长度。

关于求解方程的区别：

$$
\begin{aligned}
&q_\pi(s,a)=\mathbb{E}[G_t^{(1)}\mid s,a]=\mathbb{E}[R_{t+1}+\gamma q_\pi(S_{t+1},A_{t+1})\mid s,a].\\
&q_\pi(s,a)=\mathbb{E}[G_t^{(n)}\mid s,a]=\mathbb{E}[R_{t+1}+\gamma R_{t+2}+\cdots+\gamma^n q_\pi(S_{t+n},A_{t+n})\mid s,a]\\
&q_\pi(s,a)=\mathbb{E}[G_t^{(\infty)}\mid s,a]=\mathbb{E}[R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\ldots\mid s,a].
\end{aligned}
\tag{209}
$$

表 7.5.1 算法对比

特性	Sarsa	Expected Sarsa	n-step Sarsa
更新目标	一步 TD 目标，用一步动作价值近似状态价值	一步期望 TD 目标，用一步平均动作价值近似状态价值	n 步实际回报，用 n 步回报近似状态价值
策略类型	同策略：行为策略和目标策略相同	可同策也可异策：行为策略和目标策略可以不同。当目标策略为贪婪策略时，它就变成了 Q-learning。	同策略：行为策略和目标策略相同
偏差/方差	中等偏差，中等方差： • 偏差：因使用估计的 Q 值而有偏 • 方差：依赖于单次动作采样，有一定方差	低方差： • 通过对下一个状态所有动作的期望消除了因动作选择带来的方差，方差显著低于 Sarsa。 • 仍有因状态转移和奖励带来的偏差和方差。	偏差-方差权衡： • n=1 时：同 Sarsa（高偏差，低方差） • n→∞时：同蒙特卡洛（无偏，高方差） • 通过调整 n 在偏差和方差间取得平衡
更新时机	在每一步后立即更新	在每一步后立即更新	必须等到 n 步之后才能更新前 n 步的状态动作对
学习效率/速度	稳定的在线学习	通常比 Sarsa 更快更稳定：由于方差更低，通常能获得更稳定、更快的收敛	比单步 Sarsa 更快，因为每次更新包含更多信息。但需要等待 n 步，有轻微延迟
安全性	更安全、更保守：会考虑到探索带来的风险（可能探索到危险动作），策略会更谨慎	如果目标策略是贪婪的，则与 Q-learning 一样乐观；如果目标策略是 ε-greedy 的，则与 Sarsa 一样考虑探索风险	由于是同策略，会考虑探索带来的风险，策略相对保守。
计算复杂度	低：只需一次查表和计算	中：需要计算下一个状态所有可能动作的期望值（动作空间大时成本高）	中：需要存储 n 步的轨迹，并在 n 步后执行更新
比喻	“摸着石头过河”：根据下一步实际要踩的石头来调整当前步伐。	“纵观全局再落脚”：考虑下一步所有可能落脚点的平均质量，再调整当前步伐。	“走几步看一步”：先走 n 步，然后回头看看这 n 步的整体效果，再来调整最初的步伐。

7.6 Q-learning 算法

相比于 TD 算法估计状态价值，Sarsa 算法估计动作价值，Q-learing 算法则是不需要 PE 过程，因为它直接估计了最优动作价值。

Q-learning 算法的迭代公式为：

$$q_{t+1}(s_t,a_t)=q_t(s_t,a_t)-\alpha_t(s_t,a_t)\left[q_t(s_t,a_t)-[r_{t+1}+\gamma\max_{a\in\mathcal{A}}q_t(s_{t+1},a)]\right],$$
$$q_{t+1}(s,a)=q_t(s,a),\quad \forall(s,a)\neq(s_t,a_t) \tag{210}$$

可以看出 Q-learning 与 Sarsa 非常像，但是他们在 TD target 上有区别，Sarsa 是立即奖励+下一个策略执行动作的价值，Q-learning 是立即奖励+下一最优动作的价值。

在求解方程上，Sarsa 和 TD 都在求解贝尔曼方程，而 Q-learning 则是在求解一个贝尔曼最优方程。

$$q(s,a)=\mathbb{E}\left[R_{t+1}+\gamma\max_a q(S_{t+1},a)\mid S_t=s,A_t=a\right] \tag{211}$$

接下来我们可以介绍一些有意思的性质或概念：

- Behavior Policy：智能体与环境交互得到一些经验。
- Target Policy：对策略的持续更新和优化。
- On-policy：当 Behavior Policy 与 Target Policy 一致的时候，称此类学习叫 On-policy。比如我首先初始化一个策略，然后与环境交互得到经验，然后立马用这个经验改进策略，重复上述过程。常见算法有 Sarsa。
- Off-policy：当 Behavior Policy 与 Target Policy 不一致的时候，称此类学习叫 Off-policy。常见算法有 Q-learning，因为它可以不同也可以相同。

分辨一个算法是 On-policy 还是 Off-policy，首先检查它的算法在数学上进行了什么样的操作，其次，检查这个算法需要什么，它解决了什么问题。（本质上，你通过某个策略获得经验，通过经验改进策略，如果这两个策略是同一个，那么就是 On-policy，简单来说就是用自己收集的经验来改进自己）。

我们分析 Sarsa 算法。首先，Sarsa 算法求解了一个给定策略的贝尔曼方程。其次，算法需要经验序列为$\{(s_t,a_t,r_{t+1},s_{t+1},a_{t+1})\}_t$，其中$(s_t,a_t)$给定了，得到的奖励和下一个状态即给定了，只有$a_{t+1}$是由策略 π 决定的。因此，Sarsa 是一个 On-policy 算法，它首先根据策略采集经验，然后用经验更新动作价值，然后立马更新策略，其中策略 π 是 Behavior Policy 又是 Target Policy。

我们分析 MC 算法。首先，MC 算法求解了一个给定策略的贝尔曼方程。其次，算法需要使用该策略下的样本，去计算动作价值，基于这个动作价值，去改进策略，因此 MC 算法是一个 On-policy 算法。

我们分析 Q-learning 算法。首先，Q-learning 算法求解了一个没有任何策略的贝尔曼最优方程。其次，算法通过计算最优动作价值，选择最优的动作作为策略，它的行为策略是随机动作，它的目标策略是最优动作，因此 Q-learning 算法是一个 Off-policy 算法。

表 7.5.2 算法对比

特性	Sarsa	蒙特卡洛方法	Q-learning
行为策略	ε-greedy 策略	任意策略（可以是 ε-greedy、随机策略等）	任意策略（通常是 ε-greedy）
目标策略	ε-greedy 策略	同策略 MC：与行为策略相同 异策略 MC：通常为贪婪策略	贪婪策略
On/Off-policy	On-policy	可 On 可 Off • 同策略 MC：On-policy • 异策略 MC：Off-policy	Off-policy
更新时机	单步在线更新 每一步后立即更新	回合结束更新 必须等待整个回合结束	单步在线更新 每一步后立即更新
学习效率	中等 在线学习，数据效率中等	较低 需要完整回合，数据效率低	较高 在线学习，数据效率高
比喻	"摸着石头过河" 根据下一步实际要踩的石头调整	"事后诸葛亮" 等到结局才知道好坏	"理想主义者" 假设下一步总是最优选择

7.7 Q-learning 例子

表 7.7.1 Q-learning 算法（On-policy）伪代码

伪代码：Q-learning 算法
For each episode If 当前 s_t 不是终点状态 在策略 π_t 下收集一系列经验$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$。 Update q-value： $q_{t+1}(s_t, a_t) = q_t(s_t, a_t) - \alpha_t(s_t, a_t)\left[q_t(s_t, a_t) - [r_{t+1} + \gamma \max_{a \in \mathcal{A}} q_t(s_{t+1}, a)]\right]$ Update Policy π_t： $a^* = arg\max_a q(s_t, a)$ $\pi(a\|s_t) = \begin{cases} 1 - \frac{\|\mathcal{A}(s_t)\| - 1}{\|\mathcal{A}(s_t)\|}\varepsilon, a = a^* \\ \frac{1}{\|\mathcal{A}(s_t)\|}\varepsilon, a \neq a^* \end{cases}$ 判断策略是否收敛

表 7.7.2 Q-learning 算法（Off-policy）伪代码

伪代码：Q-learning 算法
For each episode If 当前 s_t 不是终点状态 在策略 π_b 下收集一系列经验$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1})$。

伪代码：Q-learning 算法
Update q-value： $q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)\left[q_t(s_t,a_t) - [r_{t+1} + \gamma \max_{a\in\mathcal{A}} q_t(s_{t+1},a)]\right]$ Update Policy： $a^* = arg\max_a q(s_t,a)$ $\pi_{T,t+1}(a\|s_t) = \begin{cases} 1 \text{ if } a = arg\max_a q_{t+1}(s_t,a) \\ 0 \text{ otherwise} \end{cases}$ 判断策略是否收敛

表 7.7.3 Q-learning 的 On-policy 和 Off-policy

特性	Off-policy Q-learning (标准)	On-policy 变体 (类似 Sarsa)
学习目标	直接学习最优策略（贪婪策略）	学习当前执行策略（通常是 ε-greedy）
数据利用	高：可以从任何策略收集的数据中学习，可重用历史数据	低：只能从当前策略产生的数据中学习
探索策略	行为策略可自由设计，与学习目标无关	探索策略直接影响学习结果，需谨慎设计
安全性	较低：可能学习到在探索时危险的策略	较高：会考虑探索带来的风险，策略更保守
收敛速度	通常较快：直接朝着最优策略学习	较慢：需要逐步改进当前策略
稳定性	较低：可能因 max 操作产生高估计	较高：更新更平滑，波动较小
适用场景	• 数据宝贵的场景 • 需要从专家数据中学习 • 多智能体系统中的集中式训练 • 安全的模拟环境	• 安全性要求高的实际系统 • 机器人控制 • 在线学习场景 • 探索成本高的环境
实际案例	• Deep Q-Networks (DQN) • 大多数游戏 AI • 推荐系统	• 无人机飞行控制 • 自动驾驶决策

7.8 TD 算法的统一形式

所有的 TD 算法都具有同一个表达形式：

$$q_{t+1}(s_t,a_t) = q_t(s_t,a_t) - \alpha_t(s_t,a_t)[q_t(s_t,a_t) - \bar{q}_t] \tag{212}$$

表 7.8.1 TD target 对比

Algorithm	**Expression of the TD target $\bar{q}_t$ in (7.20)**
Sarsa	$\bar{q}_t = r_{t+1} + \gamma q_t(s_{t+1},a_{t+1})$
n-step Sarsa	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \cdots + \gamma^n q_t(s_{t+n},a_{t+n})$
Q-learning	$\bar{q}_t = r_{t+1} + \gamma \max_a q_t(s_{t+1},a)$
Monte Carlo	$\bar{q}_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \ldots$

所有的算法都可以视作求解贝尔曼公式的随机近似算法。

表 7.8.2 求解公式对比

Algorithm	Equation to be solved
Sarsa	BE: $q_\pi(s,a)=\mathbb{E}\left[R_{t+1}+\gamma q_\pi(S_{t+1},A_{t+1})\middle\vert S_t=s,A_t=a\right]$
n-step Sarsa	BE: $q_\pi(s,a)=\mathbb{E}[R_{t+1}+\gamma R_{t+2}+\cdots+\gamma^n q_\pi(S_{t+n},A_{t+n})\vert S_t=s,A_t=a]$
Q-learning	BOE: $q(s,a)=\mathbb{E}\left[R_{t+1}+\gamma\max_a q(S_{t+1},a)\middle\vert S_t=s,A_t=a\right]$
Monte Carlo	BE: $q_\pi(s,a)=\mathbb{E}[R_{t+1}+\gamma R_{t+2}+\gamma^2 R_{t+3}+\ldots\vert S_t=s,A_t=a]$

Chapter8 值函数近似

8.1 曲线拟合

截止到目前为止，我们计算状态价值和动作价值都是基于表格的形式，也就是离散形式，状态空间和动作空间的维度都是有限的。

表 8.1 表格表示

	a_1 (upward)	a_2 (rightward)	a_3 (downward)	a_4 (leftward)	a_5 (still)
s_1	0	0.5	0.5	0	0
s_2	0	0	1	0	0
s_3	0	0	0	1	0
s_4	0	1	0	0	0
s_5	0	0	1	0	0
s_6	0	0	1	0	0
s_7	0	1	0	0	0
s_8	0	1	0	0	0
s_9	0	0	0	0	1

这种表格表达形式的优点是我们能够很直观的去分析和理解，但它的缺点是很难处理大的状态或动作空间，或者连续的状态-动作空间，在这种情况下，它很难记录所有的状态或动作对，泛化能力也很差。

为了引入曲线拟合的思想，我们先举一个例子。考虑一些一维的状态 $\{s_1,\ldots,s_n\}$，这些状态对应的状态价值为 $v(s_1)$，我们假设状态空间的维度很大，并在下图中画出了状态-状态价值的散点图。现在的问题是，如果我直接存储这些点，需要大量的存储单元，那么有没有简单的方法能够表示这些点呢？利用回归分析中的思路，曲线拟合，利用函数拟合这些点，函数的参数量是不多的，如果能够找到一条函数表示这些点，那么只需要存储函数的参数即可，这大大降低了存储量。

答案当然是可以的，最简单的拟合曲线就是直线。假设直线方程具有如下的形式：

$$\hat{v}(s,w)=as+b=\underbrace{[s,1]}_{\phi^T(s)}\underbrace{\begin{bmatrix}a\\b\end{bmatrix}}_{w}=\phi^T(s)w \tag{213}$$

其中，w是参数向量，$\phi(s)$是特征向量，$\hat{v}(s,w)$是关于参数向量的直线。

这种拟合的好处是：（1）我们之前需要一张表来存储所有的状态价值，现在我们只需要存储函数的两个参数。（2）每当我们想使用状态价值时，我们直接带

入状态，计算函数值即可。(3)虽然这种方式很方便，但实际上我们损失了精度，这是一种对状态价值的近似。

当然，状态与状态价值的关系通常情况下并非线性，所以直线拟合的精度不高，我们可以使用一个二次函数来进行拟合，假设二次函数具有如下形式：

$$\hat{v}(s,w)=as^2+bs+c=\underbrace{\left[s^2,s,1\right]}_{\phi^T(s)}\underbrace{\begin{bmatrix}a\\b\\c\end{bmatrix}}_{w}=\phi^T(s)w \tag{214}$$

利用二次函数增加了参数的维度，但精度也随之提升。其中函数对于状态是非线性的，对于参数是线性的。我们可以继续增加函数的次数，以提高拟合的精度，但同时参数的维度也会上升，过拟合的风险也会随之出现。

总结一下，关于状态和动作价值的近似，我们采用的是一个参数函数$\hat{v}(s,w)$进行近似。它的优点是降低了数据的保存维度，同时增加了数据的泛化性。

8.2 目标函数

我们利用函数拟合价值的出发点是，我们希望函数值与真实值越接近越好，进而，我们需要做的是找到一组最优参数w，使得函数值对每个状态 s 的状态价值拟合度很高。这显然是一个策略评估问题，我们可以从两步出发，第一步是定义一个目标函数，用一个指标来评估拟合程度，第二步是利用算法最优化目标函数，从而找到那组最优参数。

定义目标函数为：

$$J(w)=\mathbb{E}[(v_\pi(S)-\hat{v}(S,w))^2] \tag{215}$$

我们的目标是找到最优参数w最小化$J(w)$。这里面 S 是一个随机变量，对随机变量求期望，我们需要知道 S 的概率分布，这对于初学者是非常困惑的，我们怎么去知道 S 的概率分布呢，以下有一些方法。

第一个方法非常简单，视作平均概率。我们 将每个状态概率视作一致，那么概率就是$1/|\mathcal{S}|$。目标函数就可以写成：

$$J(w)=\mathbb{E}[(v_\pi(S)-\hat{v}(S,w))^2]=\frac{1}{|\mathcal{S}|}\sum_{s\in\mathcal{S}}(v_\pi(s)-\hat{v}(s,w))^2 \tag{216}$$

这个目标函数存在一个很明显的问题，就是它将所有的状态的重要性一直对待，但实际情况中，我们往往更希望关注重要的状态，比如更靠近终点的状态，远离终点的状态自然重要性就小一点。

有了这个想法，我们引入第二个概率分布模型：马尔科夫过程的平稳分布。在马尔科夫过程中，策略经过一段时间后会平稳，假设在这种情况下，我进行一个回合，状态 s 出现的次数为 1，状态总数为 n，那么状态 s 的概率就是 $1/n$，概

率越大，说明该状态出现次数越多，重要性自然就越高。令平稳分布概率模型为 $\{d_\pi(s)\}_{s\in\mathcal{S}}$，这个模型依赖于策略 π，于是目标函数变为：

$$J(w)=\mathbb{E}[(v_\pi(S)-\hat{v}(S,w))^2]=\sum_{s\in\mathcal{S}}d_\pi(s)\left(v_\pi(s)-\hat{v}(s,w)\right)^2 \tag{217}$$

用一个例子说明一下平稳分布：

我们给定一个策略 π，让策略执行很多步，收集四个状态出现的次数。可以看出，随着步数的进行，四个状态均逐渐平稳收敛，占比最大的是状态 4，因为它是终点状态，不管是位于其他那个状态，都有机会走到终点状态。在这种情况下，我们定义四个状态的平稳分布模型：

$$d_\pi(s)\approx\frac{n_\pi(s)}{\sum_{s'\in\mathcal{S}}n_\pi(s')} \tag{218}$$

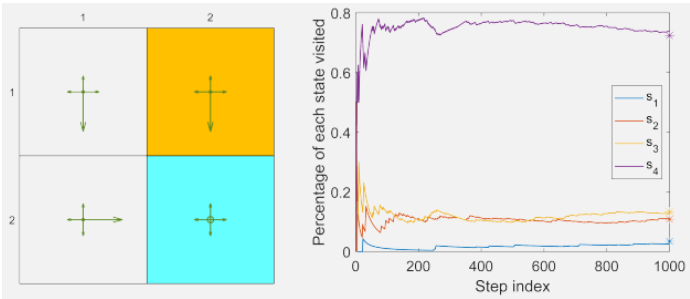


图 8.2.1 例 1

当然，我们也可以直接通过具体的策略求得状态的概率：

$$d_\pi^T=d_\pi^T P_\pi \tag{219}$$

例 1 中的状态转移概率矩阵为：

$$P_\pi=\begin{bmatrix}0.3 & 0.1 & 0.6 & 0\\ 0.1 & 0.3 & 0 & 0.6\\ 0.1 & 0 & 0.3 & 0.6\\ 0 & 0.1 & 0.1 & 0.8\end{bmatrix} \tag{220}$$

计算出特征向量为：

$$d_\pi=\left[0.0345,0.1084,0.1330,0.7241\right]^T \tag{221}$$

这个与我们实际的仿真结果是一致的。

8.3 优化算法

建立了目标函数后，我就要求解这个目标函数，我们在第六章已经介绍了求解最小化函数的方法，例如梯度下降算法。

梯度下降的迭代方程如下：

$$w_{k+1}=w_k-\alpha_k\nabla_w J(w_k) \tag{222}$$

其中函数的梯度为：

$$\begin{aligned}\nabla_w J(w) &= \nabla_w \mathbb{E}[(v_\pi(S)-\hat{v}(S,w))^2] \\ &= \mathbb{E}[\nabla_w (v_\pi(S)-\hat{v}(S,w))^2] \\ &= 2\mathbb{E}[(v_\pi(S)-\hat{v}(S,w))(-\nabla_w \hat{v}(S,w))] \\ &= -2\mathbb{E}[(v_\pi(S)-\hat{v}(S,w))\nabla_w \hat{v}(S,w)]\end{aligned} \tag{223}$$

同样的，这又是一个需要带期望求解梯度式子，我们没有模型，所以没有办法求解真实的梯度。因此，我们需要用随机梯度下降方法（将-2 视作学习率的一部分）。

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t), \tag{224}$$

其中涉及到随机变量的，均需要替换成采样。现在式 224 依旧无法计算，因为我们不知道真实的状态价值，我们需要用近似值代替真是的状态价值，有两种方法：

1) 蒙特卡洛方法：我们通过采样获得一系列轨迹，然后求平均值获得状态价值，式 224 变为：

$$w_{t+1} = w_t + \alpha_t (g_t - \hat{v}(s_t, w_t))\nabla_w \hat{v}(s_t, w_t) \tag{225}$$

2) TD 方法：我们采用 TD 目标作为状态价值的近似，式 224 变为：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)\right]\nabla_w \hat{v}(s_t, w_t) \tag{226}$$

表 8.3.1 TD learning with func_appr 算法伪代码

伪代码：TD learning with func_appr 算法
Initialization：建立一个关于参数w的函数。初始化初始参数w_0。 Aim：拟合给定策略下的真是状态价值 For 策略 π 产生的每个轨迹 For every Step(s_t, r_{t+1}, s_{t+1}) 通常情况下： $w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma\hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t)]\nabla_w \hat{v}(s_t, w_t)$ 在线性函数下： $w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma\phi^T(s_{t+1})w_t - \phi^T(s_t)w_t]\phi(s_t)$

对于拟合函数的选择是一个非常重要的问题。第一个思路是选择一个线性的函数：

$$\hat{v}(s,w) = \phi^T(s)w \tag{227}$$

其中$\phi^T(s)$为特征向量，可以是多项式基，傅里叶基等等。

第二个思路是，选择神经网络作为一个非线性拟合函数，输入的是状态，输出的是估计的状态价值，需要优化的是神经网络的参数。

针对线性的情况，可以得到拟合函数的导数为：

$$\nabla_w \hat{v}(s,w) = \phi(s) \tag{228}$$

TD 算法的迭代方程为：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t) \right] \nabla_w \hat{v}(s_t, w_t) \tag{229}$$

替换掉导数部分：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \phi^T(s_{t+1}) w_t - \phi^T(s_t) w_t \right] \phi(s_t) \tag{230}$$

这种算法也叫做 TD-linear。它的缺点非常明显，我很难选择一个特征向量使得拟合效果好，但是它的优点是理论性质非常好，能够直接数学分析，另外我们可以认为表格表达是线性拟合函数的一种特殊情况。为什么呢？

首选，考虑一个关于状态 s 的特殊的特征向量：

$$\phi(s) = e_s \in \mathbb{R}^{|\mathcal{S}|} \tag{231}$$

这个向量中，对应每个 s 的位置是 1，其他是 0。

定义拟合函数为：

$$\hat{v}(s,w) = e_s^T w = w(s) \tag{232}$$

其中$w(s)$表示向量w中的第 s 个元素。也就是说，每当有一个状态 s 被输入，就选择对应位置上的值输出，这种形式就是表格的形式。

8.4 示例与分析

考虑一个 5*5 的网格世界。对于每个状态，策略为$\pi(a|s) = 0.2$。我们的目标是估计这个策略下的状态价值，边界惩罚和障碍物惩罚为-1，目标奖励为+1，折扣因子为 0.9。b 图展示了通过模型求得的状态价值，c 图根据状态价值的表格绘制了三维曲面。

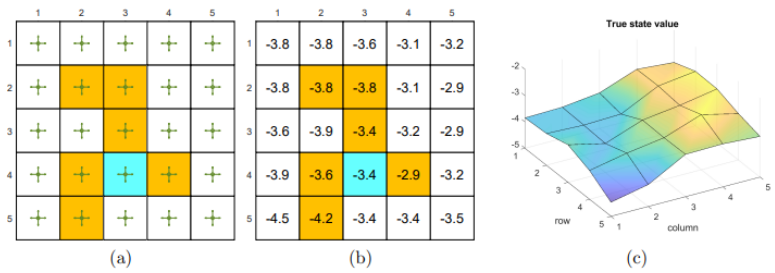


图 8.4.1 例 1

我们通过 TD 算法进行状态价值的估计，可以看出随着轮次的进行，状态价值误差越来越低，最终的结果与模型计算结果高度吻合。

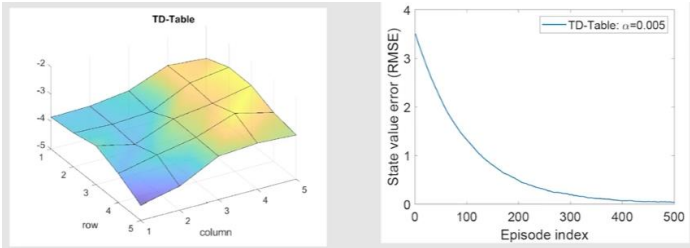


图 8.4.2 例 2

在我们使用 TD-linear 之前，我们需要确定特征向量的形式，也就是对应三维空间中的曲面，最简单的三维曲面就是平面。

$$\phi(s)=\begin{bmatrix}1\\x\\y\end{bmatrix}\in\mathbb{R}^3 \tag{233}$$

在这种情况下，拟合函数的形式为：

$$\hat{v}(s,w)=\phi^T(s)w=[1,x,y]\begin{bmatrix}w_1\\w_2\\w_3\end{bmatrix}=w_1+w_2x+w_3y. \tag{234}$$

从例 3 中可以看出结果，平面虽然成功的拟合了状态价值的变化趋势，但是毕竟它是一个平面，局部具体的形状差异还是非常大，实际的曲面都是不平滑的，难以用平面拟合。

我们用更高阶更复杂的曲面进行拟合，选择特征向量为：

$$\phi(s)=[1,x,y,x^2,y^2,xy]^T\in\mathbb{R}^6 \tag{235}$$

在这种情况下，拟合函数的形式为：

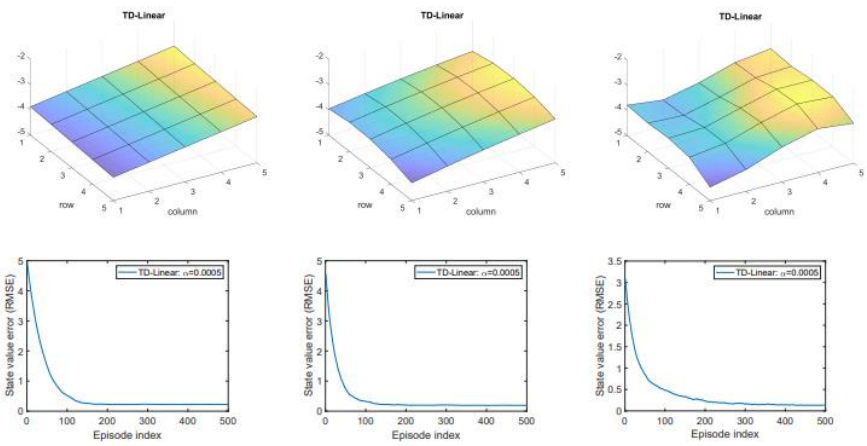
$$\hat{v}(s,w)=\phi^T(s)w=w_1+w_2x+w_3y+w_4x^2+w_5y^2+w_6xy \tag{236}$$

从例 3 中可以看出结果，6 维的特征向量不仅成功的拟合了状态价值的变化趋势，在有一些地方也有起伏，体现了最小值和最大值点。

我们再次增加曲面复杂度，选择特征向量为;

$$\phi(s)=[1,x,y,x^2,y^2,xy,x^3,y^3,x^2y,xy^2]^T\in\mathbb{R}^{10}. \tag{237}$$

从例 3 中可以看出结果，10 维的特征向量的拟合结果几乎与真实模型一致了。从上面的结果可以看出，随着参数的增多，拟合的效果也逐渐变好，但是这种拟合程度不是无限增加的，如果参数非常多，这种方式与表格形式就没有什么区别了，并且拟合误差也不是无限缩小，泛化能力会变差。



(a) $\phi(s) \in \mathbb{R}^3$ (b) $\phi(s) \in \mathbb{R}^6$ (c) $\phi(s) \in \mathbb{R}^{10}$

图 8.4.3 例 3

现在，我们已经完全的介绍完了如何利用 TD 方法进行状态价值估计函数拟合。

1) 第一步：建立目标函数：

$$J(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] \tag{238}$$

2) 第二步：利用梯度下降算法：

$$w_{t+1} = w_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t, w_t)) \nabla_w \hat{v}(s_t, w_t) \tag{239}$$

3) 第三步：由于模型未知，真实梯度无法求解，我们将真实值换为 TD 目标值：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{v}(s_{t+1}, w_t) - \hat{v}(s_t, w_t) \right] \nabla_w \hat{v}(s_t, w_t) \tag{240}$$

尽管这个过程非常清晰，但是式子数学上确实存在缺陷，因为我们定义的 TD 误差用是用两个近似值相减，这是否代表真实值的差异有待讨论。我们现在可以给出结论，TD-linear 并不是在最小化目标函数，并不是在最小化真实价值与估计价值的误差。

我们有多种目标函数的表达：

1) 真实价值误差

$$J_E(w) = \mathbb{E}[(v_\pi(S) - \hat{v}(S,w))^2] = \| \hat{v}(w) - v_\pi \|_D^2 \tag{241}$$

2) 贝尔曼误差

$$J_{BE}(w) = \| \hat{v}(w) - (r_\pi + \gamma P_\pi \hat{v}(w)) \|_D^2 \doteq \| \hat{v}(w) - T_\pi(\hat{v}(w)) \|_D^2 \tag{242}$$

3) 投影贝尔曼误差

$$J_{PBE}(w) = \| \hat{v}(w) - M T_\pi(\hat{v}(w)) \|_D^2 \tag{243}$$

TD-linear 实际上是在最小化投影贝尔曼误差，因为拟合函数维度限制的原

因，它并不能完全的与真实平面一致，所以它只能在真实平面在特征维度上的投影上进行拟合。

8.5 Sarsa 和 Q-learning

至此，我们仅仅考虑了状态价值估计的问题，但为了寻找到最优策略，我们还必须估计到动作价值。

Sarsa 算法拟合函数的公式：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t) \tag{244}$$

相比于 TD 估计状态价值，Sarsa 只是将其中的状态价值替换成了动作价值。

表 8.5.1 Sarsa with func_appr 算法伪代码

伪代码：Sarsa with func_appr 算法
Initialization：建立一个关于参数w的函数。初始化初始参数w_0。 Aim：拟合给定策略下的真实动作价值 For 每个回合 If 当前状态不是目标状态 在策略 π 下采取动作得到一些系列轨迹 Value update： $w_{t+1} = w_t + \alpha_t[r_{t+1} + \gamma\hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t)]\nabla_w\hat{q}(s_t, a_t, w_t).$ Policy update： $a^* = \arg\max_a q(s_t, a)$ $\pi(a\|s_t) = \begin{cases} 1 - \frac{\|\mathcal{A}(s_t)\|-1}{\|\mathcal{A}(s_t)\|}\varepsilon, a = a^* \\ \frac{1}{\|\mathcal{A}(s_t)\|}\varepsilon, a \neq a^* \end{cases}$

例 1 展示了 Sarsa with func_appr 算法的运行结果，可以看出在 200 步之后，算法和奖励值和回合长度都稳定收敛了，最终的策略也是最优策略。

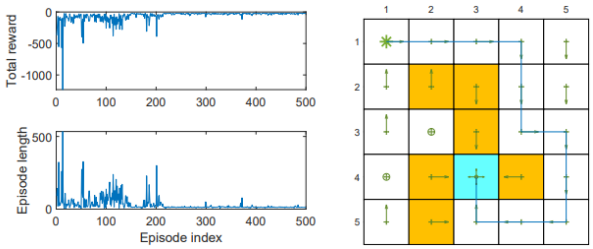


图 8.5.1 例 1

Q-learning 算法拟合函数的公式：

$$w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in \mathrm{A}(s_t+1)} \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t) \tag{245}$$

表 8.5.2 Q-learning with func_appr 算法（On-Policy）伪代码

伪代码：Q-learning with func_appr 算法
Initialization：建立一个关于参数w的函数。初始化初始参数w_0。 Aim：拟合给定策略下的真实动作价值 For 每个回合 If 当前状态不是目标状态 在策略 π 下采取动作得到一些系列轨迹 Value update： $w_{t+1} = w_t + \alpha_t \left[r_{t+1} + \gamma \max_{a \in A(s_t+1)} \hat{q}(s_{t+1}, a_{t+1}, w_t) - \hat{q}(s_t, a_t, w_t) \right] \nabla_w \hat{q}(s_t, a_t, w_t)$ Policy update： $a^* = \arg\max_a q(s_t, a)$ $\pi(a\|s_t) = \begin{cases} 1 - \frac{\|\mathcal{A}(s_t)\|-1}{\|\mathcal{A}(s_t)\|}\varepsilon, a = a^* \\ \frac{1}{\|\mathcal{A}(s_t)\|}\varepsilon, a \neq a^* \end{cases}$

例 2 展示了 Q-learning with func_appr 算法的运行结果，可以看出在训练初期没多久步，算法和奖励值和回合长度就稳定收敛了，最终的策略也是最优策略。

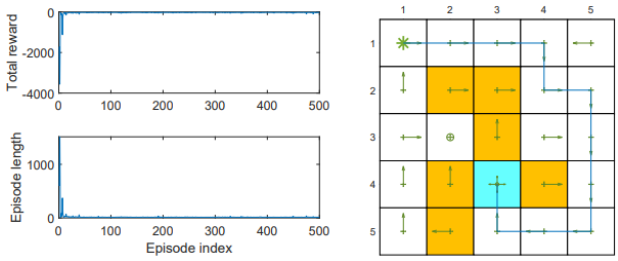


图 8.5.2 例 2

8.6 DQN-basic

Deep Q-network 是最早也是最成功将神经网络应用在强化学习中的算法，自然神经网络扮演的角色是一个非线性拟合函数，与线性拟合方法不同的是，DQN 不仅仅在这方面做了工作，而且也解决了强化学习与深度学习结合的问题。

DQN 的目标函数为：

$$J(w) = \mathbb{E}\left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w) - \hat{q}(S, A, w)\right)^2\right] \tag{246}$$

它实际上在最小化贝尔曼最优误差：

$$q(s,a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a) \middle| S_t = s, A_t = a\right], \quad \forall s, a \tag{247}$$

设计完目标函数后，DQN 采用梯度下降进行求解：

$$J(w)=\mathbb{E}\left[\left(R+\gamma\max_{a\in\mathcal{A}(S')}\hat{q}(S',a,w)-\hat{q}(S,A,w)\right)^2\right] \tag{248}$$

这里求解梯度时，另一个梯度好求，但 max 操作的梯度不好求解，DQN 首先进行了如下定义：

$$y\doteq R+\gamma\max_{a\in\mathcal{A}(S')}\hat{q}(S',a,w) \tag{249}$$

DQN 在求解梯度时，暂时将 y 中的w视为常数，这样就只需求解另一个梯度了。于是，DQN 建立了两个结构一致的神经网络：一个是主网络，代表$\hat{q}(S,A,w)$，一个是目标网络，代表$\hat{q}(S,A,w_T)$，修改目标函数为：

$$J=\mathbb{E}\left[\left(R+\gamma\max_{a\in\mathcal{A}(S')}\hat{q}(S',a,w_T)-\hat{q}(S,A,w)\right)^2\right] \tag{250}$$

$$\nabla_w J=\mathbb{E}\left[\left(R+\gamma\max_{a\in\mathcal{A}(S')}\hat{q}(S',a,w_T)-\hat{q}(S,A,w)\right)\nabla_w\hat{q}(S,A,w)\right] \tag{251}$$

这个式子的更新过程为：主网络是每轮都更新的网络，通过学习，不断调整，每隔几轮，主网络将其参数复制给目标网络，然后继续更新，再复制给目标网络，依次迭代，最终收敛。这个观点也是 DQN 最核心最基础的思想。

8.7 DQN-Experience

Experience replay 是 DQN 的非常重要的一个方法，DQN 在收集到一些经验后，不是按照收集顺序进行采样训练，而是将 tamen 存储起来，放在经验池里，每当神经网络需要训练时，从经验池里按照均匀概率取出来。

为什么要均匀采样呢？因为在样本获取时，我们不知道获取服从的概率分布，按照均匀采样的话，重要性损失是最少的。同时，为了打破样本之间的相关性，我将所有样本都放在经验池中均匀采样，这样避免样本顺序干扰。

问题：为什么表格 Q-learning 不需要经验回放？

回答：想象一下，表格 Q-learning 的学习过程就像在一个巨大的地图（Q 表）上给每个位置（状态-动作对）做标记。每当你亲自走到一个地方并发现了一条新路，你就能立刻在地图上更新这个位置的标记。你的学习是“即时的”和“局部的”。因为地图（Q 表）是离散的，更新北京到天津的路况，并不会影响上海到杭州的路况。这种“格子间相互独立”的特性，使得你只需要最新的现场样本就足够了，就像在现场立一个“此路不通”或“此路畅通”的牌子，之前的旧样本就像过时的现场照片，对你的即时标记工作帮助不大，直接丢弃也无妨。

问题：为什么 DQN 需要需要概率分布？

回答：而 DQN 的学习过程，则像是用一个巨大的、可弯曲的神经网络来

拟合整个地球的表面。这个网络是“连续的”和“相互关联”的。当你更新北京的路况信息时，这个调整会像涟漪一样影响到模型对上海地形的判断。如果你连续输入一百个“北京堵车”的样本，你的神经网络就会产生严重的“偏见”，认为全世界都在堵车，从而忘记了之前学到的“上海高架很畅通”的知识。这就像用一支笔画画，如果总在同一个地方反复涂抹，其他地方的颜色就会被覆盖掉。因此，DQN 必须从经验回放库中均匀采样，获取一个多样化的、有代表性的“世界路况概率分布”，才能保证每一笔绘画（每一次更新）都是在协调地描绘整个全局地图，而不是在局部疯狂涂抹导致模型“失忆”。

问题：表格 Q-learning 可以使用经验回放吗？

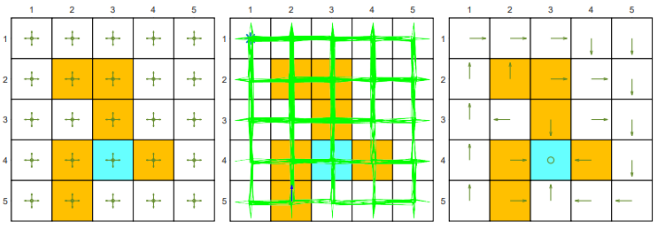
回答：当然可以！虽然表格 Q-learning 不像 DQN 那样依赖经验回放，但使用它就像一位探险家开始写探险日记。他不再仅仅依赖现场立牌子，而是把每天的探险经历（样本）都详细记录下来。当他某天在营地休息时，可以翻看日记，重新品味和消化过去的经历。通过反复阅读（回放）那些稀有且宝贵的经历——比如“如何在丛林里找到水源”或“如何躲避猛兽”——他就能更牢固地掌握这些关键时刻的生存技能。这打破了“现场学习”的时空限制，尤其对于处理那些不常发生但至关重要的关键事件，写日记（使用经验回放）能让表格 Q-learning 的学习过程更加高效和平稳。

8.8 DQN 代码与例子

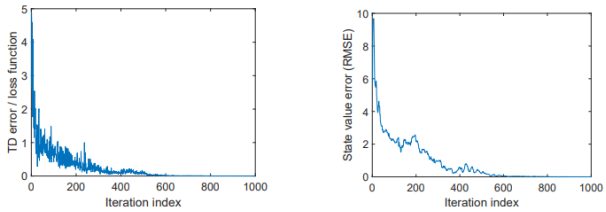
表 8.5.2 DQN 算法（Off-Policy）伪代码

伪代码：DQN 算法
Aim：学习到最优目标网络以拟合最优动作价值。 For 每次迭代 　从经验池中均匀采样 　For 每个样本 　　计算目标价值$y \doteq R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a, w_T)$ 　　使用样本更新主网络最小化目标函数 　Set $w_T = w$每隔一段迭代次数

例 1 展示了 1000 步的 DQN 训练结果，发现大部分状态动作都能覆盖到，最终的策略也是最优策略。损失函数和状态价值都正常收敛。



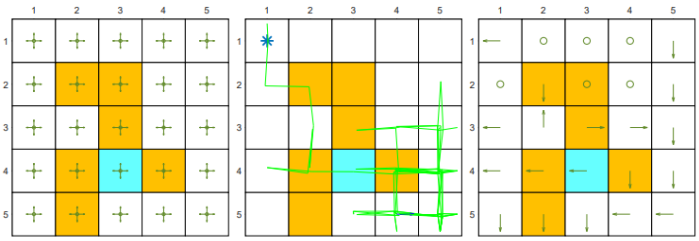
(a) The behavior policy. (b) An episode with 1,000 steps. (c) The final learned policy.



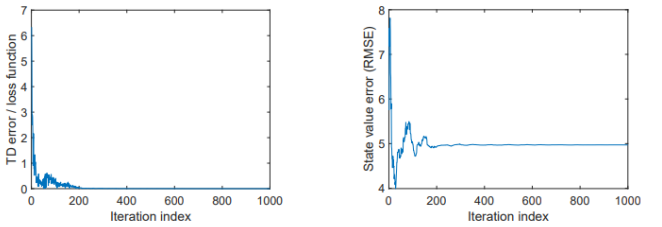
(d) The loss function converges to zero. (e) The value error converges to zero.

图 8.8.1 例 1

例 2 展示了 100 步的 DQN 训练结果，发现大部分状态动作都不能覆盖到，最终的策略也不是最优策略。损失函数和状态价值虽然正常收敛，但没有收敛到 0，说明数据还是非常重要的。



(a) The behavior policy. (b) An episode with 100 steps. (c) The final learned policy.



(d) The loss function converges to zero. (e) The value error does not converge to zero.

图 8.8.2 例 2

表 8.5.3 算法对比

特性	Q-learning	Sarsa	DQN
函数类型	线性函数	线性函数	深度神经网络
策略类型	Off-policy	On-policy	Off-policy
表征能力	有限 • 只能学习线性特征组合 • 无法表征非线性关系	有限 • 线性限制	强大 • 可学习高度非线性 • 能处理复杂特征交互
收敛性	理论上可能发散 • 线性函数近似+Off-policy 可能不稳定	理论上可能发散 • 但比 Q-learning 稍稳定	经验上有效，但理论不保证 • 实际中通过技巧稳定

特性	Q-learning	Sarsa	DQN
样本效率	中等 • 可重用经验，但特征有限	较低 • 需当前策略的新样本	高 • 经验回放充分重用数据
稳定性	较低 • 数据分布变化大 • 可能振荡	中等 • 数据分布相对一致	中等 • 通过目标网络和经验回放稳定
工程需求	高依赖 • 需要精心设计特征函数	高依赖 • 特征质量决定性能	低依赖 • 神经网络自动学习
计算复杂速度	低 • 矩阵运算简单 • 更新快速	低 • 同左，计算高效	高 • 需要 GPU 训练 • 前向传播和反向传播
内存需求	低 • 只存储权重向量	低 • 同左	高 • 存储网络参数+经验回放池
参数敏感	中等 • 学习率、特征设计很关键	中等 • 类似 Q-learning	高 • 学习率、网络结构、回放大小等众多超参数
适用规模	中小型问题 • 状态空间维度适中	中小型问题 • 同左	大规模复杂问题 • 高维状态空间
安全性	风险较高 • 忽略探索风险	较安全 • 考虑探索后果	风险较高 • 同 Q-learning，忽略探索风险
应用场景	• 资源有限的嵌入式系统 • 需要快速原型验证 • 状态特征明确的问题	• 机器人控制 • 安全性要求高的场景 • 在线学习系统	• 游戏 AI • 自动驾驶 • 复杂决策系统

Chapter9 策略梯度方法

9.1 基本思路

在第 8 章中，我们介绍了利用函数去拟合价值，目标函数关于价值，同样的，策略也可以用函数进行拟合，这里的目标函数是策略的函数。

在之前的例子中，我们用表格来表示策略，例 1 中的表格的每一格都代表在当前状态选择动作的概率，这个表示的优点是我们能够很快的找到或改变表格中的值，从而改变策略。

表 9.1.1 例 1

	a_1	a_2	a_3	a_4	a_5
s_1	$\pi(a_1\|s_1)$	$\pi(a_2\|s_1)$	$\pi(a_3\|s_1)$	$\pi(a_4\|s_1)$	$\pi(a_5\|s_1)$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
s_9	$\pi(a_1\|s_9)$	$\pi(a_2\|s_9)$	$\pi(a_3\|s_9)$	$\pi(a_4\|s_9)$	$\pi(a_5\|s_9)$

不过，当状态空间非常大的时候，表格表示法的效率就非常低了。因此，我

们还是需要利用函数进行近似，依旧采用 8 章的非线性神经网络进行函数拟合：

$$\pi(a \mid s, \theta) \tag{252}$$

其中θ是参数向量。

这种方法的优点是，减少了参数的保存，提高了算法效率，另一方面，函数也增强了不同状态之间的联系，提高了泛化性。

与表格法不同的是，首先表格法的最优策略代表它对于每个状态都能够最大化状态价值，而函数法的最优策略代表着它能够最小化一个目标函数。其次，在表格法中获取策略，我们直接根据索引找到对应位置即可，而函数法则需要输入状态，神经网络传播一次，输出每个动作的概率，计算要多一点。最后，表格法更新策略，直接改变表格中的数值即可，而函数法更新策略则需要改变参数。

策略梯度的基本思想过程是：

1) 首先，定义一个目标函数$J(\theta)$，使其能够量化最优策略。
2) 然后，利用基于梯度的优化算法求解最优策略。

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta J(\theta_t) \tag{253}$$

策略梯度的思路看似很简单，但实际进行中会存在很多问题，例如我们该如何量化最优策略？我们怎么计算策略的梯度？下面会一一解答这些问题。

9.2 价值目标函数

定义两类指标。第一类指标是平均状态价值或称为平均价值，特别的，定义指标为：

$$\bar{v}_\pi = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) \tag{254}$$

- $\bar{v}_\pi$是状态价值的权重平均。
- $d(s) \geq 0$每个状态的权重。
- $\sum_{s \in \mathcal{S}} d(s) = 1$，我们也可以认为$d(s)$是状态的概率分布，我们也可以将式 254 写成：

$$\bar{v}_\pi = \mathbb{E}[v_\pi(S)] \tag{255}$$

式 254 的矩阵形式为：

$$\bar{v}_\pi = \sum_{s \in \mathcal{S}} d(s) v_\pi(s) = d^T v_\pi \tag{256}$$

其中：

$$\begin{aligned} v_\pi &= [\ldots, v_\pi(s), \ldots]^T \in \mathbb{R}^{|\mathcal{S}|} \\ d &= [\ldots, d(s), \ldots]^T \in \mathbb{R}^{|\mathcal{S}|} \end{aligned} \tag{257}$$

矩阵形式的表示很方便我们进行接下来的梯度分析。

在式 255 中，d是未知的，我们需要分析如何选择d，这里有两种情况：

1) d与策略是互相独立的。这种情况下，计算梯度非常简单，因为两者分别求梯度就可以，我们特别的定义d为d_0，$\bar{v}_\pi$为$\bar{v}^0{}_\pi$，第一种情况是均匀分布，$d_0 = 1/s$；另一种情况是我们只关心特别的状态s_0，例如我们在网格任务中经常从指定的起点状态出发，我们只关心从起点状态开始的 return，$d_0(s_0) = 1, d_0(s \neq s_0) = 0$。
2) d与策略不独立。这种情况下有一个普遍的定义方法，就是利用策略 π 下的平稳分布。

策略 π 下的平稳分布满足：

$$d_\pi^T P_\pi = d_\pi^T \tag{258}$$

其中P_π为状态转移概率矩阵。这种定义权重的好处在于，如果一个状态出现的频率高，那么代表他重要性强，权重就大，反之，频率低，重要性弱，权重就小。

9.3 奖励目标函数

第二类指标是平均一步奖励或称为平均奖励，特别的，定义指标为：

$$\bar{r}_\pi \doteq \sum_{s\in\mathcal{S}} d_\pi(s) r_\pi(s) = \mathbb{E}[r_\pi(S)] \tag{259}$$

其中：

- $r_\pi(s) \doteq \sum_{a\in\mathcal{A}} \pi(a|s) r(s,a)$代表从当前状态出发，采取每个动作能够得到的奖励值。
- $r(s,a) = \mathbb{E}[R|s,a] = \sum_r rp(r|s,a)$代表从当前状态出发，采取动作 a 能够获得的奖励值。
- 权重d_π为平稳分布，正如指标名称所说，$\bar{r}_\pi$代表了立即奖励的权重平均值。

对于式 259 还有一种等价表示的方式，假设在智能体给定策略下，从 s_0 出发产生了一系列奖励$(R_{t+1}, R_{t+2}, \dots)$，关于这个轨迹的平均一步奖励为：

$$\begin{aligned}&\lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[R_{t+1} + R_{t+2} + \cdots + R_{t+n} \mid S_t = s_0\right]\\&= \lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n} R_{t+k} \mid S_t = s_0\right]\end{aligned} \tag{260}$$

一种重要的形式为：

$$
\begin{aligned}
\lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n}R_{t+k}\mid S_t=s_0\right] &= \lim_{n\to\infty}\frac{1}{n}\mathbb{E}\left[\sum_{k=1}^{n}R_{t+k}\right] \\
&= \sum d_\pi(s)r_\pi(s) \\
&= \bar{r}_\pi
\end{aligned}
\tag{261}
$$

这就是$\bar{r}_\pi$的另外一种表示形式，这种形式在论文中非常常见。

关于价值指标和奖励指标：

1) 这些指标都是策略 π 的函数，策略 π 又由参数 θ 决定，这些指标是 θ 的函数。换句话说，不用的 θ 值能够产生不同的指标值，因此我们的目标就是找到最优的参数 θ 去最大化指标，这就是策略梯度的基本思路。
2) 在这些指标中，我们均考虑存在折扣因子的情况。
3) 直观上看，我们会觉得$\bar{r}_\pi$比较短视，因为它只考虑了单步奖励，$\bar{v}_\pi$比较远视，因为它考虑了整个 return。但实际上，这两个指标是等价的，并且可以互相转换，考虑折扣因子的情况下，有：

$$
\bar{v}_\pi = \bar{r} + \gamma\bar{v}_\pi \rightarrow \bar{r}_\pi = (1-\gamma)\bar{v}_\pi \tag{262}
$$

与贝尔曼公式非常像。

除去以上公式，我们还会在论文中看到这个公式：

$$
J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty}\gamma^t R_{t+1}\right] \tag{263}
$$

首先，我们需要明确和理解这些指标，这个公式对任何状态都成立，状态转移概率就是策略 π 的平稳分布，可以写成：

$$
\begin{aligned}
J(\theta) = \mathbb{E}\left[\sum_{t=0}^{\infty}\gamma^t R_{t+1}\right] &= \sum_{s\in\mathcal{S}} d(s)\mathbb{E}\left[\sum_{t=0}^{\infty}\gamma^t R_{t+1}\mid S_0=s\right] \\
&= \sum_{s\in\mathcal{S}} d(s)v_\pi(s)
\end{aligned}
\tag{264}
$$

9.4 梯度计算

我们已经介绍了目标函数的定义，现在我们只需计算出其梯度，然后利用相关梯度算法求解，即可解决优化问题。但是，如前文所说，这部分是整个策略梯度中最复杂的一部分，因为我们需要区分不同的指标，还要区分带有折扣和不带折扣的情况，所以我们只给出一些简要的介绍。

不同的指标对应的梯度公式大同小异，我们用一种统计表达形式：

$$
\nabla_\theta J(\theta) = \sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a\mid s,\theta)q_\pi(s,a) \tag{265}
$$

其中：

- $J(\theta)$可以是价值目标函数或奖励目标函数。
- “=”可以代表严格相等，近似或成比例。

- η是一个分布或状态的权重。.

一些具体的公式：

$$\begin{aligned}
&\nabla_\theta \bar{r}_\pi \simeq \sum_s d_\pi(s) \sum_a \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) \\
&\nabla_\theta \bar{v}_\pi = \frac{1}{1-\gamma} \nabla_\theta \bar{r}_\pi \\
&\nabla_\theta \bar{v}_\pi^0 = \sum_{s\in\mathcal{S}} \rho_\pi(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a)
\end{aligned} \tag{266}$$

接下来，我们针对式 265 进行具体的解释，其可以写成：

$$\begin{aligned}
\nabla_\theta J(\theta) &= \sum_{s\in\mathcal{S}} \eta(s) \sum_{a\in\mathcal{A}} \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) \\
&= \mathbb{E}\left[\nabla_\theta \ln \pi(A \mid S,\theta) q_\pi(S,A)\right]
\end{aligned} \tag{267}$$

先解释一下为什么要写成这种形式，因为我们在没有模型的时候，求解期望可以用采样来进行近似，这样我就可以用随机梯度上升（下降）的方法进行求解。

$$\nabla_\theta J \approx \nabla_\theta \ln \pi(a \mid s,\theta) q_\pi(s,a) \tag{268}$$

现在，我们证明一下式 267，设$ln\,\pi$为对 π 取自然对数，对其求梯度得：

$$\nabla_\theta \ln \pi(a \mid s,\theta) = \frac{\nabla_\theta \pi(a \mid s,\theta)}{\pi(a \mid s,\theta)} \tag{269}$$

去分母：

$$\nabla_\theta \pi(a \mid s,\theta) = \pi(a \mid s,\theta) \nabla_\theta \ln \pi(a \mid s,\theta) \tag{270}$$

于是我们有：

$$\begin{aligned}
\nabla_\theta J(\theta) &= \sum_s d(s) \sum_a \nabla_\theta \pi(a \mid s,\theta) q_\pi(s,a) \\
&= \sum_s d(s) \sum_a \pi(a \mid s,\theta) \nabla_\theta \ln \pi(a \mid s,\theta) q_\pi(s,a)
\end{aligned} \tag{271}$$

由于 s 是 d 的分布，A 是 π 的分布，我们可以将其求和写成期望的形式（通常情况我们省略了下标）：

$$\begin{aligned}
\nabla_\theta J(\theta) &= \sum_s d(s) \sum_a \pi(a \mid s,\theta) \nabla_\theta \ln \pi(a \mid s,\theta) q_\pi(s,a) \\
&= \mathbb{E}_{S\sim d, A\sim\pi}\left[\nabla_\theta \ln \pi(a \mid s,\theta) q_\pi(s,a)\right] \\
&= \mathbb{E}\left[\nabla_\theta \ln \pi(a \mid s,\theta) q_\pi(s,a)\right]
\end{aligned} \tag{272}$$

对于式 272 有一些说明：

- 因为要求解对数，所以必须要求$\pi(a|s,\theta) > 0$，但实际中某些动作的概率可以为 0，所以用 softmax 对策略进行归一化，例如：

$$z_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}} \tag{273}$$

这样策略函数就可以写成：

$$\pi(a \mid s,\theta)=\frac{e^{h(s,a,\theta)}}{\sum_{a'\in\mathcal{A}} e^{h(s,a',\theta)}} \tag{274}$$

其中是$h(s,a,\theta)$是另一个函数，也就是神经网络拟合的策略函数。

- 如果用神经网络拟合策略函数，网络的输入是状态 s，输出是每个动作的概率，如果保证每个动作的概率都不为 0，这个策略本质上就是随机的并且具有探索性。对于无穷多个动作，这种方法显然就不行了，所以我们会有 DPG（Determine Policy Gradient）方法，在后文会介绍。

9.5 梯度上升算法

现在我们可以介绍第一个策略梯度算法 Reinforce 去寻找最优策略。当我们有了目标函数，首先就可以用梯度上升算法求解：

$$\begin{aligned}\theta_{t+1}&=\theta_t+\alpha\nabla_\theta J(\theta)\\&=\theta_t+\alpha\mathbb{E}\left[\nabla_\theta\ln\pi(A\mid S,\theta_t)q_\pi(S,A)\right]\end{aligned} \tag{275}$$

但式 275 的真实梯度无法求解，我们用随机梯度表示：

$$\theta_{t+1}=\theta_t+\alpha\nabla_\theta\ln\pi(a_t\mid s_t,\theta_t)q_\pi(s_t,a_t) \tag{276}$$

式 276 仍然有关于模型的部分，真实的动作价值无法求解，我们进行近似或采样计算：

$$\theta_{t+1}=\theta_t+\alpha\nabla_\theta\ln\pi(a_t\mid s_t,\theta_t)q_t(s_t,a_t) \tag{277}$$

例如蒙特卡洛的方法，基于 MC 方法的梯度计算叫做 REINFORCE。除了这个 MC 之类，还有 TD 方法等等很多，我们在前文在一个章节进行了专门的介绍。

从采样计算中我们需要解决两个问题，首先是如何进行采样：

$$\mathbb{E}_{S\sim d,A\sim\pi}\left[\nabla_\theta\ln\pi(A\mid S,\theta_t)q_\pi(S,A)\right]\rightarrow\nabla_\theta\ln\pi(a\mid s,\theta_t)q_\pi(s,a) \tag{278}$$

状态 s 是服从于 d 的分布的，而平稳分布需要在策略 π 下经过长时间的运行才能获得，我们显然不能等这么长时间。动作 A 是服从于策略 π 的分布，因此我们需要对在状态 s_t 执行的动作 a_t 进行采样，以上过程的采样的策略和更新的策略是同一个，所以这是一个 On-Policy 的算法。

第二个问题是如何理解这个算法。我们之前定义了对数函数，得到其梯度为：

$$\nabla_\theta\ln\pi(a_t\mid s_t,\theta_t)=\frac{\nabla_\theta\pi(a_t\mid s_t,\theta_t)}{\pi(a_t\mid s_t,\theta_t)} \tag{279}$$

梯度上升算法为：

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta \ln \pi(a_t \mid s_t, \theta_t) q_t(s_t, a_t) \\ &= \theta_t + \alpha \underbrace{\left(\frac{q_t(s_t, a_t)}{\pi(a_t \mid s_t, \theta_t)} \right)}_{\beta_t} \nabla_\theta \pi(a_t \mid s_t, \theta_t)\end{aligned} \tag{280}$$

我们就得到了一个很重要的表示形式：

$$\theta_{t+1} = \theta_t + \alpha \beta_t \nabla_\theta \pi(a_t \mid s_t, \theta_t) \tag{281}$$

这个式子表明我们在对$\pi(a_t|s_t,\theta_t)$进行最大化操作。如果β_t大于 0，那么$\pi(a_t|s_t,\theta_{t+1}) > \pi(a_t|s_t,\theta_t)$，也就是会增强对该动作的选择概率，如果$\beta_t$小于 0，则会减弱对该动作的选择概率。

仔细分析一下β_t的表达式，其实它平衡了探索和利用。首先，如果分子变大，也就是在该状态采取动作得到的动作价值大，导致β_t变大，也就是下一刻对该动作的概率调整就大，这符合我们的利用预期，我们希望动作价值大的动作更容易被选择。其次，如果分母变小，也就是该状态采取该动作的概率小，那么β_t也会变大，下一刻对该动作的概率调整就大，符合我们的利用预期，我们希望概率小的动作同样有机会被选择，从而跳出局部最优。

表 9.5.1 REINFORCE 算法伪代码

伪代码：REINFORCE 算法
Initialization：定义一个参数函数$\pi(a\|s,\theta)$，γ 和 α
Aim：找到最优策略最小化目标函数。
For 每次迭代
选择初始状态并且进行一些回合，获得经验$\{s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T\}$
For t=0,1,2….,T-1
Value update：$q_t(s_t, a_t) = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$
Policy update：$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t\|s_t, \theta_t) q_t(s_t, a_t)$
$\theta_k = \theta_T$

Chapter10 Actor-Critic 方法

10.1 QAC 算法

Actor-Critic 方法是当前强化学习领域应用最广泛、热度最高的方法之一，它融合了值函数近似与策略梯度的核心思路。Actor 负责策略更新（Policy Update），命名源于其主动执行动作的特性，如同演员依照剧本演绎；Critic 负责策略评估（Policy Evaluation）或价值估计（Value Estimation），恰似评论家一般，对 Actor 的策略与动作表现进行价值评判。

回顾一下在上节策略梯度方法的流程：

1) 首先定义一个目标函数$J(\theta)$，可以包括价值目标或奖励目标。

2) 然后给出梯度上升算法最大化目标函数：

$$\begin{aligned}\theta_{t+1} &= \theta_t + \alpha \nabla_\theta J(\theta_t) \\ &= \theta_t + \alpha \mathbb{E}_{S\sim\eta, A\sim\pi}\left[\nabla_\theta \ln \pi(A|S,\theta_t) q_\pi(S,A)\right]\end{aligned} \tag{282}$$

3) 由于确定梯度无法计算，替换为随机梯度上升算法：

$$\theta_{t+1} = \theta_t + \alpha \nabla_\theta \ln \pi(a_t | s_t, \theta_t) q_t(s_t, a_t) \tag{283}$$

在上面的流程中其实已经包含了 Actor 和 Critic，式 283 就是 Actor，因为它在对策略进行更新，式 283 中的$q_t(s_t, a_t)$就是 Critic，因为它利用值函数近似对当前策略下的动作价值进行计算，从而进行评估。

那我们如何去获得$q_t(s_t, a_t)$呢？目前为止，我们有两种方法去进行状态价值估计：

1) MC 方法：如果在式 283 的基础上，使用 MC 进行估计，那么这个算法叫 REINFORCE 或 MC Policy gradient。

2) TD 方法：如果在式 283 的基础上，使用 TD 方法进行估计，这类算法我们都称为 Actor-Critic 方法。

表 10.1.1 QAC 算法伪代码

伪代码：QAC 算法
Aim：找到最优策略最大化目标函数。 At time step *t* in each episode 在策略$\pi(a\|s_t,\theta_t)$下，执行动作a_t，产生观测值，然后继续执行动作a_{t+1}。 Critic（Value update）： $w_{t+1} = w_t + \alpha_w[r_{t+1} + \gamma q(s_{t+1}, a_{t+1}, w_t) - q(s_t, a_t, w_t)]\nabla_w q(s_t, a_t, w_t)$ Actor（Policy update）： $\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \ln \pi(a_t\|s_t,\theta_t) q(s_t, a_t, w_{t+1})$

补充说明：

- 在 QAC 算法中，Critic 是 Sarsa+Value Function approximation。
- Actor 是策略更新算法。
- QAC 算法是 On-policy 的，因为它计算梯度时样本服从当前策略的分布，而更新的策略也是当前策略。
- 尽管这个算法非常简单，但这个算法仍然是 AC 算法中最核心的思路，它能够扩展非常多的算法。

10.2 A2C 算法

现在，我们从 QAC 算法扩展到 Advantage Actor Critic 算法（A2C）,它的核心思想是引入一个基线来减少样本方差，从而提高。

在策略梯度中引入基线不影响梯度计算：

$$\begin{aligned}\nabla_\theta J(\theta) &= \mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta \ln\pi(A|S,\theta_t)q_\pi(S,A)\right] \\ &= \mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta \ln\pi(A|S,\theta_t)(q_\pi(S,A)-b(S))\right]\end{aligned} \tag{284}$$

其中$b(S)$是关于 s 的一个函数。

首先证明一下为什么式 284 成立，首先由式 284 可得：

$$\mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta \ln\pi(A|S,\theta_t)b(S)\right]=0 \tag{285}$$

展开期望得：

$$\mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta \ln\pi(A|S,\theta_t)b(S)\right]=\sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\pi(a|s,\theta_t)\nabla_\theta \ln\pi(a|s,\theta_t)b(s) \tag{286}$$

将 $ln\pi$ 还原成原始梯度形式：

$$\sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\pi(a|s,\theta_t)\nabla_\theta \ln\pi(a|s,\theta_t)b(s)=\sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta_t)b(s) \tag{287}$$

将基线提到前面：

$$\sum_{s\in\mathcal{S}}\eta(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta_t)b(s)=\sum_{s\in\mathcal{S}}\eta(s)b(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta_t) \tag{288}$$

将导数提到求和符号外面，先求和在求导：

$$\sum_{s\in\mathcal{S}}\eta(s)b(s)\sum_{a\in\mathcal{A}}\nabla_\theta\pi(a|s,\theta_t)=\sum_{s\in\mathcal{S}}\eta(s)b(s)\nabla_\theta\sum_{a\in\mathcal{A}}\pi(a|s,\theta_t) \tag{289}$$

已知动作空间内每个动作的概率之和为 1，对常数求导为 0，可得：

$$\mathbb{E}_{S\sim\eta,A\sim\pi}\left[\nabla_\theta \ln\pi(A|S,\theta_t)b(S)\right]=\sum_{s\in\mathcal{S}}\eta(s)b(s)\nabla_\theta 1=0 \tag{290}$$

所以引入基线是不影响策略梯度计算的。

第二，我们解释一下为什么引入基线由有效的。我们令：

$$X(S,A)\doteq\nabla_\theta \ln\pi(A|S,\theta_t)(q_\pi(S,A)-b(S)) \tag{291}$$

目标函数的梯度变为：

$$\nabla_\theta J(\theta)=\mathbb{E}[X] \tag{292}$$

我们给出结论：$b(S)$与$\mathbb{E}[X]$无关，但是$b(S)$与 var(X)有关。下面是证明过程，首先已知：

$$\mathrm{tr}[\mathrm{var}(X)]=\mathbb{E}[X^TX]-\bar{x}^T\bar{x} \tag{293}$$

其中：

$$\begin{aligned}\mathbb{E}[X^TX] &= \mathbb{E}\left[(\nabla_\theta \ln\pi)^T(\nabla_\theta \ln\pi)(q(S,A)-b(S))^2\right] \\ &= \mathbb{E}\left[\|\nabla_\theta \ln\pi\|^2\,(q(S,A)-b(S))^2\right]\end{aligned} \tag{294}$$

显然可以看出，$b(S)$影响着梯度的方差，同时我们可以猜想存在一个最优基线使得方差最小。

所以，我们接下来的目标就是找到一个最优的基线使得 X 的方差最小，例如 sin$x$ 和 5sin$x$ 的采样的均值都为 0，但是一个方差大，一个方差小，我们自然希望样本的方差越小越好。在之前介绍过的 REINFROCE 和 QAC 中没有基线存在，其实可以变相认它们的基线为 0，但 0 不见得是一个好的基线。

我们直接给出最优基线的计算公式：

$$b^*(s)=\frac{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta \ln\pi(A|s,\theta_t)\|^2\, q(s,A)]}{\mathbb{E}_{A\sim\pi}[\|\nabla_\theta \ln\pi(A|s,\theta_t)\|^2]} \tag{295}$$

这个公式太复杂了，以至于没办法去实际应用，我们将权重去掉，使用一个简化版的最优公式：

$$b(s)=\mathbb{E}_{A\sim\pi}[q(s,A)]=v_\pi(s) \tag{296}$$

有了基线，我们重新定义梯度上升算法：

$$\begin{aligned}\theta_{t+1}&=\theta_t+\alpha\mathbb{E}\left[\nabla_\theta \ln\pi(A|S,\theta_t)[q_\pi(S,A)-v_\pi(S)]\right]\\&\doteq\theta_t+\alpha\mathbb{E}\left[\nabla_\theta \ln\pi(A|S,\theta_t)\delta_\pi(S,A)\right]\end{aligned} \tag{297}$$

其中$\delta_\pi(S,A)\doteq q_\pi(S,A)-v_\pi(S)$被称为优势函数（advantage function），$v_\pi(S)$是状态在当前策略的平均动作价值，而这个差值越大，说明这个策略的价值就越大，这就是优势函数名称的来源。

式 297 的随机梯度上升算法是：

$$\begin{aligned}\theta_{t+1}&=\theta_t+\alpha\nabla_\theta \ln\pi(a_t|s_t,\theta_t)[q_t(s_t,a_t)-v_t(s_t)]\\&=\theta_t+\alpha\nabla_\theta \ln\pi(a_t|s_t,\theta_t)\delta_t(s_t,a_t)\end{aligned} \tag{298}$$

我们将其改写成：

$$\begin{aligned}\theta_{t+1}&=\theta_t+\alpha\nabla_\theta \ln\pi(a_t|s_t,\theta_t)\delta_t(s_t,a_t)\\&=\theta_t+\alpha\frac{\nabla_\theta\pi(a_t|s_t,\theta_t)}{\pi(a_t|s_t,\theta_t)}\delta_t(s_t,a_t)\\&=\theta_t+\alpha\underbrace{\left(\frac{\delta_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)}\right)}_{\text{stepsize}}\nabla_\theta\pi(a_t|s_t,\theta_t)\end{aligned} \tag{299}$$

对比一下原始随机梯度上升算法：

$$\begin{aligned}\theta_{t+1}&=\theta_t+\alpha\nabla_\theta \ln\pi(a_t|s_t,\theta_t)q_t(s_t,a_t)\\&=\theta_t+\alpha\underbrace{\left(\frac{q_t(s_t,a_t)}{\pi(a_t|s_t,\theta_t)}\right)}_{\beta_t}\nabla_\theta\pi(a_t|s_t,\theta_t)\end{aligned} \tag{300}$$

我们发现，两个式子不同的是原始式子的分子是动作价值，而 A2C 的式子的分子是相对动作价值，这更符合我们直观的思考，因为动作价值大不代表这个动作相对于其他动作优势大，也许动作价值的范围本身就很大。这个式子分母一

样，总体上看，同样平衡了探索和利用。

而且，优势函数也是 TD 误差的一个近似：

$$\delta_t = q_t(s_t, a_t) - v_t(s_t) \to r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t) \tag{301}$$

这个近似是有理由的因为：

$$\mathbb{E}[q_\pi(S,A) - v_\pi(S) \mid S = s_t, A = a_t] = \mathbb{E}\left[R + \gamma v_\pi(S') - v_\pi(S) \mid S = s_t, A = a_t\right] \tag{302}$$

这样的话，我就不使用两个神经网络了，我只需要用一个神经网络来近似状态价值就可以了。

表 10.2.1 A2C 算法伪代码

伪代码：A2C 算法
Aim：找到最优策略最大化目标函数。 At time step *t* in each episode 在策略$\pi(a\|s_t,\theta_t)$下，执行动作a_t，产生观测值，然后继续执行动作a_{t+1}。 TD error（advantage function）： $\delta_t = r_{t+1} + \gamma v(s_{t+1}, w_t) - v(s_t, w_t)$ Critic（Value update）： $w_{t+1} = w_t + \alpha_w \delta_t \nabla_w v(s_t, w_t)$ Actor（Policy update）： $\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \ln \pi(a_t\|s_t, \theta_t)$

10.3 重要性采样与 Off-Policy

之前我们讲过的所有策略梯度的算法都是 On-policy 的，因为策略梯度的计算与样本的产生都是基于同一个策略。现在，如果我们想复用一些样本经验的话，我们可以将 On-policy 转换为 Off-policy，通过重要性采样（importance sampling），当然重要性采样不仅仅能应用到策略梯度中，只要涉及到计算期望值的都可以使用。

我们用一个简单的例子进行介绍。考虑一个随机变量$X \in \mathcal{X} = \{+1, -1\}$，如果 X 的概率分布为 P_0：

$$p_0(X = +1) = 0.5, \quad p_0(X = -1) = 0.5 \tag{303}$$

X 的期望计算如下：

$$\mathbb{E}_{X \sim p_0}[X] = (+1) \cdot 0.5 + (-1) \cdot 0.5 = 0 \tag{304}$$

现在，我们考虑我不知道模型，或者模型很复杂的时候，我们能否通过采样去计算期望，当然可以。

1) 第一种情况：样本$\{x_i\}$由概率分布 P_0 产生。

$$\mathbb{E}[x_i]=\mathbb{E}[X],\quad \mathrm{var}[x_i]=\mathrm{var}[X] \tag{305}$$

于是，平均值随着样本的数量增加能够收敛到期望：

$$\bar{x}=\frac{1}{n}\sum_{i=1}^{n}x_i\to\mathbb{E}[X],\quad \text{as } n\to\infty \tag{306}$$

因为：

$$\mathbb{E}[\bar{x}]=\mathbb{E}[X],\mathrm{var}[\bar{x}]=\frac{1}{n}\mathrm{var}[X] \tag{307}$$

2) 第二种情况：样本$\{x_i\}$由概率分布 P_1 产生。

$$p_1(X=+1)=0.8,\quad p_1(X=-1)=0.2 \tag{308}$$

期望计算如下：

$$\mathbb{E}_{X\sim p_1}[X]=(+1)\cdot 0.8+(-1)\cdot 0.2=0.6 \tag{309}$$

如果我们使用样本的均值，不出意外的：

$$\bar{x}=\sum_{i=1}^{n}\frac{1}{n}x_i\to\mathbb{E}_{X\sim p_1}[X]=0.6\neq\mathbb{E}_{X\sim p_0}[X] \tag{310}$$

这个时候，我们大胆猜想一下，能不能使用概率分布 P_1 的样本去近似概率分布 P_0 的期望？因为在 Off-policy 中，执行动作的策略和策略改进不是同一个策略，如果能解决上述问题，我们就可以用策略 π_1 的样本去改进策略 π_2。

解决方法就是重要性采样。

我们给出一个式子：

$$\mathbb{E}_{X\sim p_0}[X]=\sum_x p_0(x)x=\sum_x p_1(x)\underbrace{\frac{p_0(x)}{p_1(x)}}_{f(x)}x=\mathbb{E}_{X\sim p_1}[f(X)] \tag{311}$$

现在我们能用概率分布 P_1 的样本去近似概率分布 P_0 的期望了。如何估计$\mathbb{E}_{X\sim p_1}[f(X)]$呢？

$$\bar{f}\doteq\frac{1}{n}\sum_{i=1}^{n}f(x_i),\quad \text{where } x_i\sim p_1 \tag{312}$$

因此：

$$\begin{aligned}\mathbb{E}_{X\sim p_1}[\bar{f}]&=\mathbb{E}_{X\sim p_1}[f(X)]\\ \mathrm{var}_{X\sim p_1}[\bar{f}]&=\frac{1}{n}\mathrm{var}_{X\sim p_1}[f(X)]\end{aligned} \tag{313}$$

因此，$\bar{f}$就是一个对概率分布 P_0 的期望很好的近似，重要性采样公式如下：

$$\mathbb{E}_{X\sim p_0}[X]\approx\bar{f}=\frac{1}{n}\sum_{i=1}^{n}f(x_i)=\frac{1}{n}\sum_{i=1}^{n}\frac{p_0(x_i)}{p_1(x_i)}x_i \tag{314}$$

其中：$\frac{p_0(x_i)}{p_1(x_i)}$称为重要性权重。

- 如果$p_1(x_i) = p_0(x_i)$，重要性为 1，此时$\bar{f}$就等于期望值。
- 如果$p_0(x_i) \geq p_1(x_i)$，说明样本x_i在 P_0 下很好采样，在 P_1 下不好采样，进而样本x_i在 P_1 下的数量较小，如果 P_0 需要用，则需提高其采样重要性，避免 P_0 丢失重要样本。

这里也许有的人会问，既然我计算重要性需要知道 P_0，那我为什么不能直接通过概率分布 P_0 计算期望呢？答案很简单，因为有些情况下，我知道概率分布，但求不了期望值。例如，连续情况下，如何被积函数不满足可积性，无法求期望；或者利用神经网络进行策略梯度计算时，我们只知道概率而不知道具体表达式。

我们现在给出 Off-policy 的策略梯度方法的过程：

1) 假设 β 是行为策略用来产生经验样本。
2) 我们的目标是使用这些样本去更新目标策略 π，使得目标策略能够最大化指标。

$$J(\theta) = \sum_{s \in \mathcal{S}} d_\beta(s) v_\pi(s) = \mathbb{E}_{S \sim d_\beta}[v_\pi(S)] \tag{315}$$

其中d_β是策略 β 下的平稳分布。

3) 目标函数的梯度为：

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \rho, A \sim \beta}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta) q_\pi(S,A)\right] \tag{316}$$

其中$\frac{\pi(A|S,\theta)}{\beta(A|S)}$为重要性权重。

4) 当然，也可以引入基线来减少方差。

$$\nabla_\theta J(\theta) = \mathbb{E}_{S \sim \rho, A \sim \beta}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta)\big(q_\pi(S,A) - b(S)\big)\right] \tag{317}$$

将基线设置为状态价值：

$$\nabla_\theta J(\theta) = \mathbb{E}\left[\frac{\pi(A|S,\theta)}{\beta(A|S)} \nabla_\theta \ln \pi(A|S,\theta)\big(q_\pi(S,A) - v_\pi(S)\big)\right] \tag{318}$$

5) 因为真实梯度无法求解，用随机梯度算法：

$$\theta_{t+1} = \theta_t + \alpha_\theta \frac{\pi(a_t|s_t,\theta_t)}{\beta(a_t|s_t)} \nabla_\theta \ln \pi(a_t|s_t,\theta_t)\big(q_t(s_t,a_t) - v_t(s_t)\big) \tag{319}$$

与 On-policy 方法一致，引入 TD 误差：

$$q_t(s_t,a_t) - v_t(s_t) \approx r_{t+1} + \gamma v_t(s_{t+1}) - v_t(s_t) \doteq \delta_t(s_t,a_t) \tag{320}$$

因此算法变为：

$$\theta_{t+1}=\theta_t+\alpha_\theta\frac{\pi(a_t\mid s_t,\theta_t)}{\beta(a_t\mid s_t)}\nabla_\theta\ln\pi(a_t\mid s_t,\theta_t)\delta_t(s_t,a_t) \tag{321}$$

6) 对式 321 进行稍微的变形：

$$\theta_{t+1}=\theta_t+\alpha_\theta\left(\frac{\delta_t(s_t,a_t)}{\beta(a_t\mid s_t)}\right)\nabla_\theta\pi(a_t\mid s_t,\theta_t) \tag{322}$$

可以看出式 322 与之前算法的形式是很类似的，只是在步长上不太一致，这个公式是纯利用的，因为它的步长分母是固定的。

表 10.3.1 AC with-off on IS 算法伪代码

伪代码：AC with-off on IS 算法
Initialization：一个给定的行为策略$\beta(a\|s)$，一个目标策略$\pi(a\|s,\theta_0)$，一个初始化参数向量θ_0，一个价值函数$v(s,w_0)$的初始化参数向量w_0。 Aim：找到最优策略最大化目标函数。 At time step *t* in each episode 在策略$\beta(a\|s)$下，执行动作a_t，产生观测值，然后继续执行动作a_{t+1}。 TD error（advantage function）： $\delta_t=r_{t+1}+\gamma v(s_{t+1},w_t)-v(s_t,w_t)$ Critic（Value update）： $w_{t+1}=w_t+\alpha_w\frac{\pi(a_t\|s_t,\theta_t)}{\beta(a_t\|s_t)}\delta_t\nabla_w v(s_t,w_t)$ Actor（Policy update）： $\theta_{t+1}=\theta_t+\alpha_\theta\frac{\pi(a_t\|s_t,\theta_t)}{\beta(a_t\|s_t)}\delta_t\nabla_\theta\, ln\,\pi\,(a_t\|s_t,\theta_t)$

10.4 DPG 算法

第十章直到现在，我们介绍的策略都是随机的，并且对于每个状态的策略都满足$\pi(a|s,\theta)>0$。但是在确定性情况下，随机性的公式就不适用了，而且在动作空间为连续的情况下，我们没办法列举出每个动作的概率，基于以上原因我们必须在这些情况下使用确定性策略。

首先我们定义确定性策略的表达式：

$$a=\mu(s,\theta)\doteq\mu(s) \tag{323}$$

因为我们是确定性策略，所以动作和状态是一一对应的，μ可以是神经网络，它的输入是状态，输出是动作（不是动作的概率，而是确定的一个值）。

然后，我们先写出目标函数的表达式：

$$J(\theta)=\mathbb{E}[v_\mu(s)]=\sum_{s\in\mathcal{S}}d_0(s)v_\mu(s) \tag{324}$$

其中，d_0的几种情况与离散一致。

给出目标函数的梯度表达式;

$$\begin{aligned}\nabla_\theta J(\theta) &= \sum_{s\in\mathcal{S}} \rho_\mu(s)\nabla_\theta \mu(s)\left(\nabla_a q_\mu(s,a)\right)|_{a=\mu(s)} \\ &= \mathbb{E}_{S\sim\rho_\mu}\left[\nabla_\theta \mu(S)\left(\nabla_a q_\mu(S,a)\right)|_{a=\mu(S)}\right]\end{aligned} \tag{325}$$

其中，我们能够发现梯度计算中没有了随机变量 A，所以确定性策略是天然的 off-policy，因为它的期望计算没有要求策略，所以用任何一个策略进行采样都可以，这样行为策略与目标策略就不一致了。

接下来，我们用梯度上升的算法进行优化：

$$\theta_{t+1} = \theta_t + \alpha_\theta \mathbb{E}_{S\sim\rho_\mu}\left[\nabla_\theta \mu(S)\left(\nabla_a q_\mu(S,a)\right)|_{a=\mu(S)}\right] \tag{326}$$

由于真实梯度无法求得，采用随机梯度算法：

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu(s_t)\left(\nabla_a q_\mu(s_t,a)\right)|_{a=\mu(s_t)} \tag{327}$$

表 10.4.1 DAC 算法伪代码

伪代码：DAC 算法
Initialization：一个给定的行为策略$\beta(a\|s)$，一个目标策略$\mu(s,\theta_0)$，一个初始化参数向量θ_0，一个价值函数$v(s,w_0)$的初始化参数向量w_0。 Aim：找到最优策略最大化目标函数。 At time step *t* in each episode 在策略$\beta(a\|s)$下，执行动作a_t，产生观测值，然后继续执行动作a_{t+1}。 TD error（advantage function）： $\delta_t = r_{t+1} + \gamma q(s_{t+1}, \mu(s_{t+1},\theta_t), w_t) - q(s_t, a_t, w_t)$ Critic（Value update）： $w_{t+1} = w_t + \alpha_w \delta_t \nabla_w q(s_t, a_t, w_t)$ Actor（Policy update）： $\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu(s_t,\theta_t)(\nabla_a q(s_t, a, w_{t+1}))\|_{a=\mu(s_t)}$

补充说明：

- 这是一个 Off-policy 的算法。
- β可以被$\mu + noise$代替，这在连续性空间中比较常见的探索方式。

表 10.4.2 算法对比

算法	AC	A2C	重要性采样 AC	DPG
核心思想	基础 AC 框架	同步的 AC。使用优势函数替代 TD 误差，减少方差	引入重要性采样，使 AC 框架能够可以复用过去的经验数据，提高样本效率	Actor 输出一个确定的动作，而不是概率分布

算法	AC	A2C	重要性采样 AC	DPG
策略类型	随机性策略	随机性策略	随机性策略	确定性策略
更新方式	在线策略	在线策略	离线策略	离线策略
Critic 评估对象	状态价值或动作价值	优势函数（TD 误差）	状态价值或动作价值	动作价值
Actor 更新目标	最大化期望回报	最大化期望回报，但用优势函数加权	最大化期望回报，但通过重要性采样纠正分布差异	最大化 Q 值
关键公式/技巧	TD 误差	用 n 步回报估计优势函数	重要性采样比率	确定性策略梯度定理
主要优势	1.单步更新，比 MC 快 2.奠定了 AC 的基础	1.使用优势函数，方差更低，收敛更稳定 2.同步并行，效率更高	1.支持离线学习，样本效率高 2.可以结合经验回放	1.在连续动作空间中效率极高 2.无需在高维空间积分，计算更高效
主要劣势/挑战	1.在线策略，样本效率低 2.基于 TD，仍有方差 3.训练不稳定	1.仍是在线策略，样本效率相对离线方法较低 2.需要协调多个环境	1.重要性采样比率方差可能很大，尤其当 π 和 β 相差较大时，训练不稳定	1.探索不足 2.需要额外技巧进行探索
应用场景	离散或低维连续动作空间的问题	作为 AC 的稳定改进版，适用于大多数需要随机策略的场景	需要重用历史数据的任务，是连接在线与离线策略 AC 的重要桥梁	高维连续动作空间控制问题，如机器人控制、自动驾驶